

```
In [1]: import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
import seaborn as sns
sns.set(style="whitegrid")
import matplotlib.pyplot as plt
from collections import Counter
%matplotlib inline
```

```
In [2]: # ignore warnings
import warnings
warnings.filterwarnings('ignore')
```

```
In [3]: fifa=pd.read_csv(r"C:\Users\LIKHITH\OneDrive\Documents\FIFA.csv")
```

```
In [4]: fifa
```

```
Out[4]:
```

	Unnamed: 0	ID	Name	Age	Photo
0	0	158023	L. Messi	31	https://cdn.sofifa.org/players/4/19/158023.png
1	1	20801	Cristiano Ronaldo	33	https://cdn.sofifa.org/players/4/19/20801.png
2	2	190871	Neymar Jr	26	https://cdn.sofifa.org/players/4/19/190871.png
3	3	193080	De Gea	27	https://cdn.sofifa.org/players/4/19/193080.png
4	4	192985	K. De Bruyne	27	https://cdn.sofifa.org/players/4/19/192985.png
...	...	...	...	...	...
18202	18202	238813	J. Lundstram	19	https://cdn.sofifa.org/players/4/19/238813.png
18203	18203	243165	N. Christoffersson	19	https://cdn.sofifa.org/players/4/19/243165.png
18204	18204	241638	B. Worman	16	https://cdn.sofifa.org/players/4/19/241638.png
18205	18205	246268	D. Walker-Rice	17	https://cdn.sofifa.org/players/4/19/246268.png
18206	18206	246269	G. Nugent	16	https://cdn.sofifa.org/players/4/19/246269.png

18207 rows × 89 columns



```
In [5]: fifa.head()
```

```
Out[5]:
```

	Unnamed: 0	ID	Name	Age	Photo	Nationality
0	0	158023	L. Messi	31	https://cdn.sofifa.org/players/4/19/158023.png	Argentina
1	1	20801	Cristiano Ronaldo	33	https://cdn.sofifa.org/players/4/19/20801.png	Portugal
2	2	190871	Neymar Jr	26	https://cdn.sofifa.org/players/4/19/190871.png	Brazil
3	3	193080	De Gea	27	https://cdn.sofifa.org/players/4/19/193080.png	Spain
4	4	192985	K. De Bruyne	27	https://cdn.sofifa.org/players/4/19/192985.png	Belgium

5 rows × 89 columns



```
In [6]: fifa.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 18207 entries, 0 to 18206
```

```
Data columns (total 89 columns):
```

#	Column	Non-Null Count	Dtype
0	Unnamed: 0	18207 non-null	int64
1	ID	18207 non-null	int64
2	Name	18207 non-null	object
3	Age	18207 non-null	int64
4	Photo	18207 non-null	object
5	Nationality	18207 non-null	object
6	Flag	18207 non-null	object
7	Overall	18207 non-null	int64
8	Potential	18207 non-null	int64
9	Club	17966 non-null	object
10	Club Logo	18207 non-null	object
11	Value	18207 non-null	object
12	Wage	18207 non-null	object
13	Special	18207 non-null	int64
14	Preferred Foot	18159 non-null	object
15	International Reputation	18159 non-null	float64
16	Weak Foot	18159 non-null	float64
17	Skill Moves	18159 non-null	float64
18	Work Rate	18159 non-null	object
19	Body Type	18159 non-null	object
20	Real Face	18159 non-null	object
21	Position	18147 non-null	object
22	Jersey Number	18147 non-null	float64
23	Joined	16654 non-null	object
24	Loaned From	1264 non-null	object
25	Contract Valid Until	17918 non-null	object
26	Height	18159 non-null	object
27	Weight	18159 non-null	object
28	LS	16122 non-null	object
29	ST	16122 non-null	object
30	RS	16122 non-null	object
31	LW	16122 non-null	object
32	LF	16122 non-null	object
33	CF	16122 non-null	object
34	RF	16122 non-null	object
35	RW	16122 non-null	object
36	LAM	16122 non-null	object
37	CAM	16122 non-null	object
38	RAM	16122 non-null	object
39	LM	16122 non-null	object
40	LCM	16122 non-null	object
41	CM	16122 non-null	object
42	RCM	16122 non-null	object
43	RM	16122 non-null	object
44	LWB	16122 non-null	object
45	LDM	16122 non-null	object
46	CDM	16122 non-null	object
47	RDM	16122 non-null	object
48	RWB	16122 non-null	object
49	LB	16122 non-null	object
50	LCB	16122 non-null	object

```

51 CB 16122 non-null object
52 RCB 16122 non-null object
53 RB 16122 non-null object
54 Crossing 18159 non-null float64
55 Finishing 18159 non-null float64
56 HeadingAccuracy 18159 non-null float64
57 ShortPassing 18159 non-null float64
58 Volleys 18159 non-null float64
59 Dribbling 18159 non-null float64
60 Curve 18159 non-null float64
61 FKAccuracy 18159 non-null float64
62 LongPassing 18159 non-null float64
63 BallControl 18159 non-null float64
64 Acceleration 18159 non-null float64
65 SprintSpeed 18159 non-null float64
66 Agility 18159 non-null float64
67 Reactions 18159 non-null float64
68 Balance 18159 non-null float64
69 ShotPower 18159 non-null float64
70 Jumping 18159 non-null float64
71 Stamina 18159 non-null float64
72 Strength 18159 non-null float64
73 LongShots 18159 non-null float64
74 Aggression 18159 non-null float64
75 Interceptions 18159 non-null float64
76 Positioning 18159 non-null float64
77 Vision 18159 non-null float64
78 Penalties 18159 non-null float64
79 Composure 18159 non-null float64
80 Marking 18159 non-null float64
81 StandingTackle 18159 non-null float64
82 SlidingTackle 18159 non-null float64
83 GKDiving 18159 non-null float64
84 GKHandling 18159 non-null float64
85 GKKicking 18159 non-null float64
86 GKPositioning 18159 non-null float64
87 GKReflexes 18159 non-null float64
88 Release Clause 16643 non-null object

```

dtypes: float64(38), int64(6), object(45)

memory usage: 12.4+ MB

```
In [7]: fifa['Body Type'].value_counts()
```

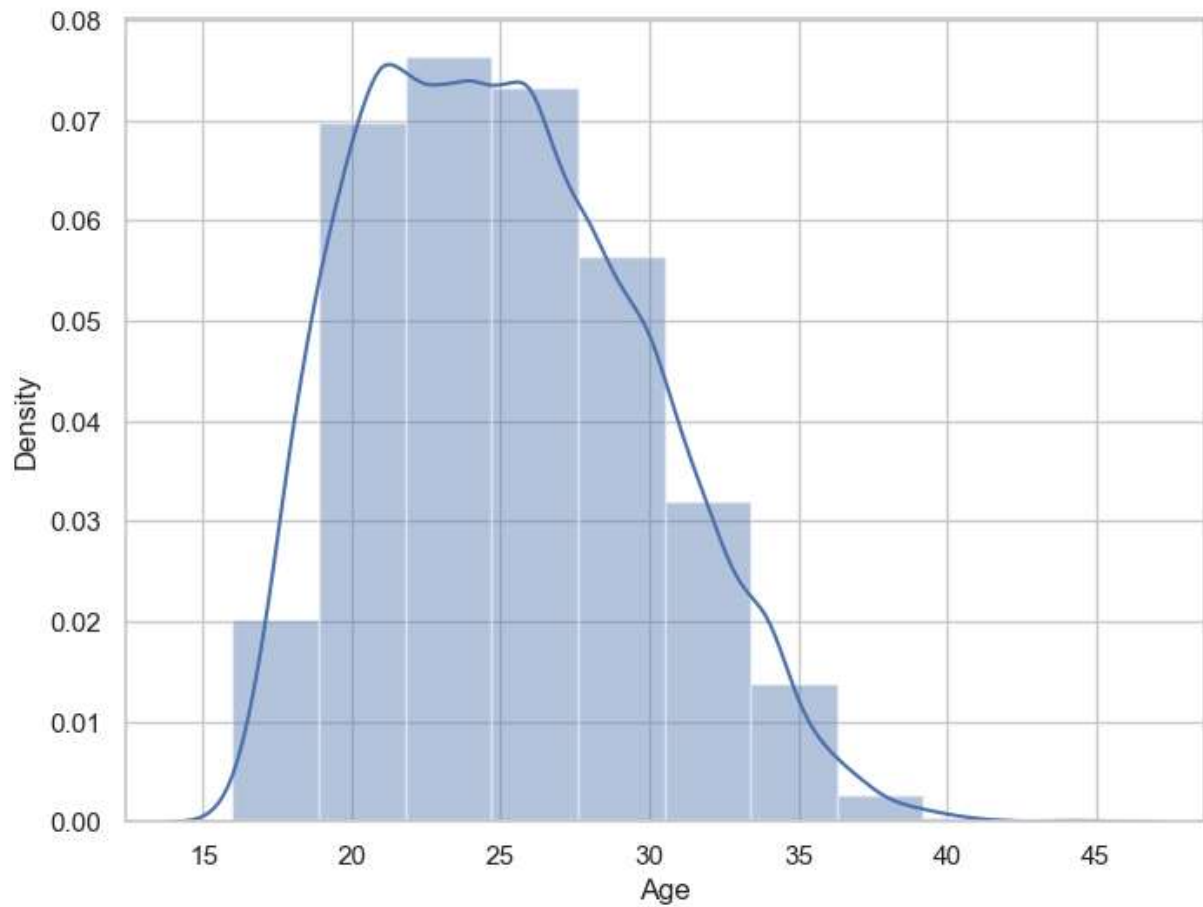
```

Out[7]: Body Type
Normal      10595
Lean         6417
Stocky      1140
Messi         1
C. Ronaldo   1
Neymar        1
Courtois      1
PLAYER_BODY_TYPE_25  1
Shaqiri        1
Akinfenwa      1
Name: count, dtype: int64

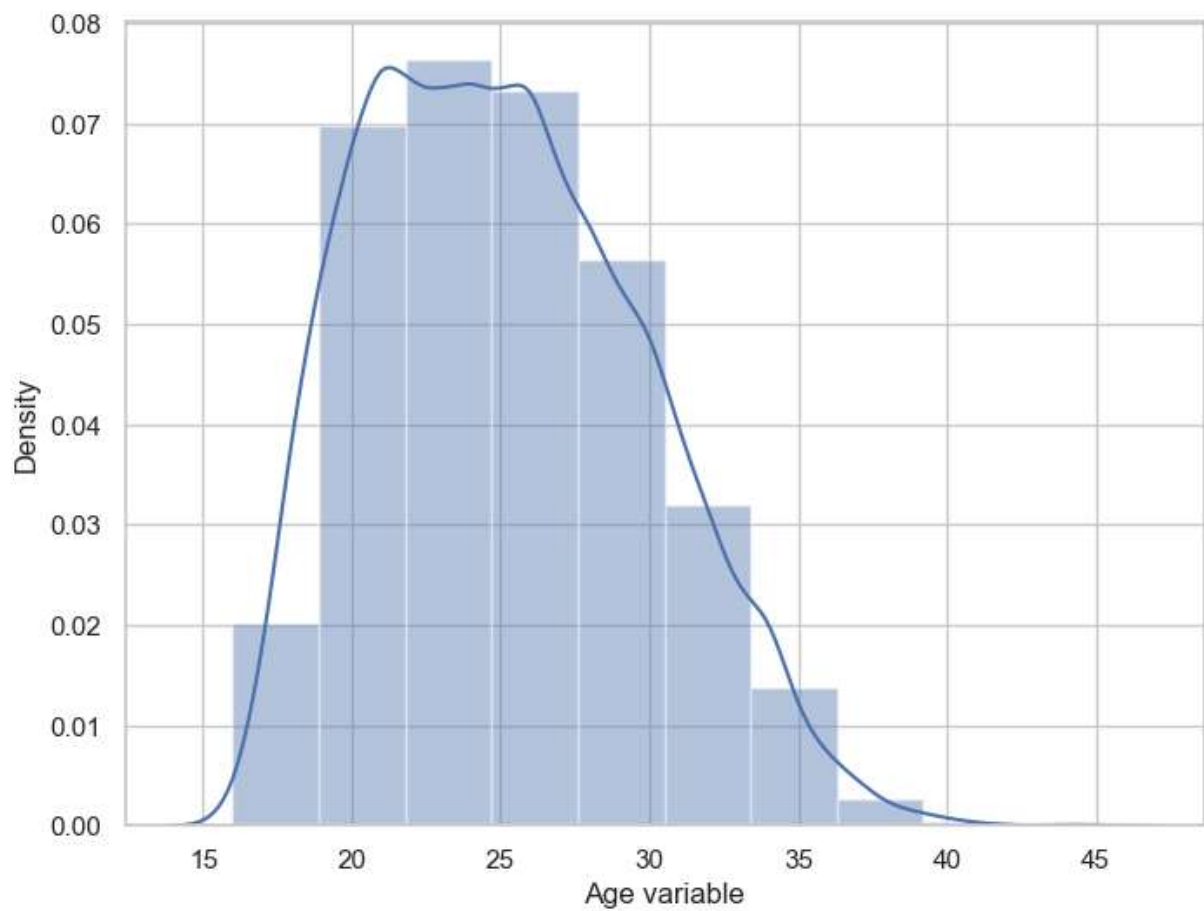
```



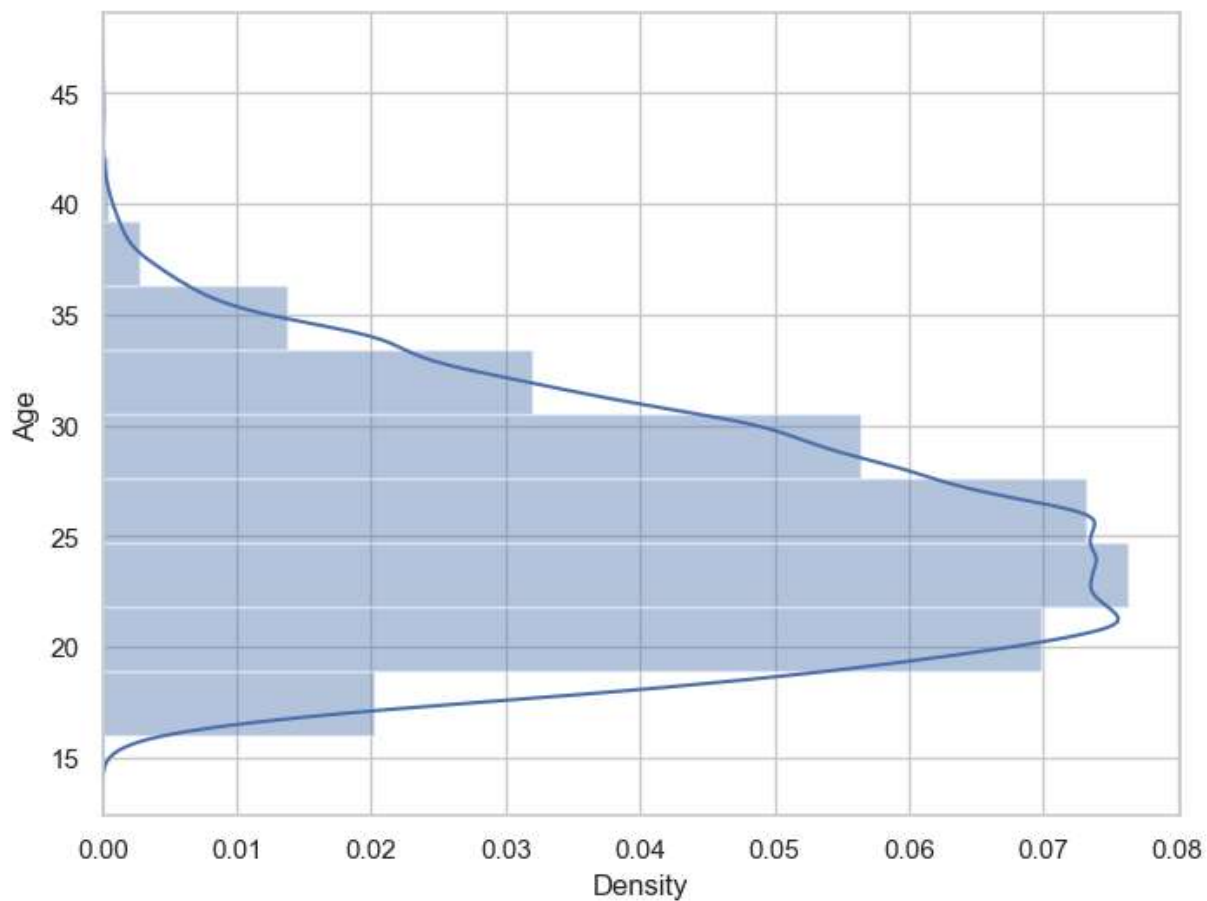
```
In [8]: f, ax = plt.subplots(figsize=(8,6))
x = fifa['Age']
ax = sns.distplot(x, bins=10)
plt.show()
```



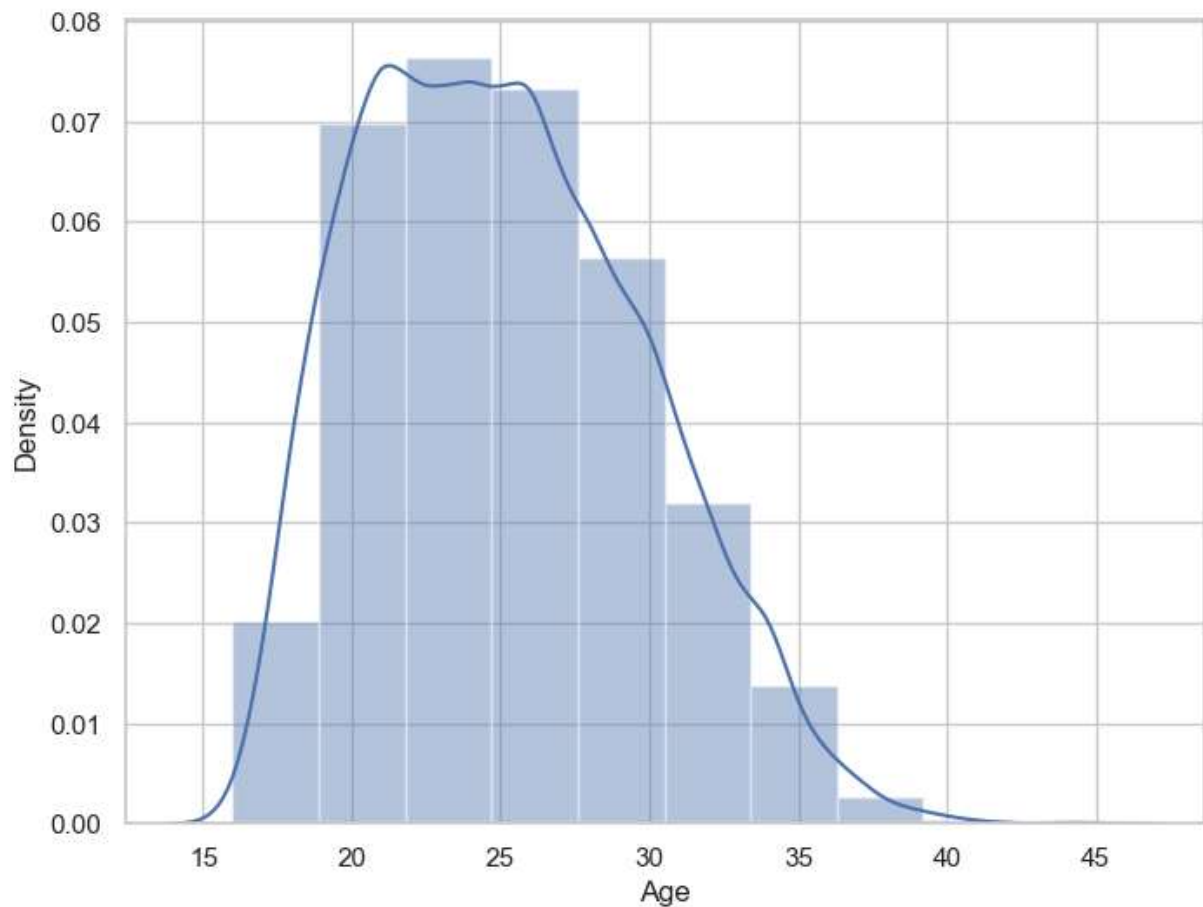
```
In [9]: f, ax = plt.subplots(figsize=(8,6))
x = fifa['Age']
x = pd.Series(x, name="Age variable")
ax = sns.distplot(x, bins=10)
plt.show()
```



```
In [10]: f, ax = plt.subplots(figsize=(8,6))
x = fifa['Age']
ax = sns.distplot(x, bins=10, vertical = True)
plt.show()
```



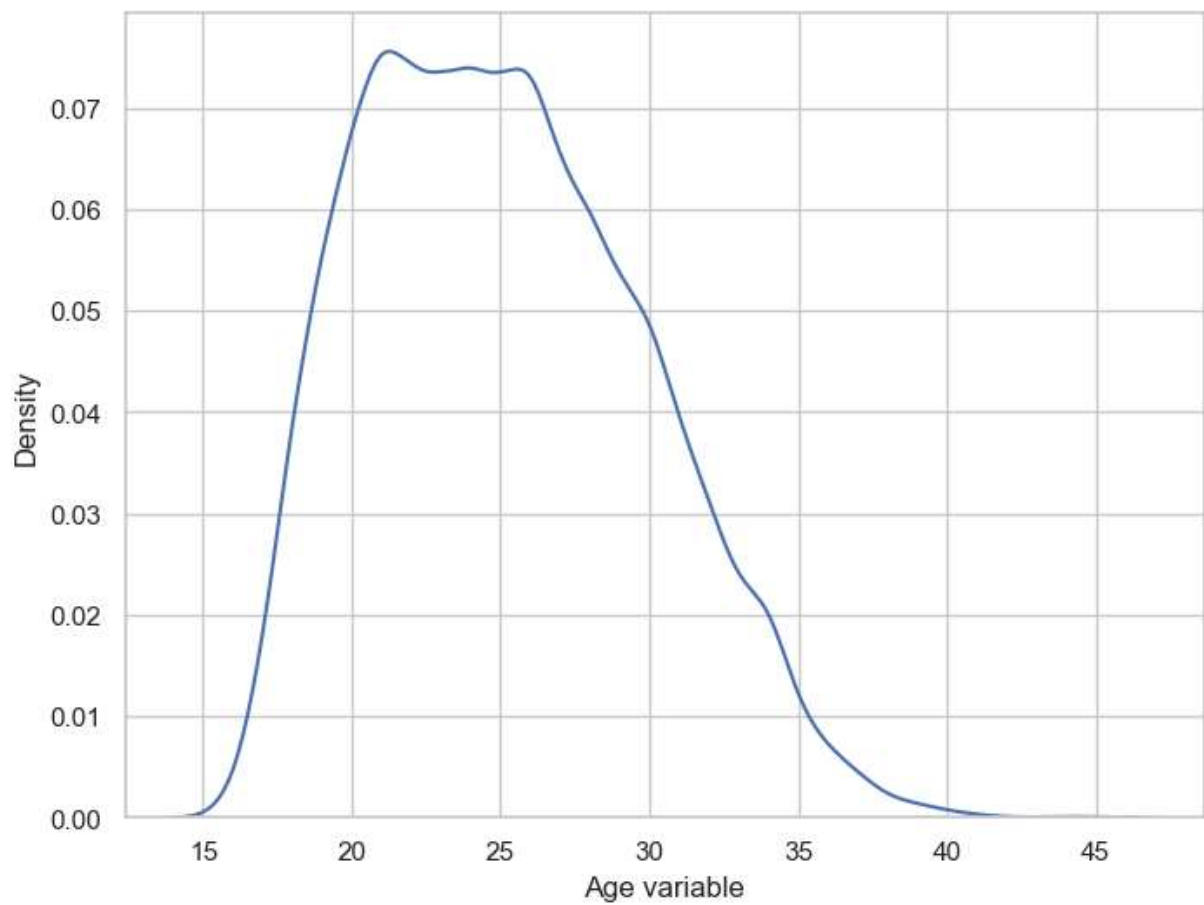
```
In [11]: f, ax = plt.subplots(figsize=(8,6))
x = fifa['Age']
ax = sns.distplot(x, bins=10, vertical = False)
plt.show()
```



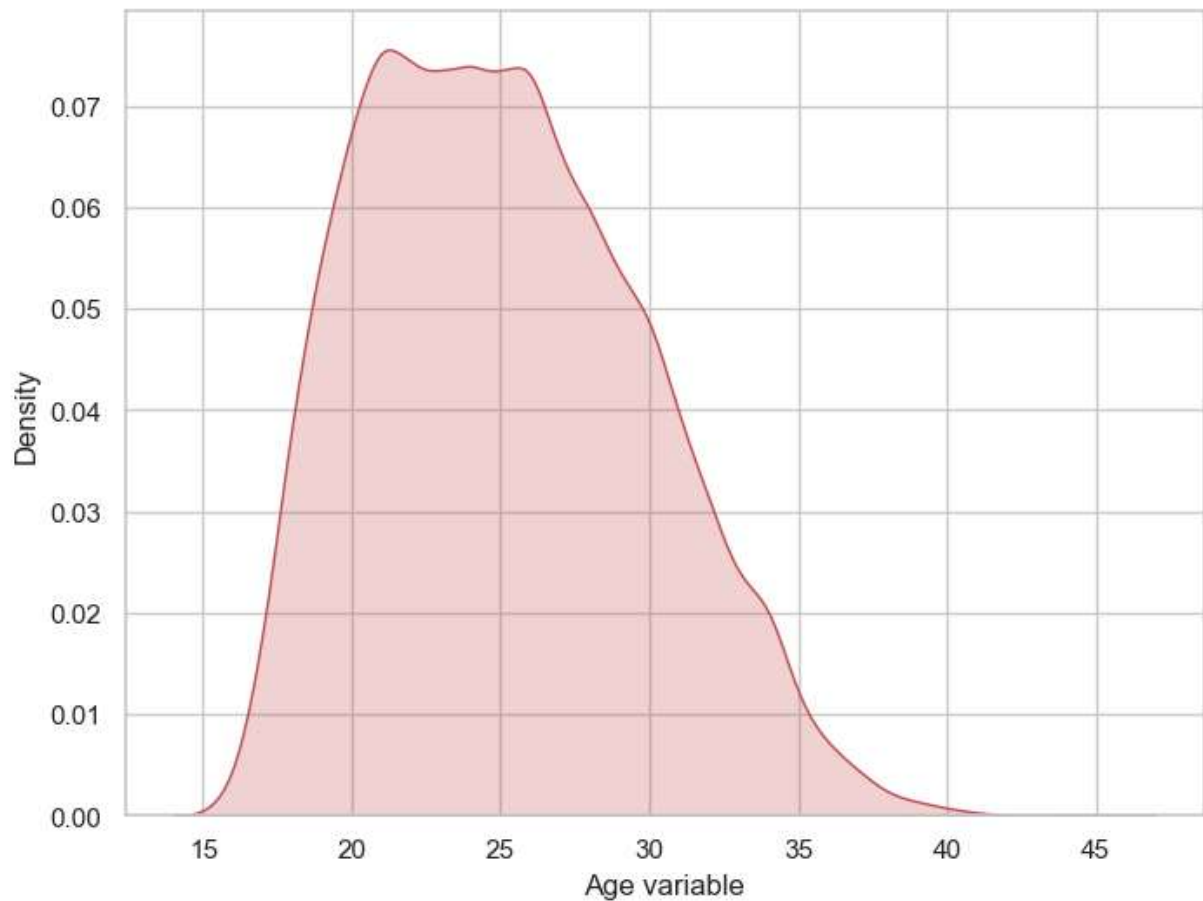
seaborn kernel density estimation(kde) plot

- The kernel density estimate (KDE) plot is a useful tool for plotting the shape of a distribution.
- Seaborn kdeplot is another seaborn plotting function that fits and plot a univariate or bivariate kernel density estimate.
- Like the histogram, the KDE plots encode the density of observations on one axis with height along the other axis.

```
In [12]: f, ax = plt.subplots(figsize=(8,6))
x = fifa['Age']
x = pd.Series(x, name="Age variable")
ax = sns.kdeplot(x)
plt.show()
```



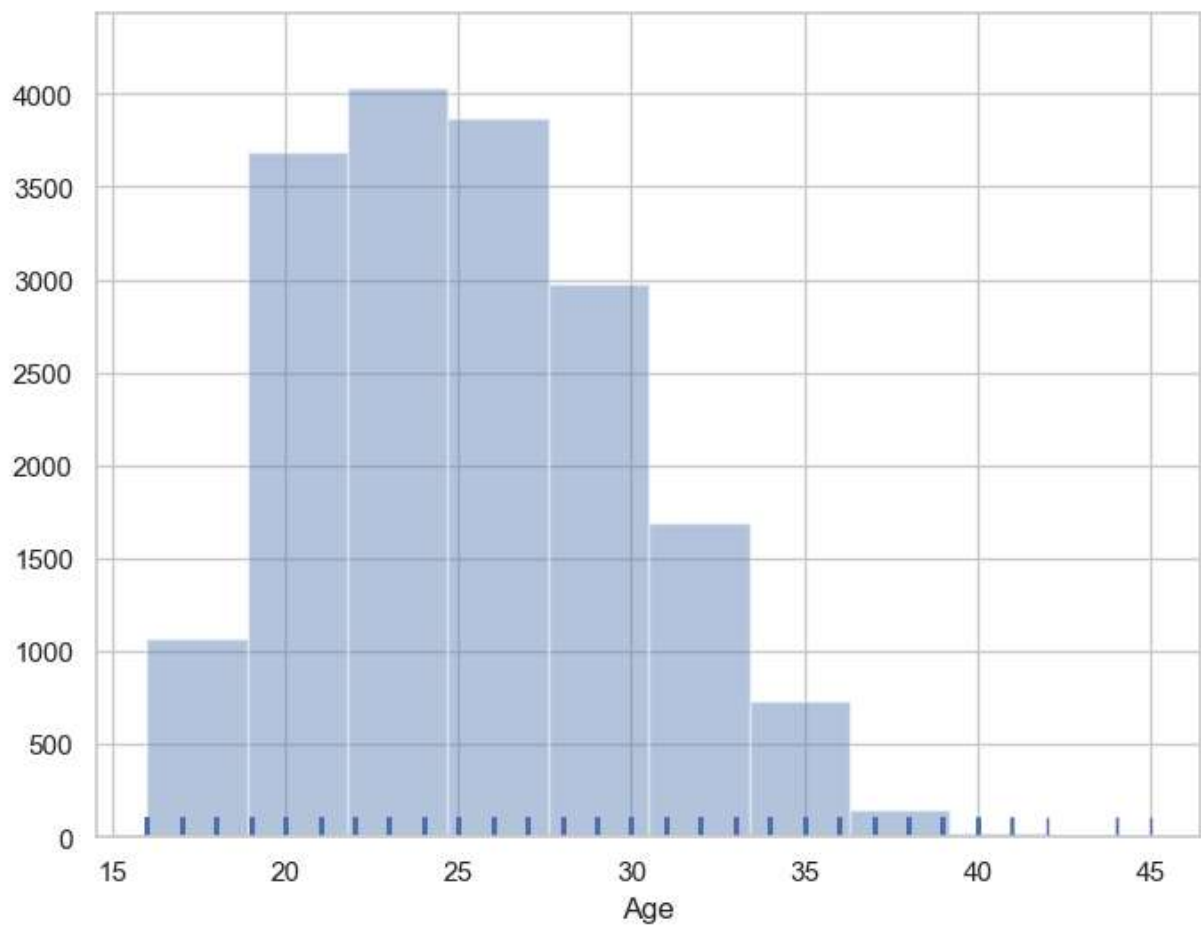
```
In [13]: f, ax = plt.subplots(figsize=(8,6))
x = fifa['Age']
x = pd.Series(x, name="Age variable")
ax = sns.kdeplot(x, shade=True, color='r')
plt.show()
```



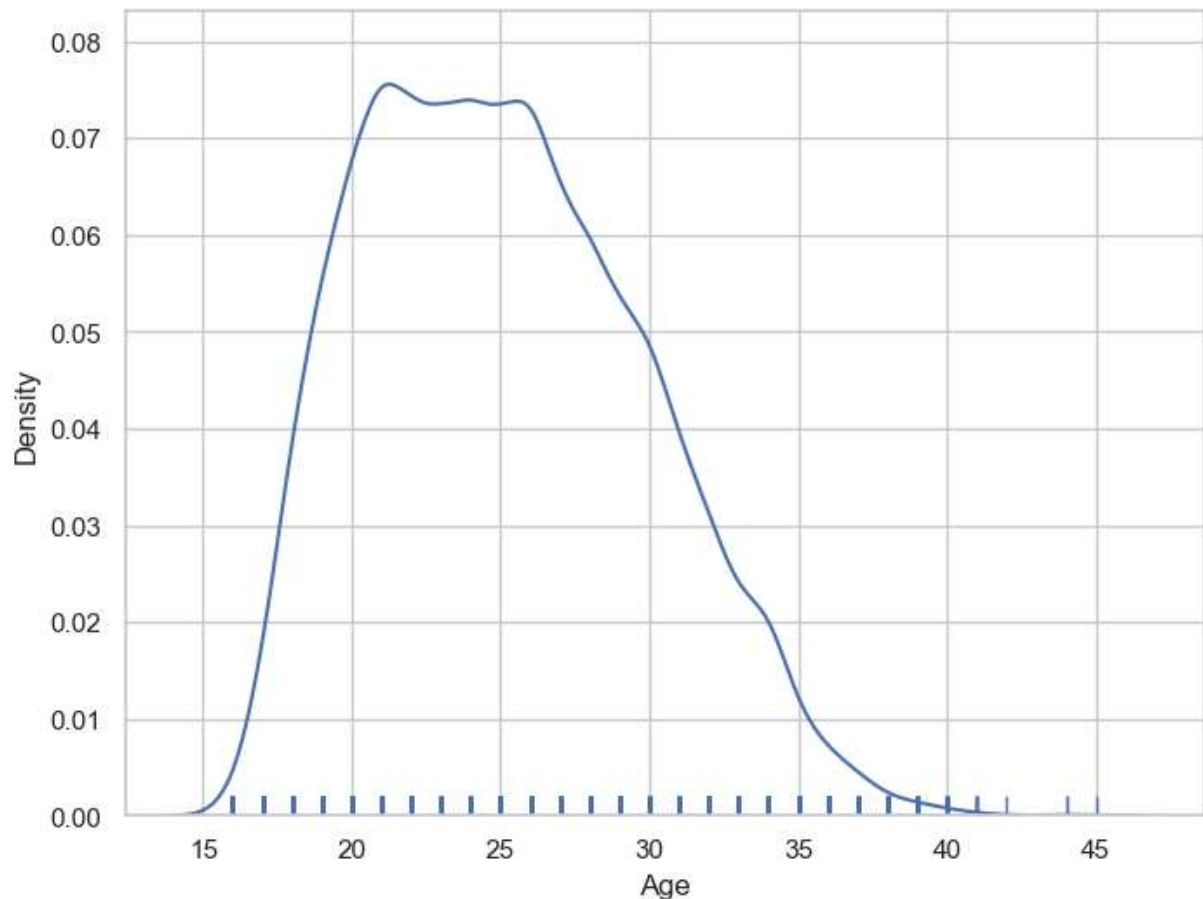
Histograms

- A histogram represents the distribution of data by forming bins along the range of the data and then drawing bars to show the number of observations that fall in each bin.
- A `hist()` function already exists in matplotlib.

```
In [14]: f, ax = plt.subplots(figsize=(8,6))
x = fifa['Age']
ax = sns.distplot(x, kde=False, rug=True, bins=10)
plt.show()
```



```
In [15]: f, ax = plt.subplots(figsize=(8,6))
x = fifa['Age']
ax = sns.distplot(x, hist=False, rug=True, bins=10)
plt.show()
```



explore preferred foot variable

```
In [16]: fifa['Preferred Foot'].nunique()
```

```
Out[16]: 2
```

```
In [17]: fifa['Preferred Foot'].value_counts()
```

```
Out[17]: Preferred Foot
Right    13948
Left     4211
Name: count, dtype: int64
```

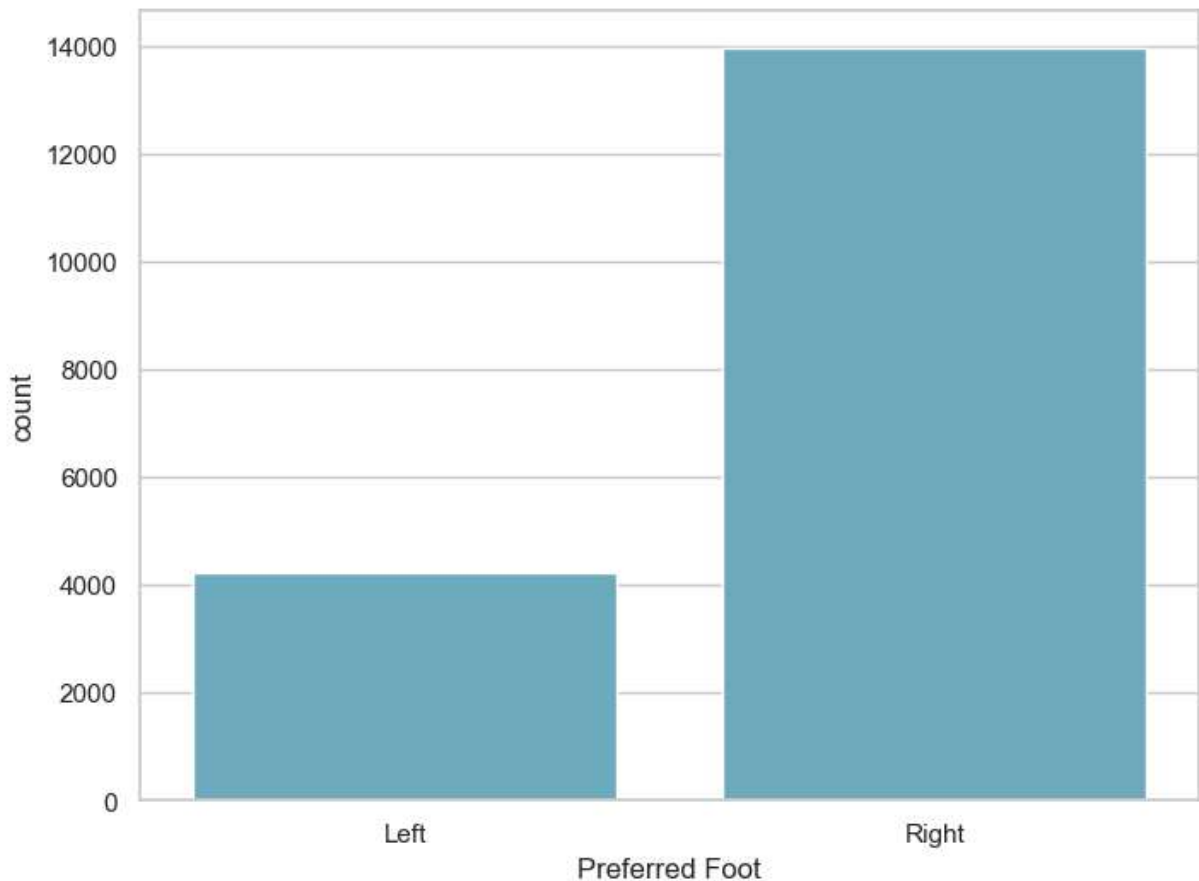
The Preferred Foot variable contains two types of values - Right and Left.

Visualize distribution of values with Seaborn countplot() function.

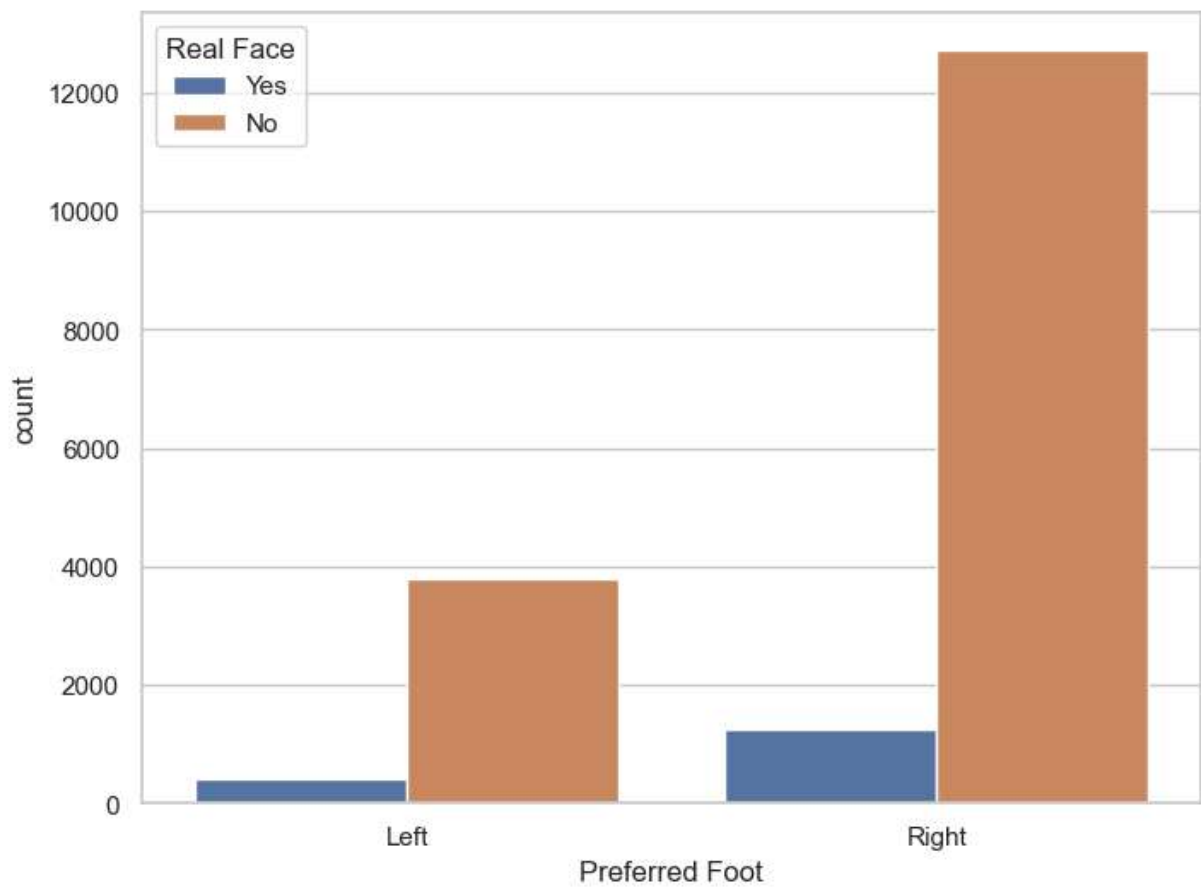
- A countplot shows the counts of observations in each categorical bin using bars.

- It can be thought of as a histogram across a categorical, instead of quantitative, variable.
 - This function always treats one of the variables as categorical and draws data at ordinal positions (0, 1, ... n) on the relevant axis, even when the data has a numeric or date type.
1. • We can visualize the distribution of values with Seaborn `countplot()` function as follows-

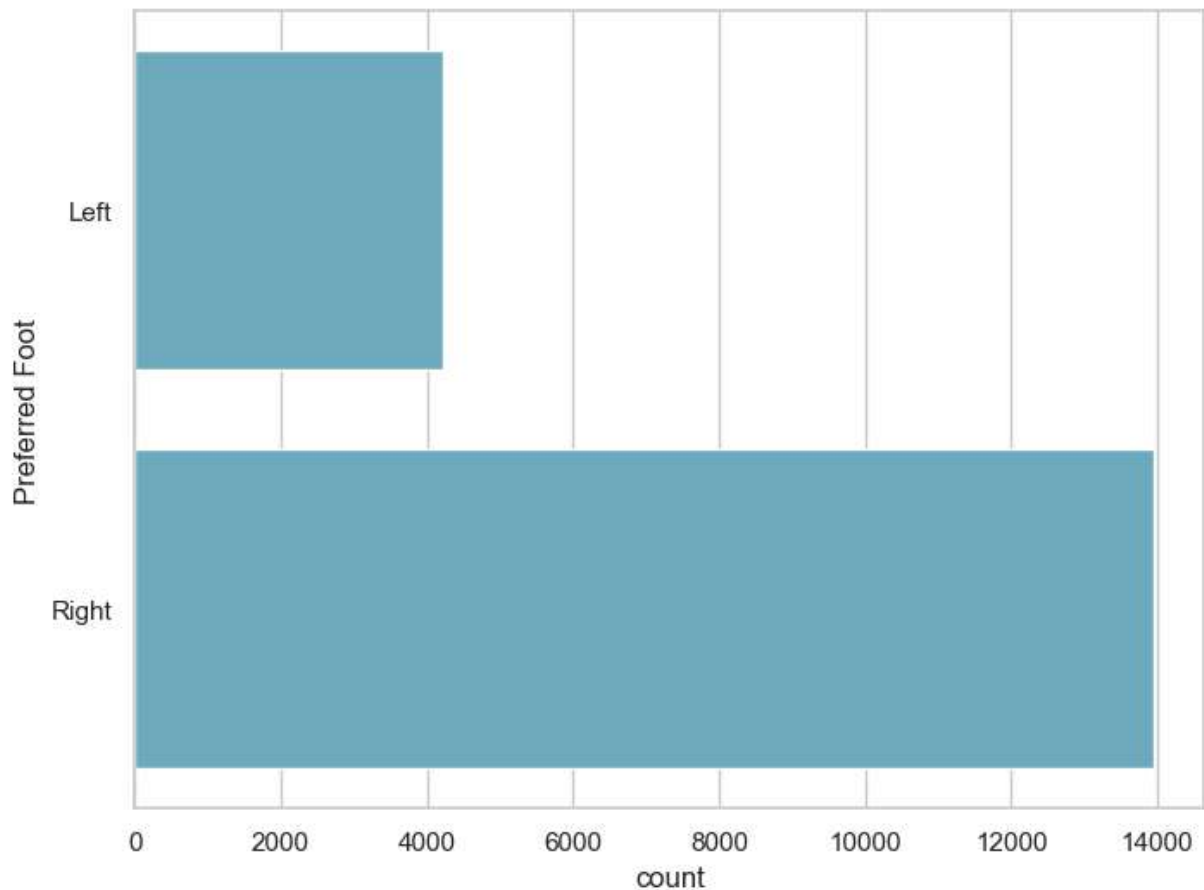
```
In [18]: f, ax = plt.subplots(figsize=(8, 6))
sns.countplot(x="Preferred Foot", data=fifa, color="c")
plt.show()
```



```
In [19]: f, ax = plt.subplots(figsize=(8, 6))
sns.countplot(x="Preferred Foot", hue="Real Face", data=fifa)
plt.show()
```



```
In [20]: f, ax = plt.subplots(figsize=(8, 6))  
sns.countplot(y="Preferred Foot", data=fifa, color="c")  
plt.show()
```

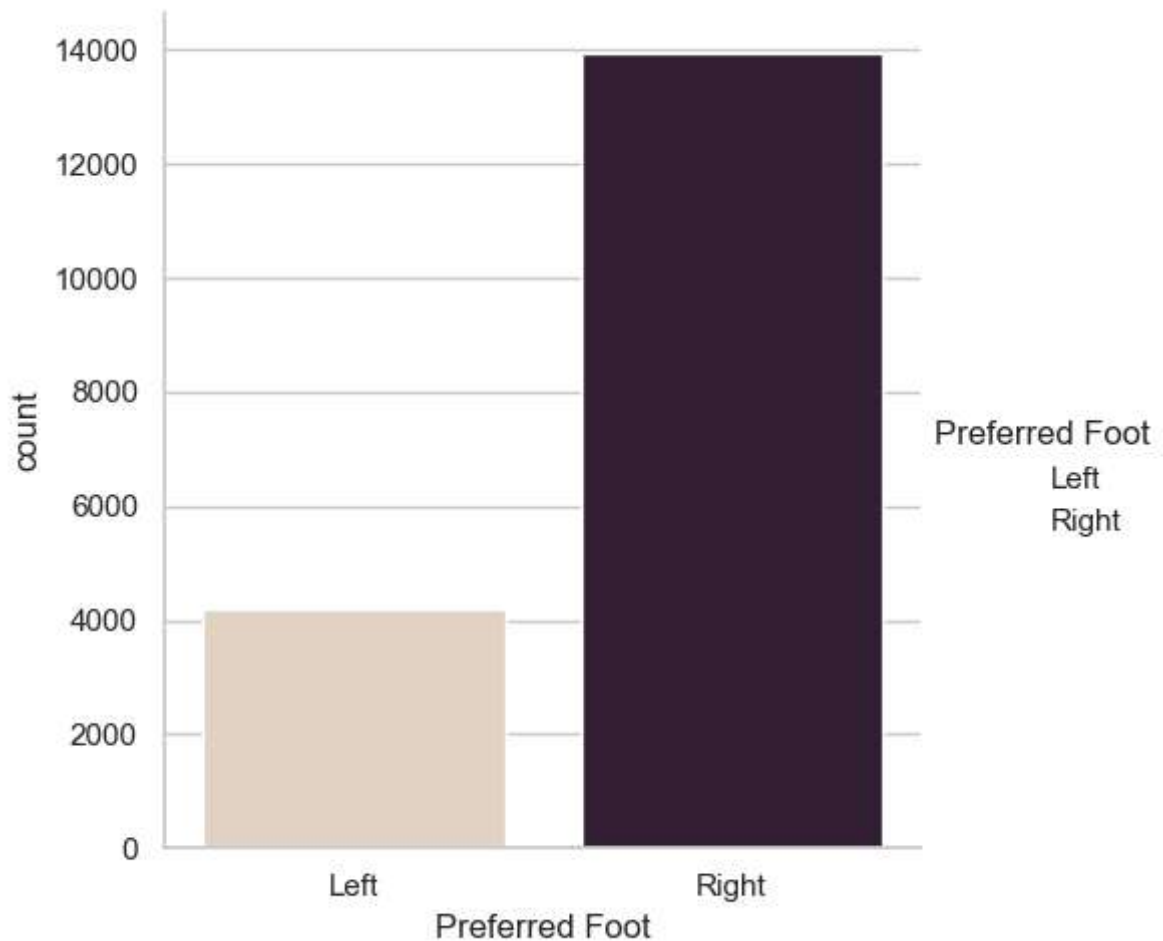


Seaborn Catplot() function

- We can use Seaborn `Catplot()` function to plot categorical scatterplots.
- The default representation of the data in `catplot()` uses a scatterplot.
- It helps to draw figure-level interface for drawing categorical plots onto a `facetGrid`.
- This function provides access to several axes-level functions that show the relationship between a numerical and one or more categorical variables using one of several visual representations.
- The `kind` parameter selects the underlying axes-level function to use.

```
In [21]: g = sns.catplot(x="Preferred Foot", kind="count", palette="ch:.25", data=fifa)
```

```
In [22]: plt.show()
```



Explore International Reputation variable

Check the number of unique values in International Reputation variable

```
In [23]: fifa['International Reputation'].nunique()
```

```
Out[23]: 5
```

Check the distribution of values in International Reputation variable

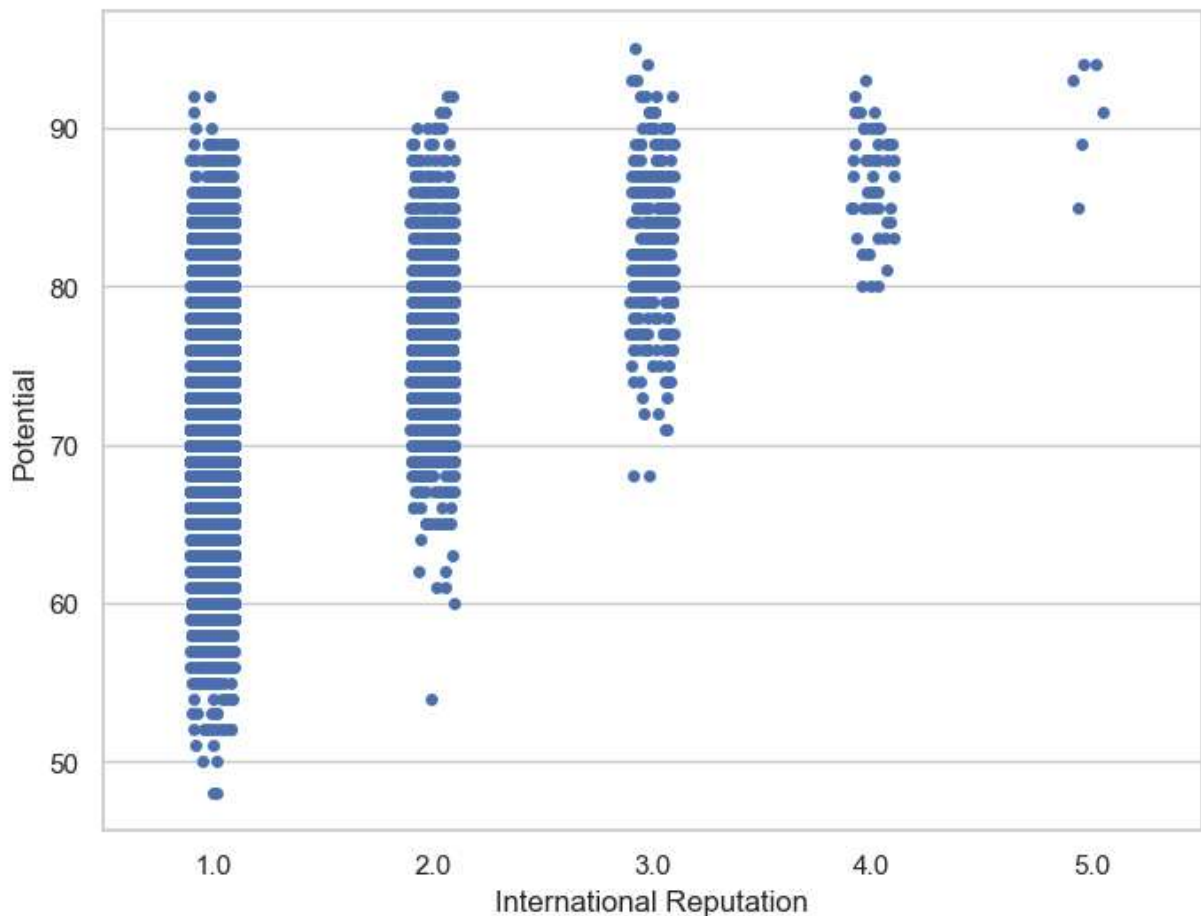
```
In [24]: fifa['International Reputation'].value_counts()
```

```
Out[24]: International Reputation
1.0      16532
2.0       1261
3.0        309
4.0         51
5.0          6
Name: count, dtype: int64
```

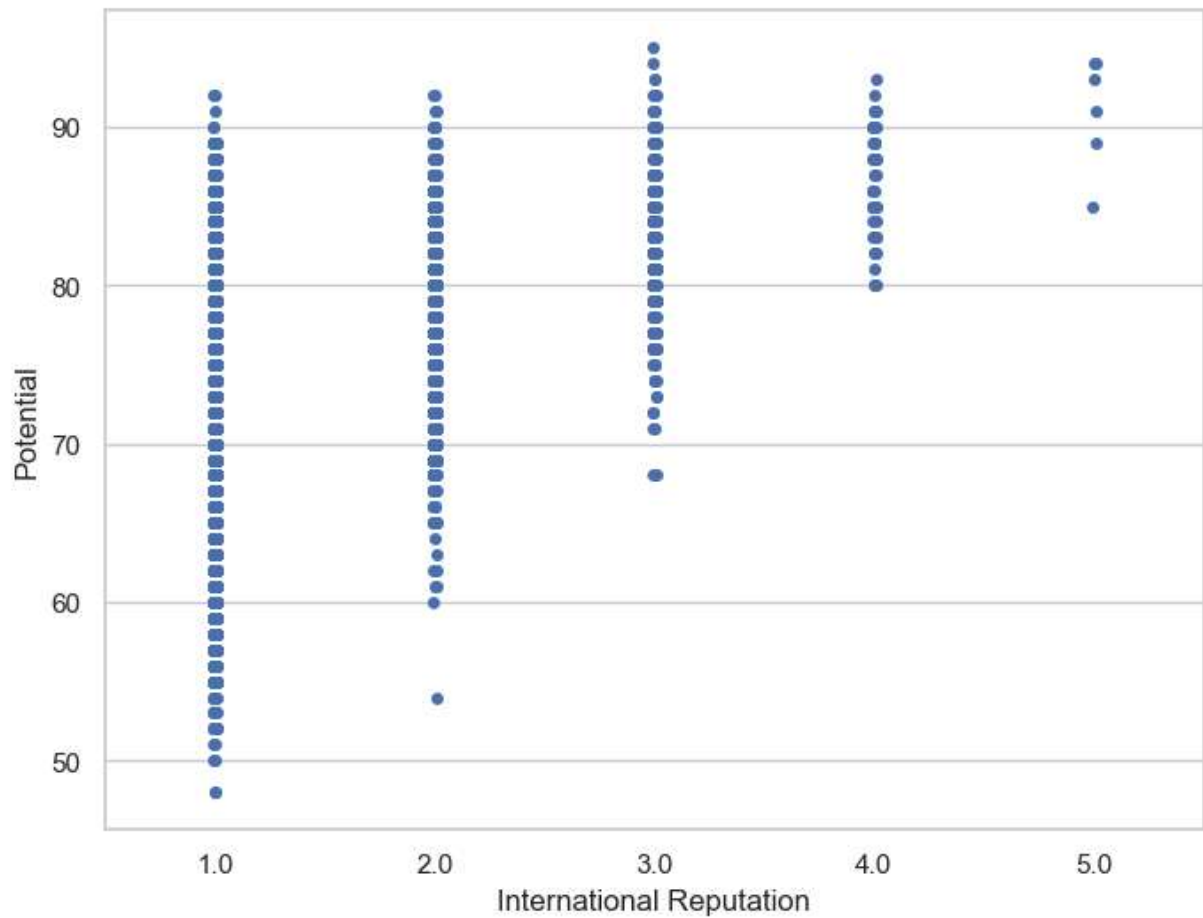
Seaborn Stripplot() function

- This function draws a scatterplot where one variable is categorical.
- A strip plot can be drawn on its own, but it is also a good complement to a box or violin plot in cases where we want to show all observations along with some representation of the underlying distribution.
- I will plot a stripplot with `International Reputation` as categorical variable and `Potential` as the other variable.

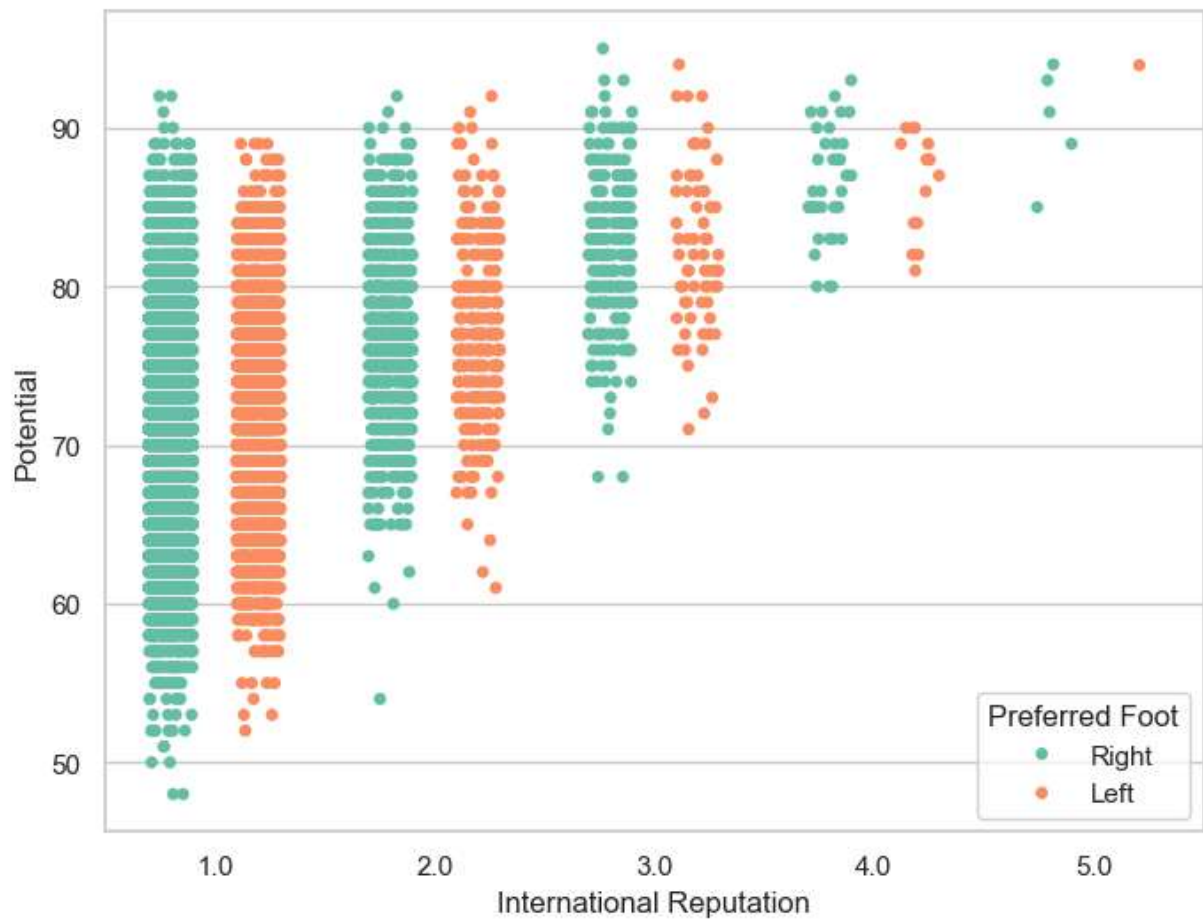
```
In [25]: f, ax = plt.subplots(figsize=(8, 6))
sns.stripplot(x="International Reputation", y="Potential", data=fifa)
plt.show()
```



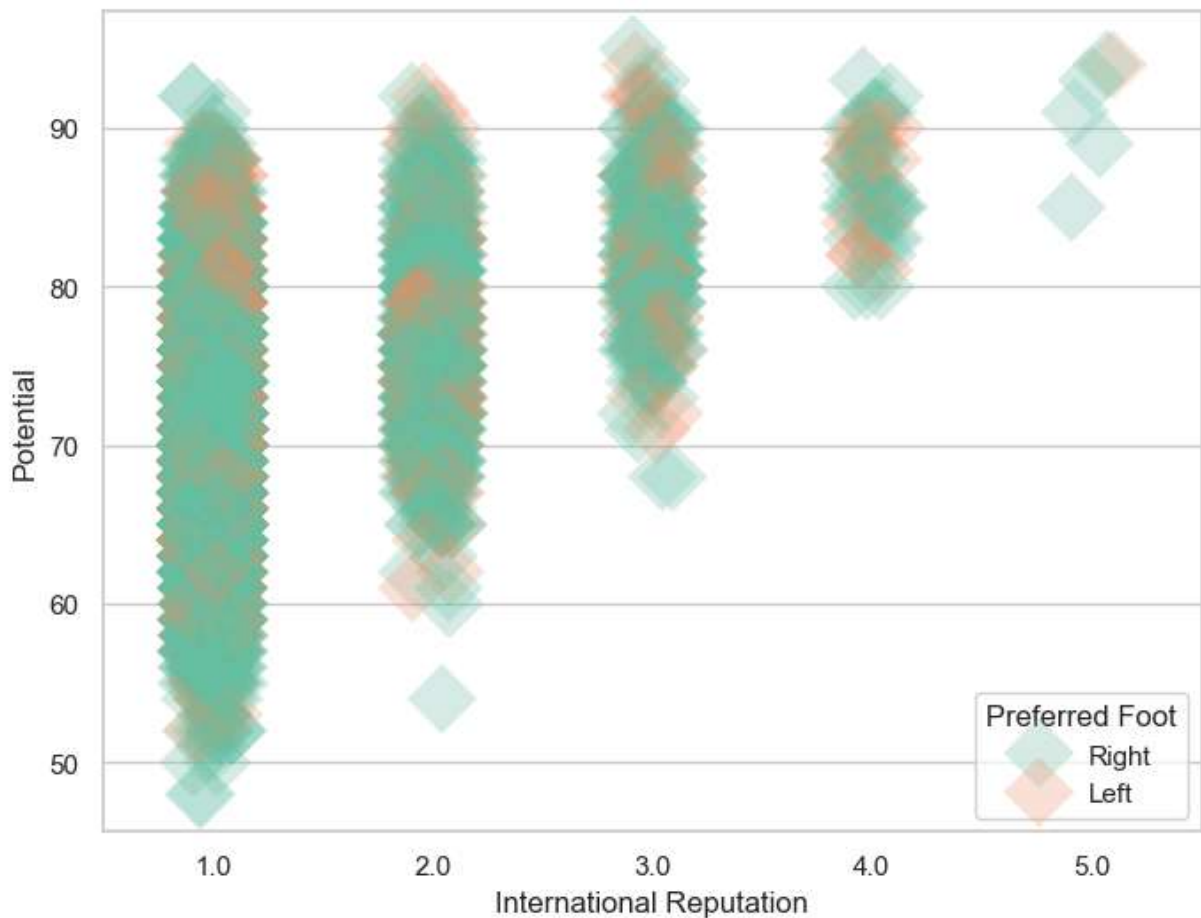
```
In [26]: f, ax = plt.subplots(figsize=(8, 6))  
sns.stripplot(x="International Reputation", y="Potential", data=fifa, jitter=0.01)  
plt.show()
```



```
In [27]: f, ax = plt.subplots(figsize=(8, 6))  
sns.stripplot(x="International Reputation", y="Potential", hue="Preferred Foot",  
              data=fifa, jitter=0.2, palette="Set2", dodge=True)  
plt.show()
```



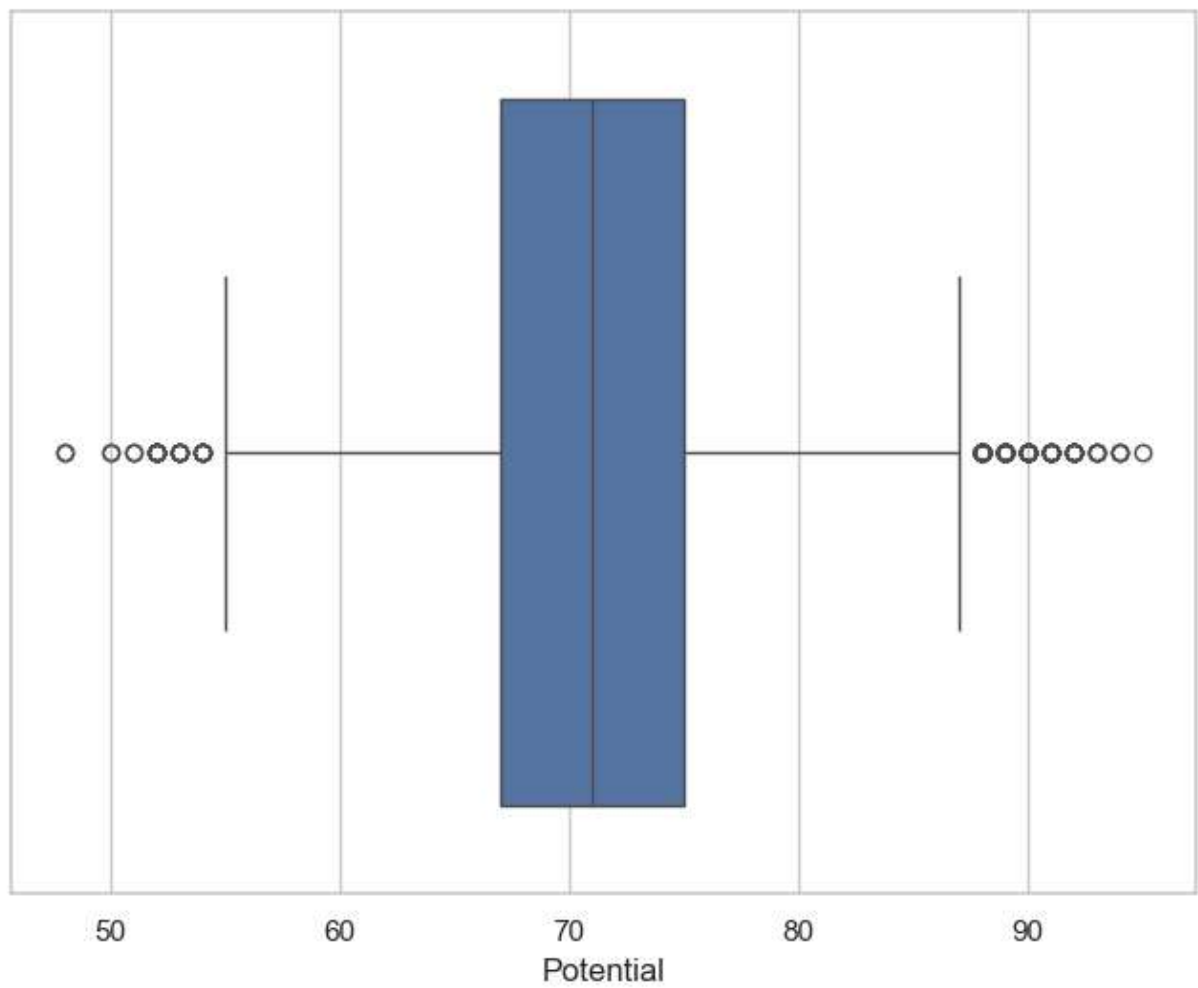
```
In [28]: f, ax = plt.subplots(figsize=(8, 6))
sns.stripplot(x="International Reputation", y="Potential", hue="Preferred Foot",
              data=fifa, palette="Set2", size=20, marker="D",
              edgecolor="gray", alpha=.25)
plt.show()
```



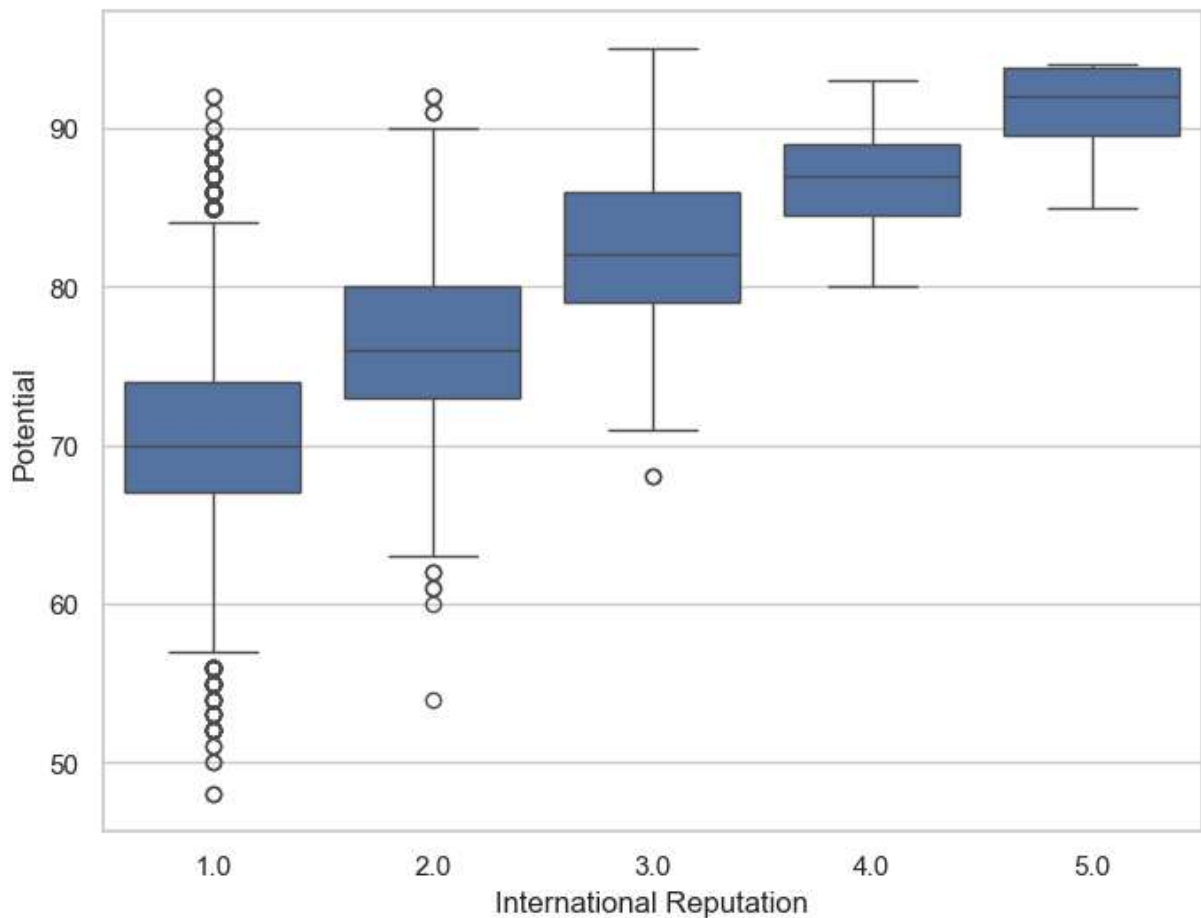
Seaborn `boxplot()` function

- This function draws a box plot to show distributions with respect to categories.
- A box plot (or box-and-whisker plot) shows the distribution of quantitative data in a way that facilitates comparisons between variables or across levels of a categorical variable.
- The box shows the quartiles of the dataset while the whiskers extend to show the rest of the distribution, except for points that are determined to be "outliers" using a method that is a function of the inter-quartile range.

```
In [29]: f, ax = plt.subplots(figsize=(8, 6))
sns.boxplot(x=fifa["Potential"])
plt.show()
```

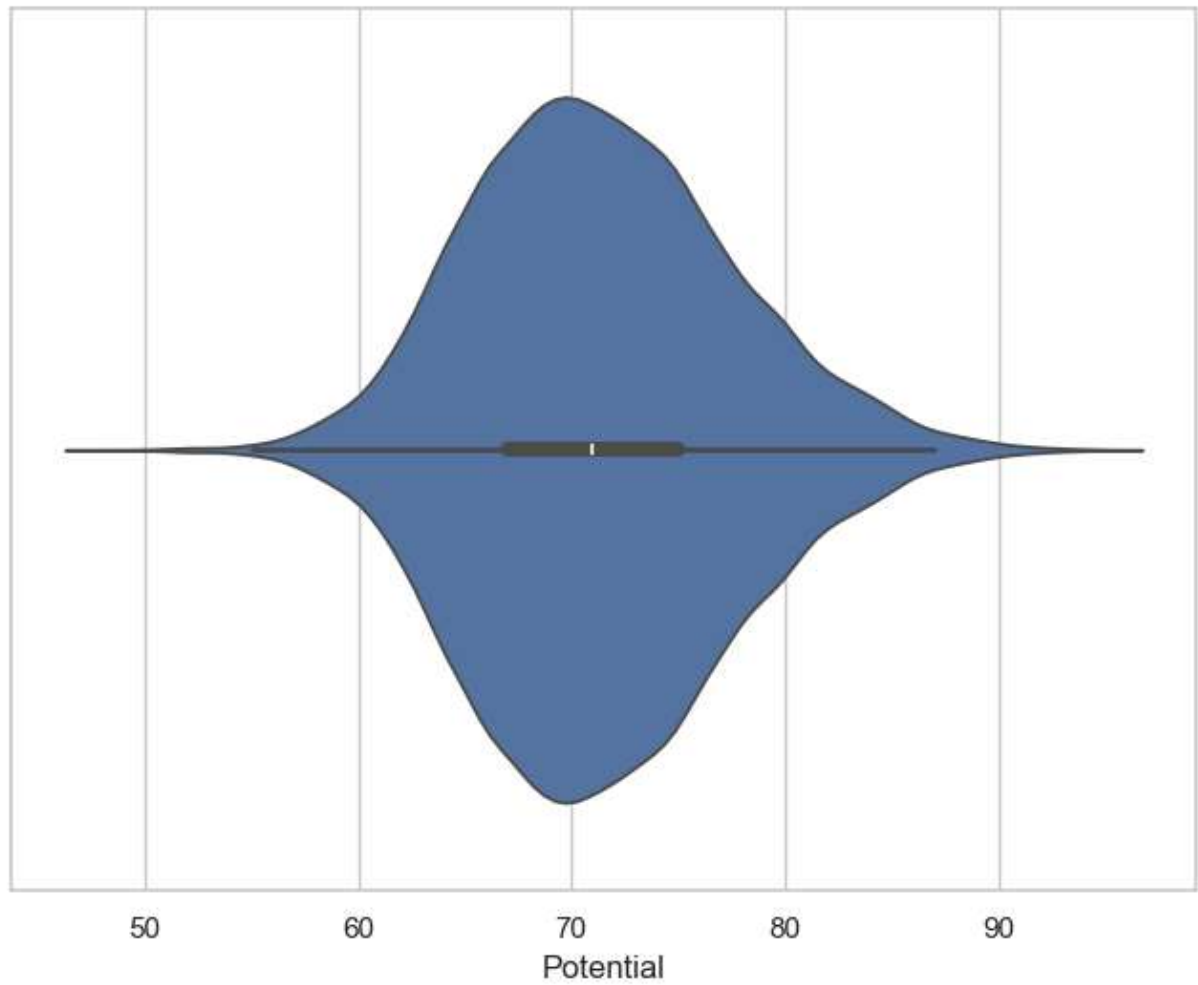
```
In [30]: f, ax = plt.subplots(figsize=(8, 6))
sns.boxplot(x="International Reputation", y="Potential", data=fifa)
plt.show()
```



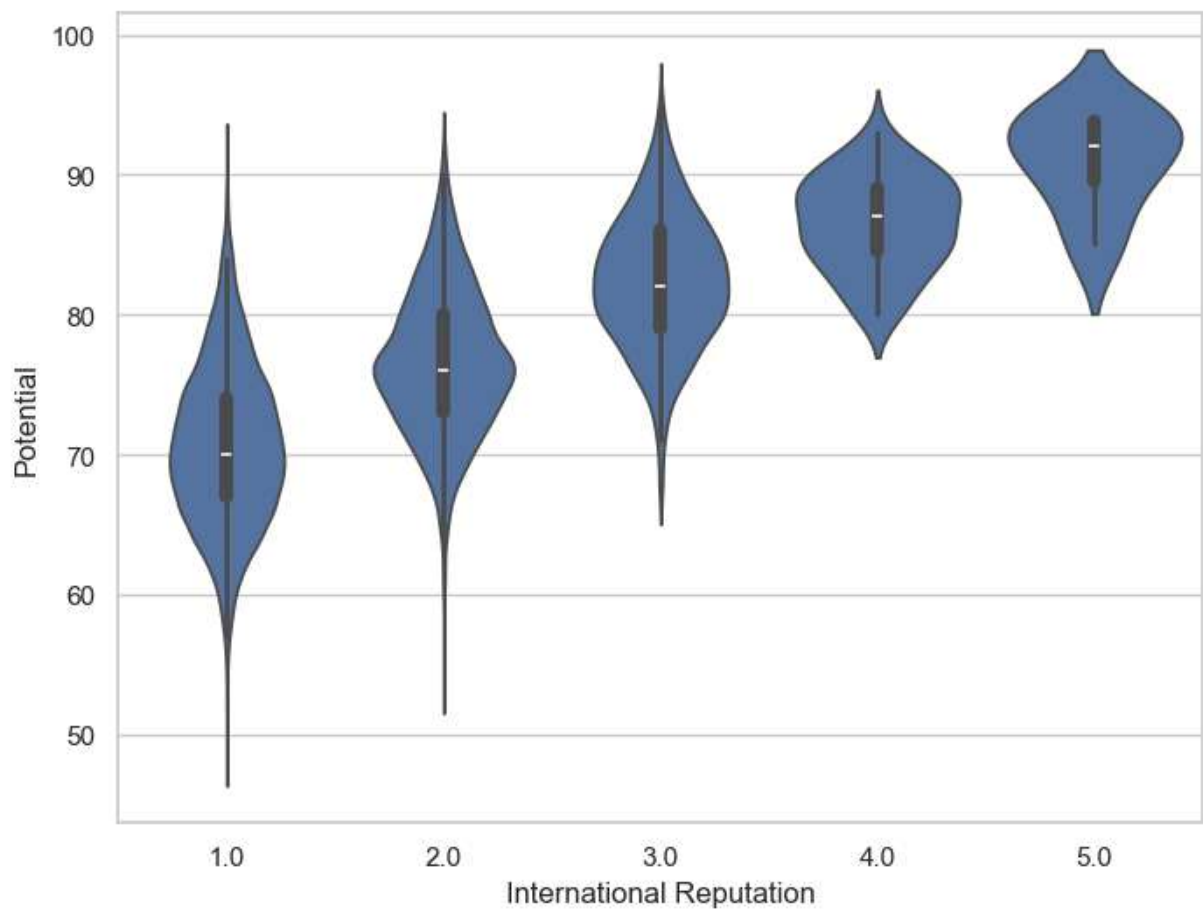
Seaborn violinplot() function

- This function draws a combination of boxplot and kernel density estimate.
- A violin plot plays a similar role as a box and whisker plot.
- It shows the distribution of quantitative data across several levels of one (or more) categorical variables such that those distributions can be compared.
- Unlike a box plot, in which all of the plot components correspond to actual datapoints, the violin plot features a kernel density estimation of the underlying distribution.
- I will plot the violinplot of `Potential` variable as follows-

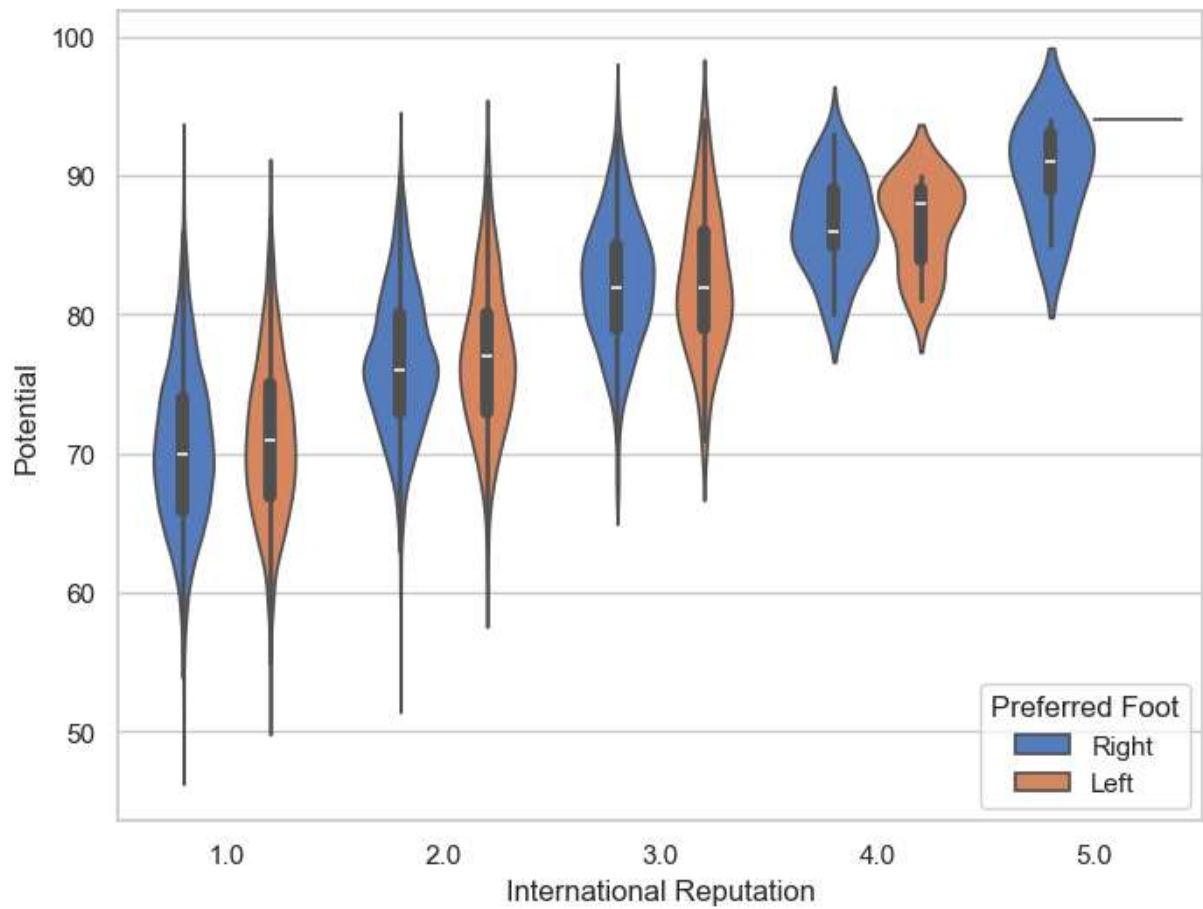
```
In [31]: f, ax = plt.subplots(figsize=(8, 6))
sns.violinplot(x=fifa["Potential"])
plt.show()
```



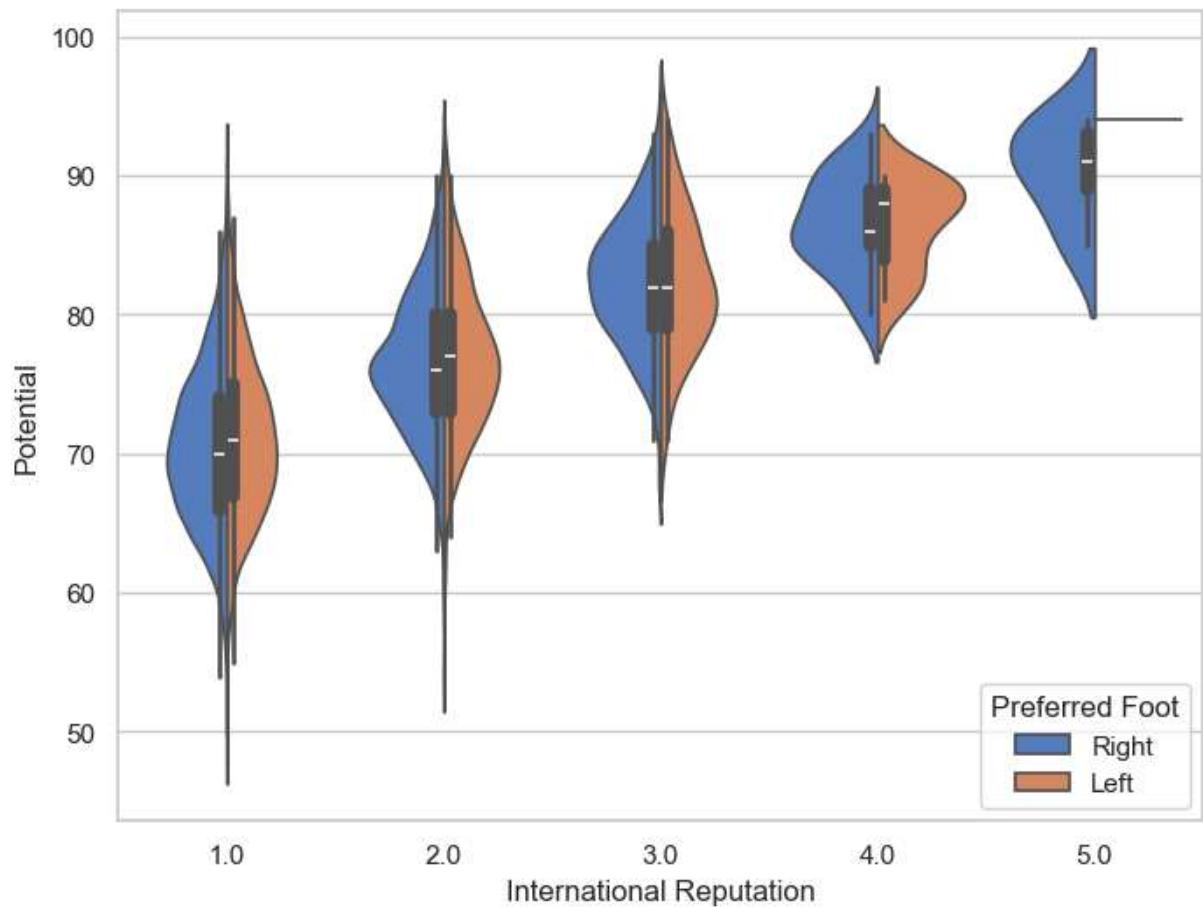
```
In [32]: f, ax = plt.subplots(figsize=(8, 6))
sns.violinplot(x="International Reputation", y="Potential", data=fifa)
plt.show()
```



```
In [33]: f, ax = plt.subplots(figsize=(8, 6))
sns.violinplot(x="International Reputation", y="Potential", hue="Preferred Foot", d
plt.show()
```



```
In [34]: f, ax = plt.subplots(figsize=(8, 6))
sns.violinplot(x="International Reputation", y="Potential", hue="Preferred Foot",
               data=fifa, palette="muted", split=True)
plt.show()
```



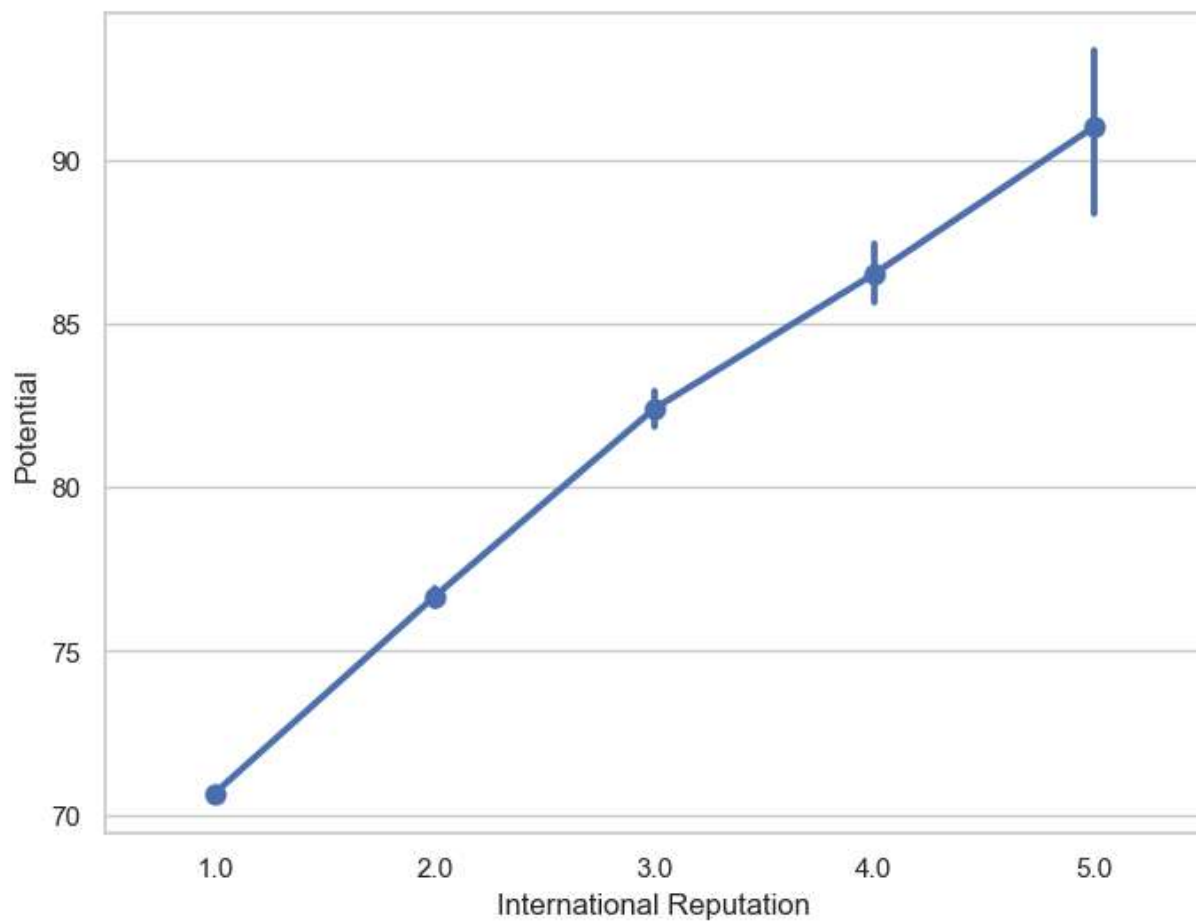
Seaborn `pointplot()` function

- This function shows point estimates and confidence intervals using scatter plot glyphs.
- A point plot represents an estimate of central tendency for a numeric variable by the position of scatter plot points and provides some indication of the uncertainty around that estimate using error bars.

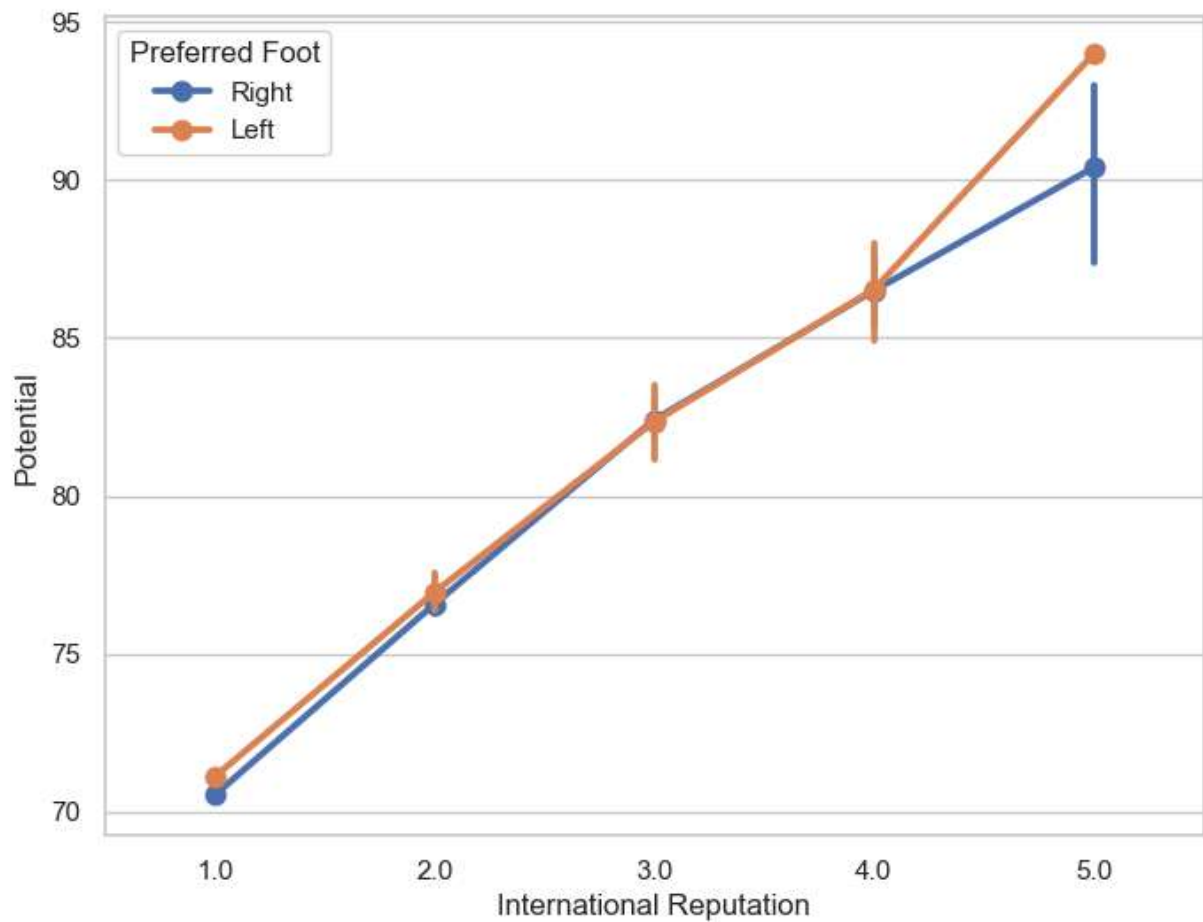
```
In [35]: f, ax = plt.subplots(figsize=(8, 6))
sns.pointplot(x="International Reputation", y="Potential", data=fifa)
plt.show
```

```
Out[35]: <function matplotlib.pyplot.show(close=None, block=None)>
```

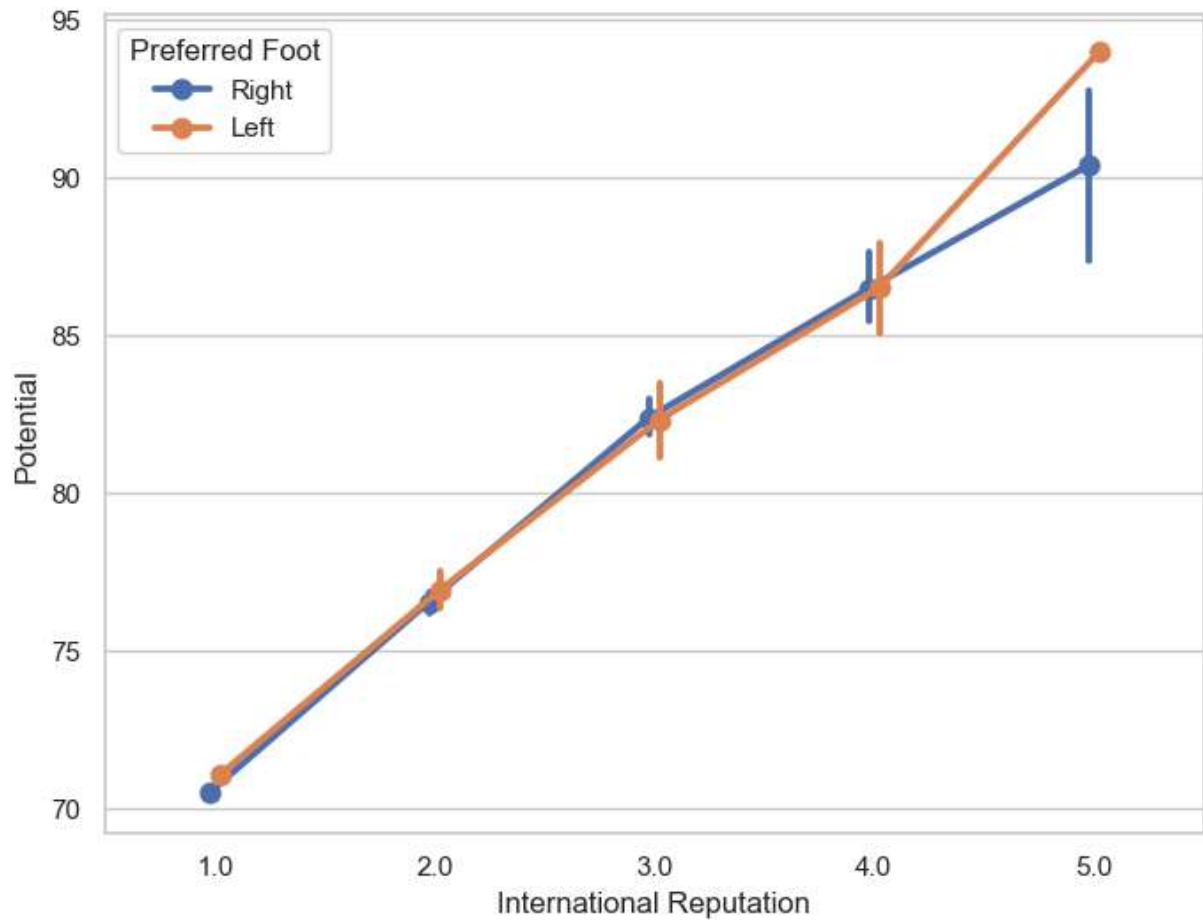
```
In [36]: plt.show()
```



```
In [37]: f, ax = plt.subplots(figsize=(8, 6))
sns.pointplot(x="International Reputation", y="Potential", hue="Preferred Foot", da
plt.show()
```



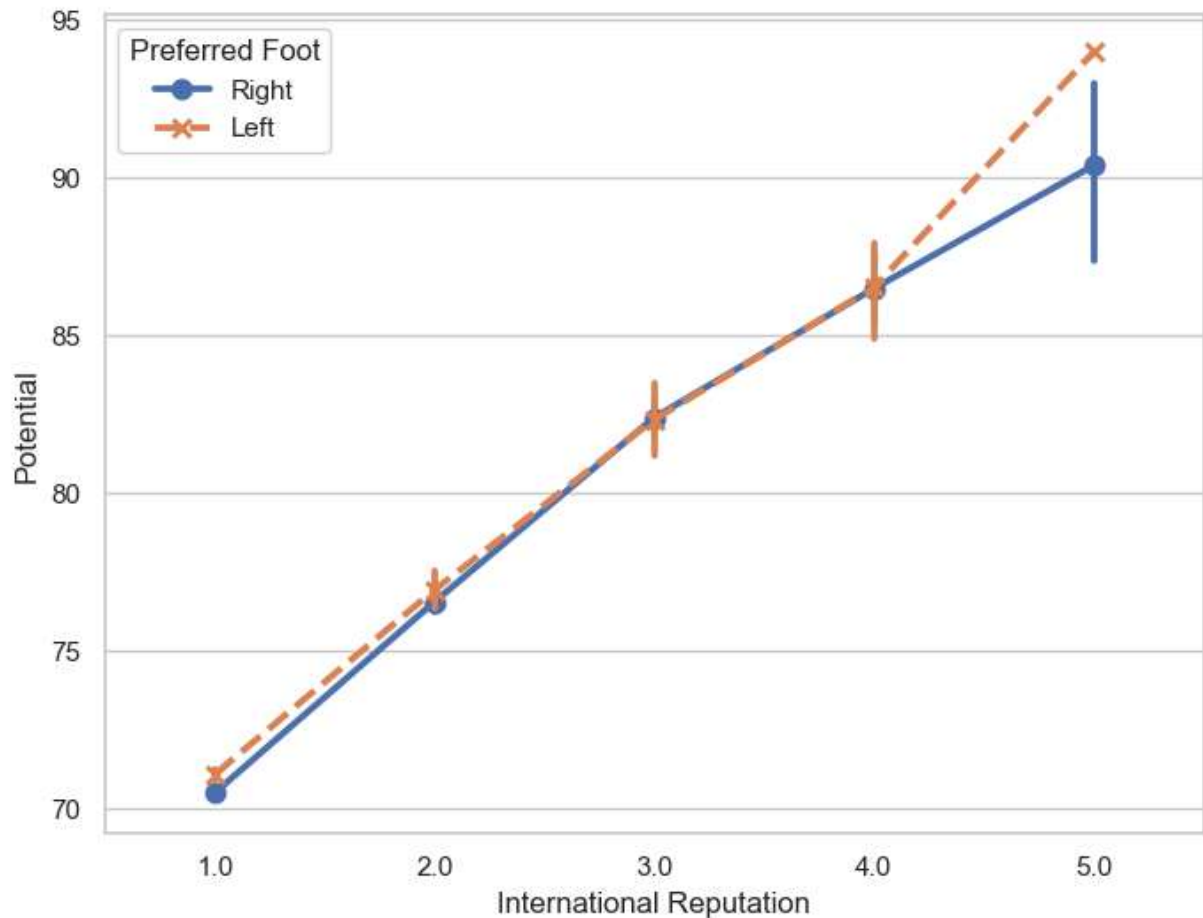
```
In [38]: f, ax = plt.subplots(figsize=(8, 6))
sns.pointplot(x="International Reputation", y="Potential", hue="Preferred Foot", data=df)
plt.show()
```

```
In [39]: f, ax = plt.subplots(figsize=(8, 6))
sns.pointplot(x="International Reputation", y="Potential", hue="Preferred Foot",
              data=fifa, markers=["o", "x"], linestyles=["-", "--"])
plt.show
```

```
Out[39]: <function matplotlib.pyplot.show(close=None, block=None)>
```

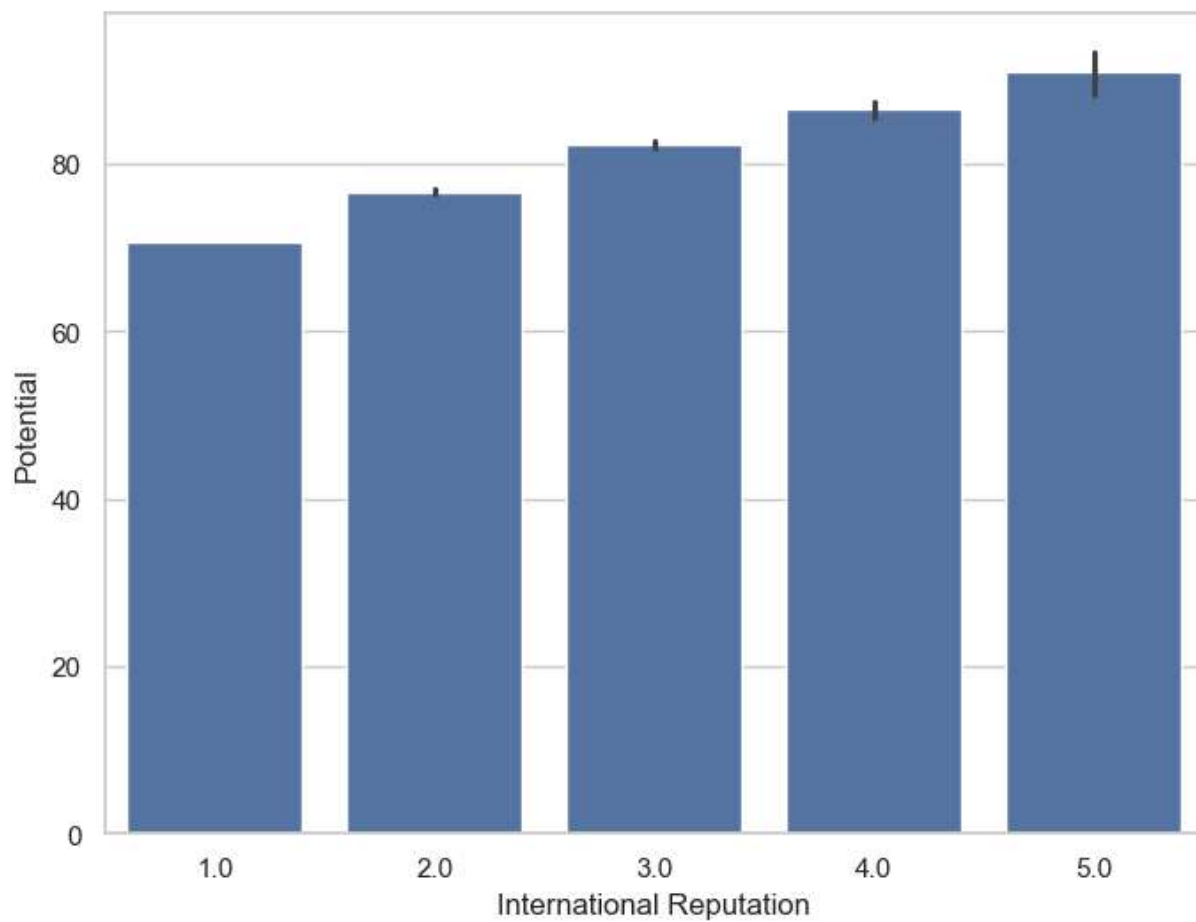
```
In [40]: plt.show()
```



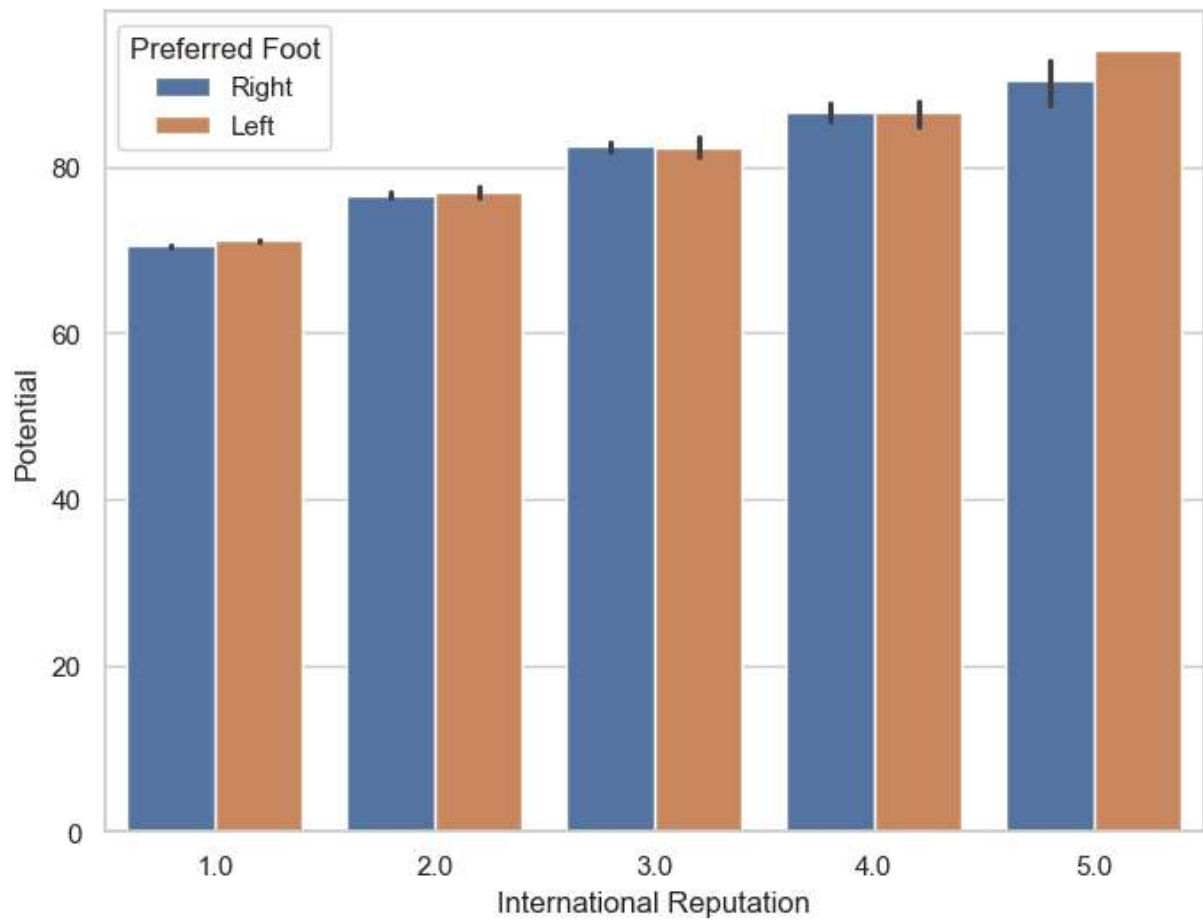
Seaborn `barplot()` function

- This function shows point estimates and confidence intervals as rectangular bars.
- A bar plot represents an estimate of central tendency for a numeric variable with the height of each rectangle and provides some indication of the uncertainty around that estimate using error bars.
- Bar plots include 0 in the quantitative axis range, and they are a good choice when 0 is a meaningful value for the quantitative variable, and you want to make comparisons against it.

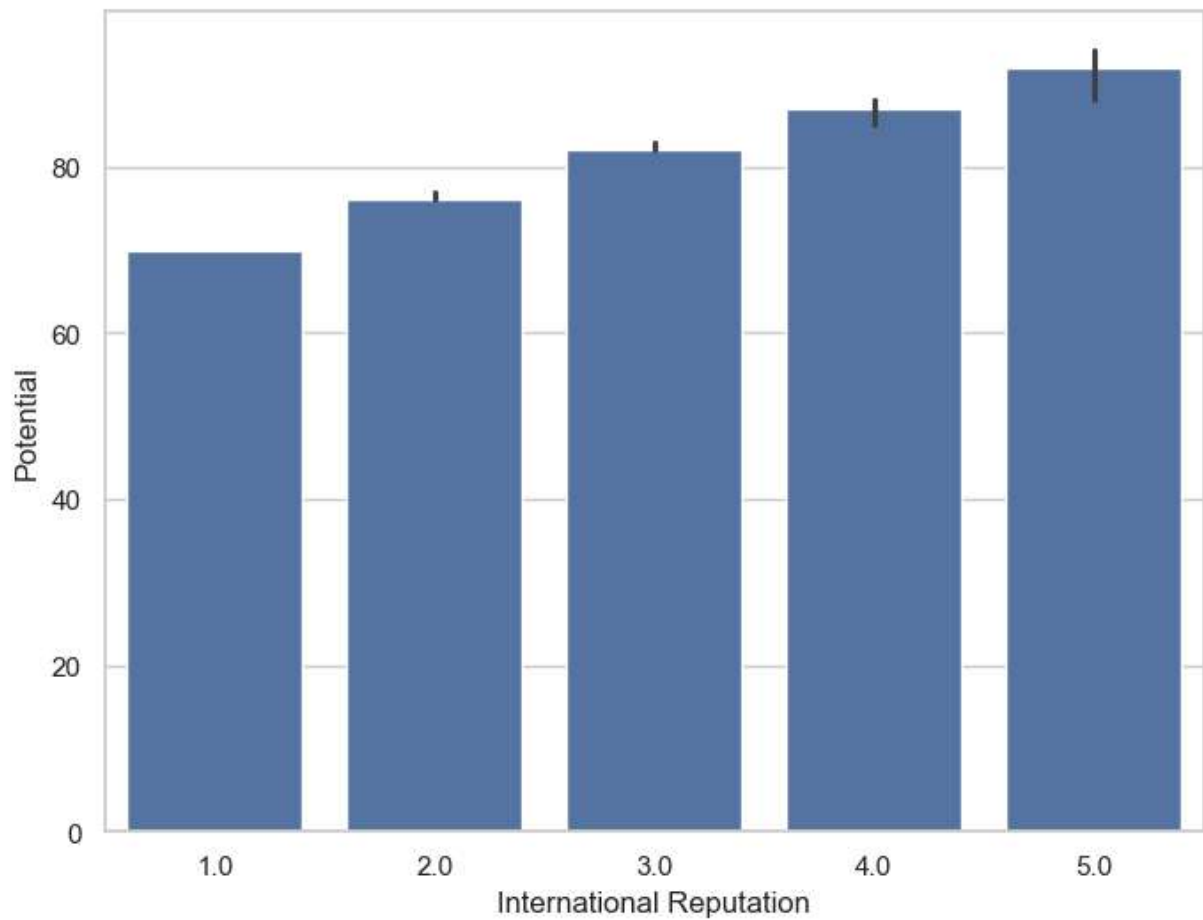
```
In [41]: f, ax = plt.subplots(figsize=(8, 6))
sns.barplot(x="International Reputation", y="Potential", data=fifa)
plt.show()
```



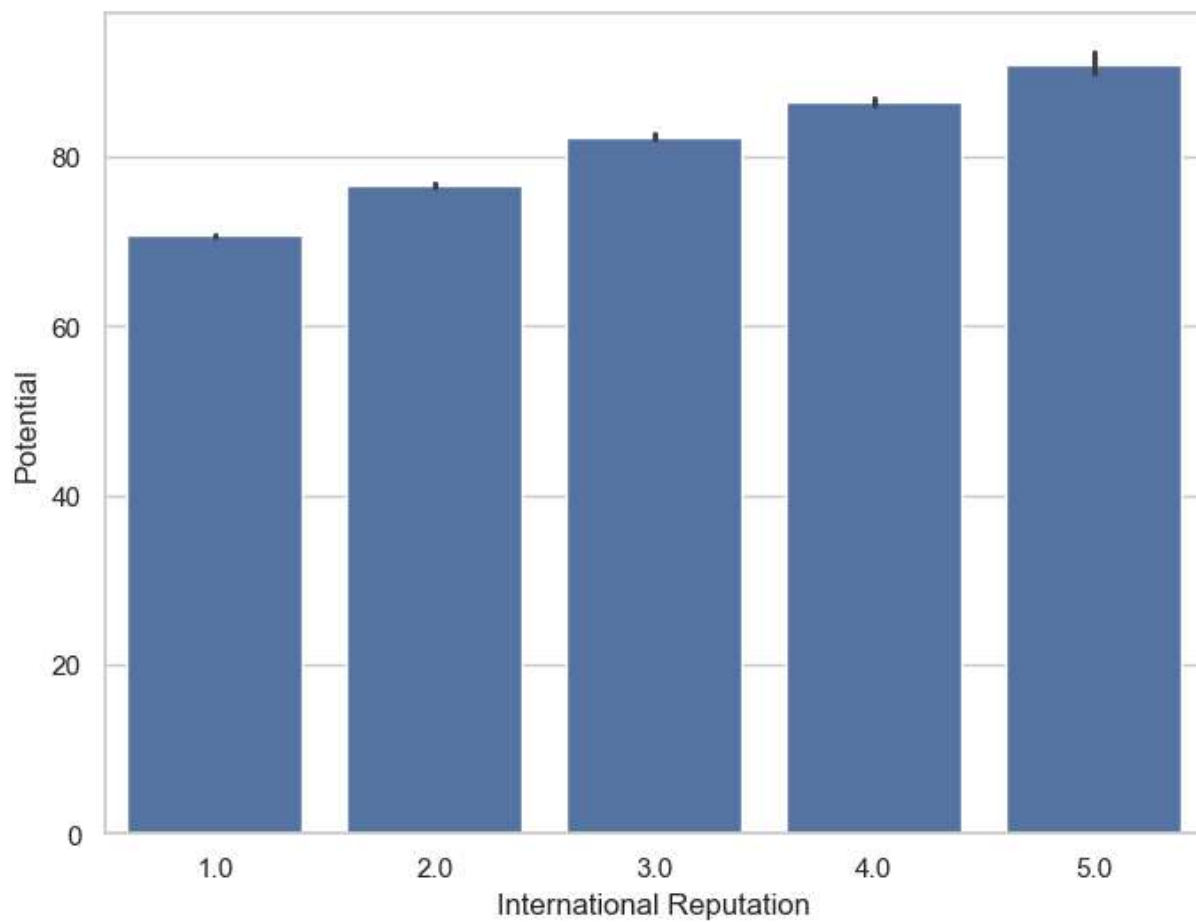
```
In [42]: f, ax = plt.subplots(figsize=(8, 6))
sns.barplot(x="International Reputation", y="Potential", hue="Preferred Foot", data=
plt.show()
```



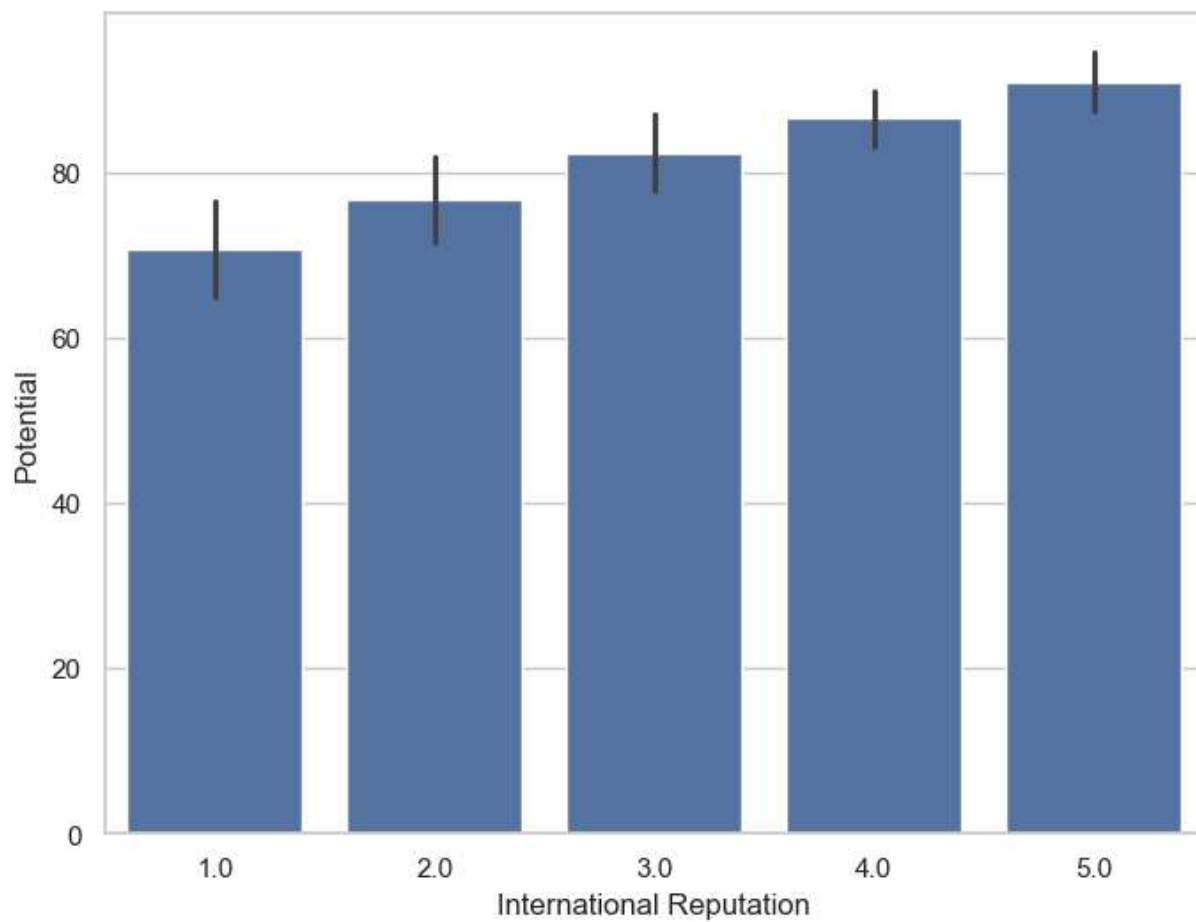
```
In [43]: from numpy import median
f, ax = plt.subplots(figsize=(8, 6))
sns.barplot(x="International Reputation", y="Potential", data=fifa, estimator=median)
plt.show()
```



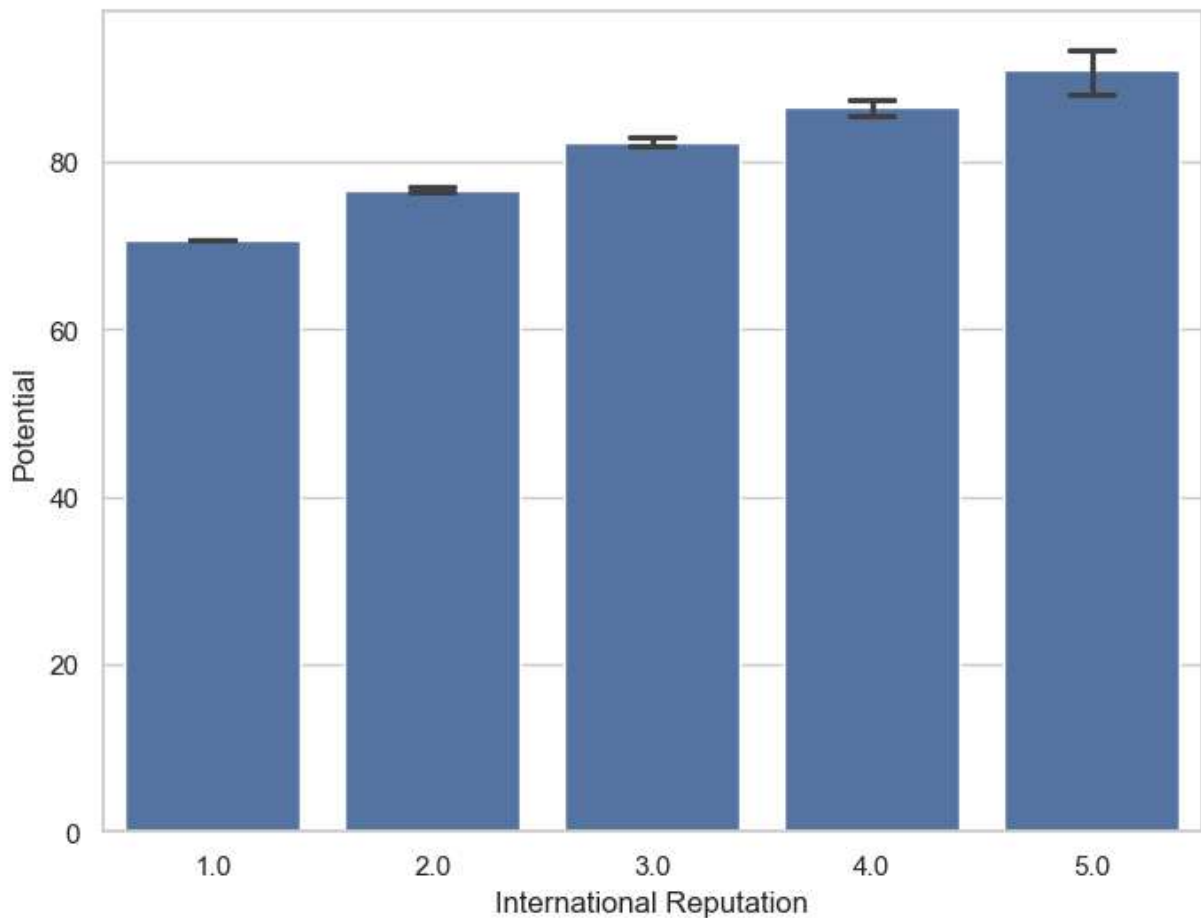
```
In [44]: f, ax = plt.subplots(figsize=(8, 6))  
sns.barplot(x="International Reputation", y="Potential", data=fifa, ci=68)  
plt.show()
```



```
In [45]: f, ax = plt.subplots(figsize=(8, 6))  
sns.barplot(x="International Reputation", y="Potential", data=fifa, ci="sd")  
plt.show()
```



```
In [46]: f, ax = plt.subplots(figsize=(8, 6))
sns.barplot(x="International Reputation", y="Potential", data=fifa, capsize=0.2)
plt.show()
```



Seaborn `relplot()` function

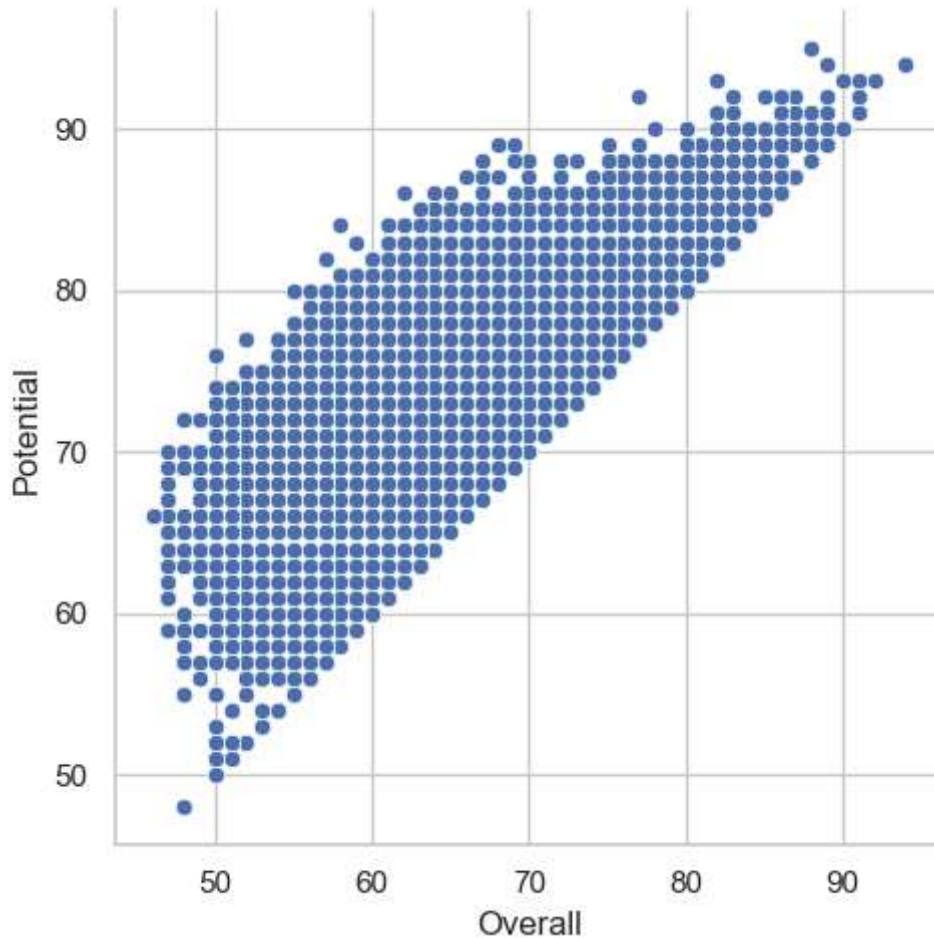
- Seaborn `relplot()` function helps us to draw figure-level interface for drawing relational plots onto a FacetGrid.
- This function provides access to several different axes-level functions that show the relationship between two variables with semantic mappings of subsets.
- The `kind` parameter selects the underlying axes-level function to use-
- `scatterplot()` (with `kind="scatter"`; the default)
- `lineplot()` (with `kind="line"`)

```
In [47]: g = sns.relplot(x="Overall", y="Potential", data=fifa)
```

```
In [48]: g
```

```
Out[48]: <seaborn.axisgrid.FacetGrid at 0x2540b34ca50>
```

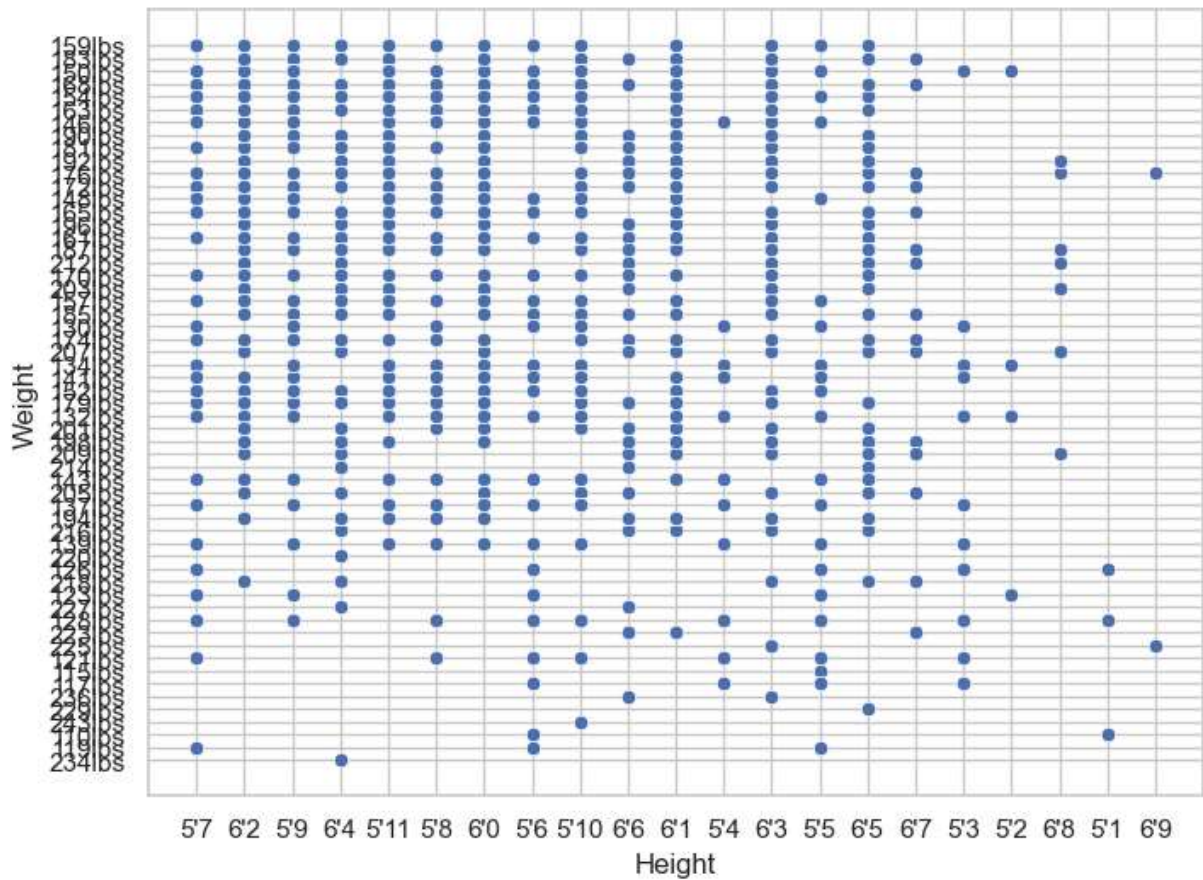
```
In [49]: plt.show()
```

Seaborn `scatterplot()` function

- This function draws a scatter plot with possibility of several semantic groups.
- The relationship between x and y can be shown for different subsets of the data using the `hue`, `size` and `style` parameters.
- These parameters control what visual semantics are used to identify the different subsets.

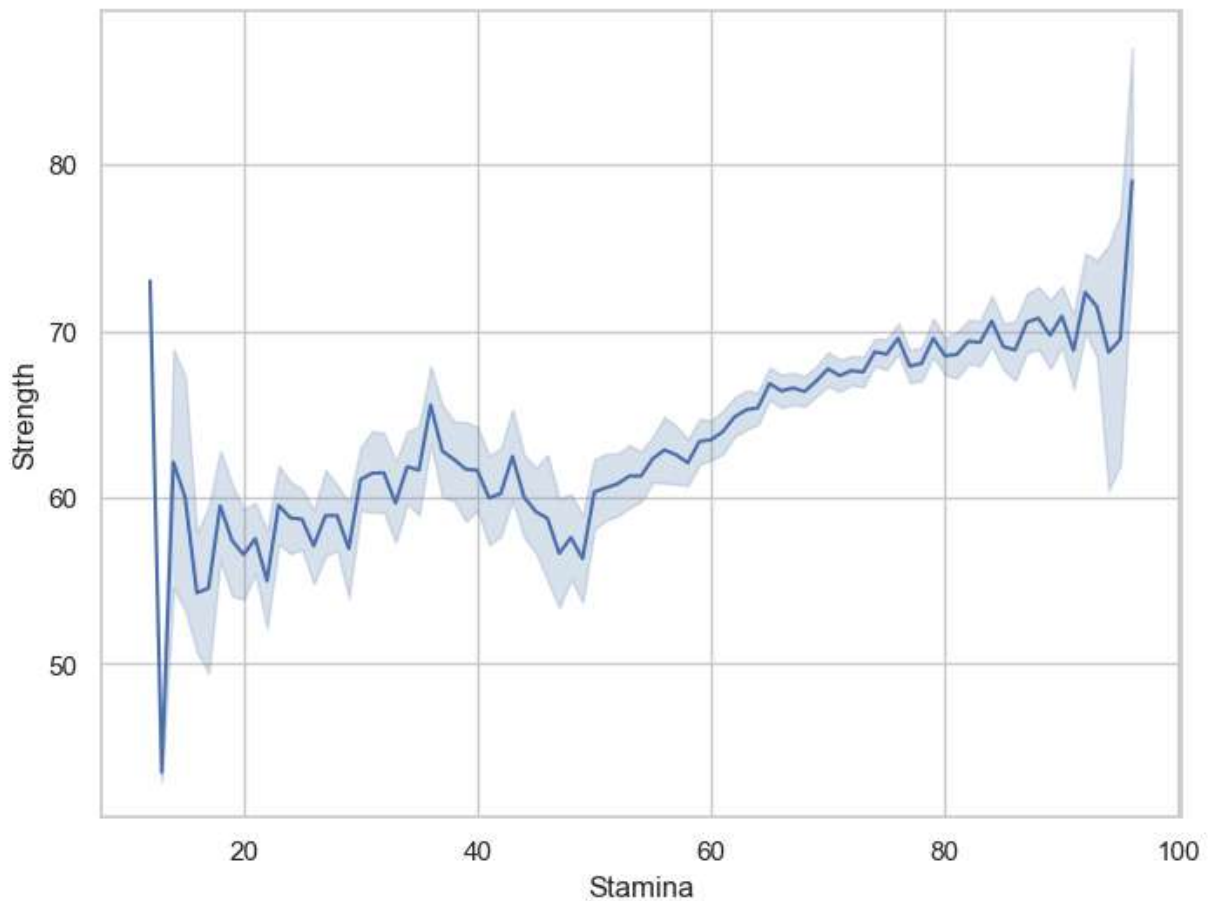
```
In [50]: f, ax = plt.subplots(figsize=(8, 6))
sns.scatterplot(x="Height", y="Weight", data=fifa)
plt.show()
```



Seaborn `lineplot()` function

- This function draws a line plot with possibility of several semantic groupings.
- The relationship between x and y can be shown for different subsets of the data using the `hue`, `size` and `style` parameters.
- These parameters control what visual semantics are used to identify the different subsets.

```
In [51]: f, ax = plt.subplots(figsize=(8, 6))
ax = sns.lineplot(x="Stamina", y="Strength", data=fifa)
plt.show()
```

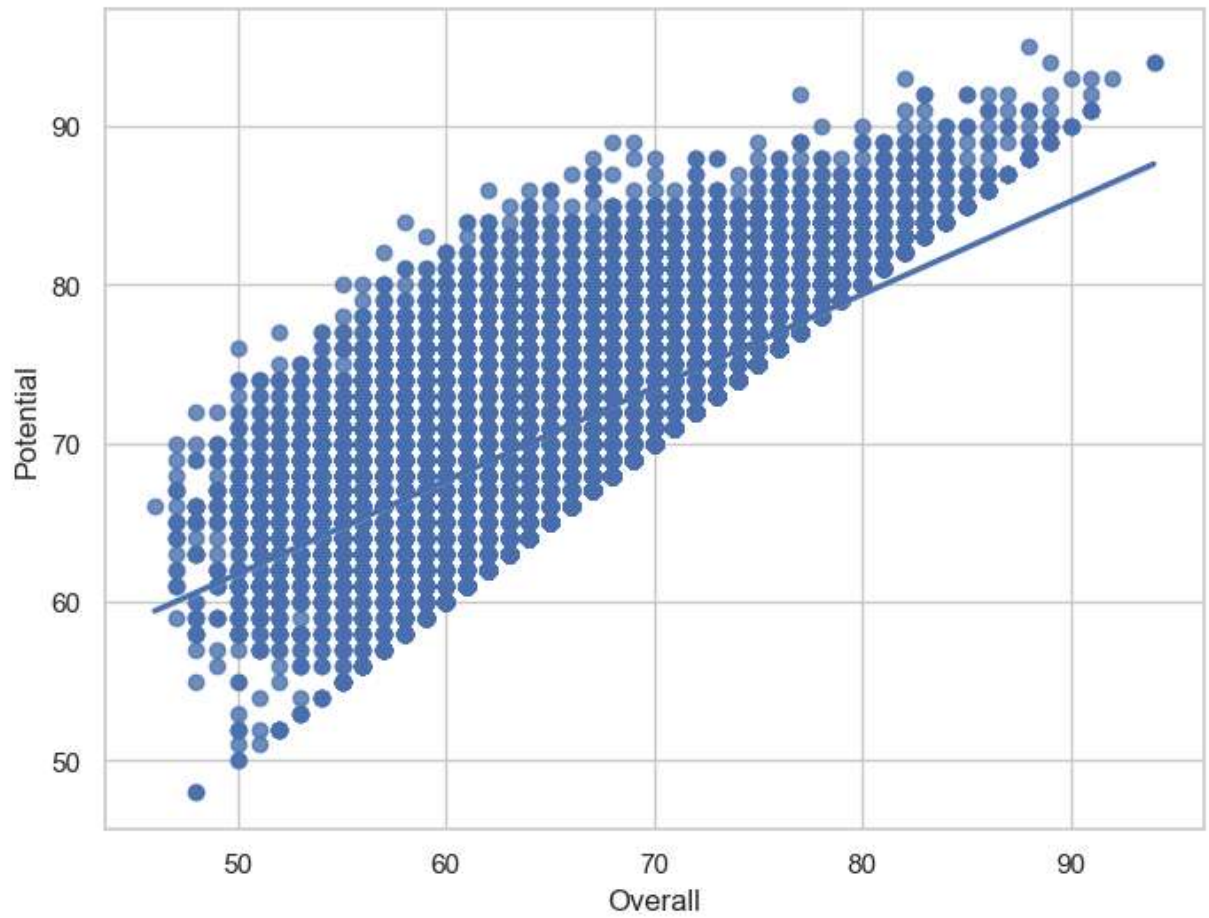


Visualize linear relationship with Seaborn `regplot()` function

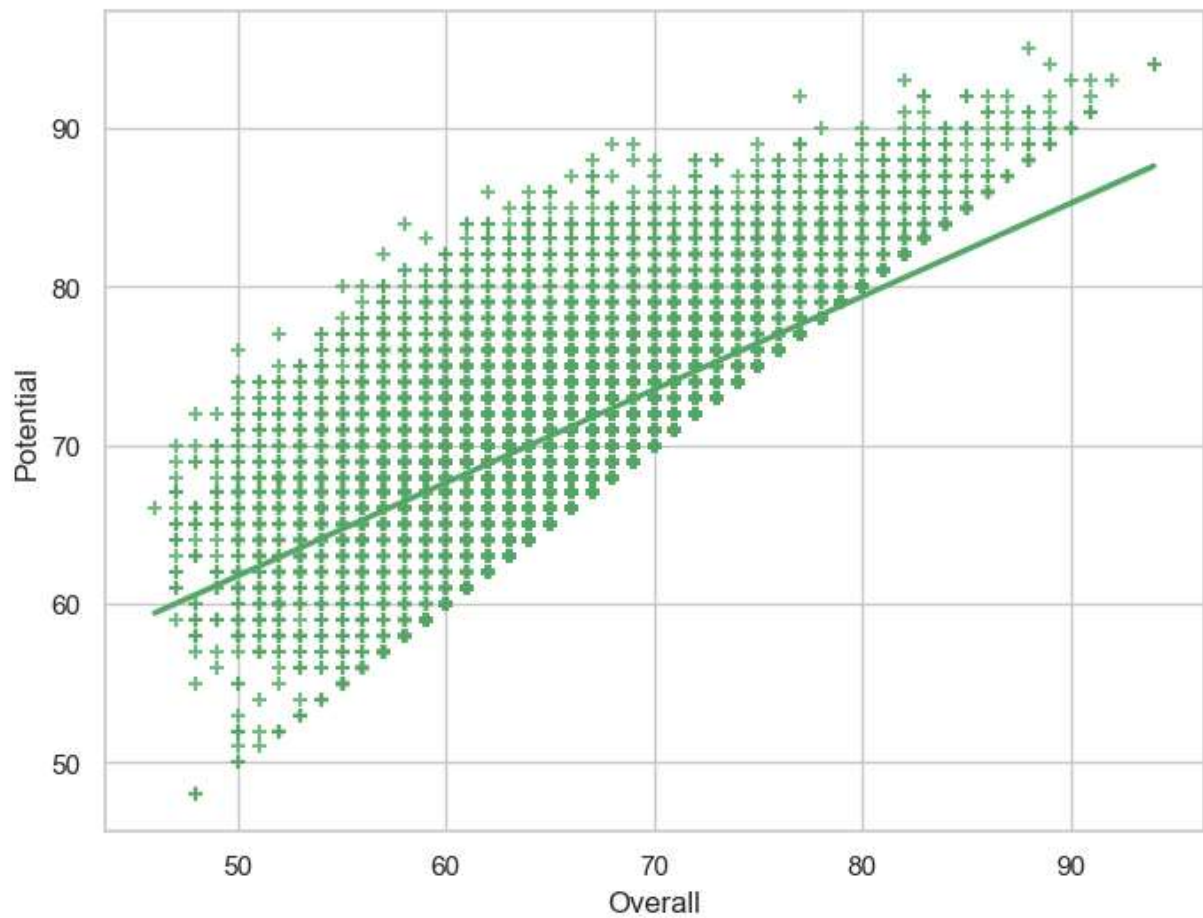
Seaborn `regplot()` function

- This function plots data and a linear regression model fit.
- We can plot a linear regression model between `Overall` and `Potential` variable with `regplot()` function as follows-

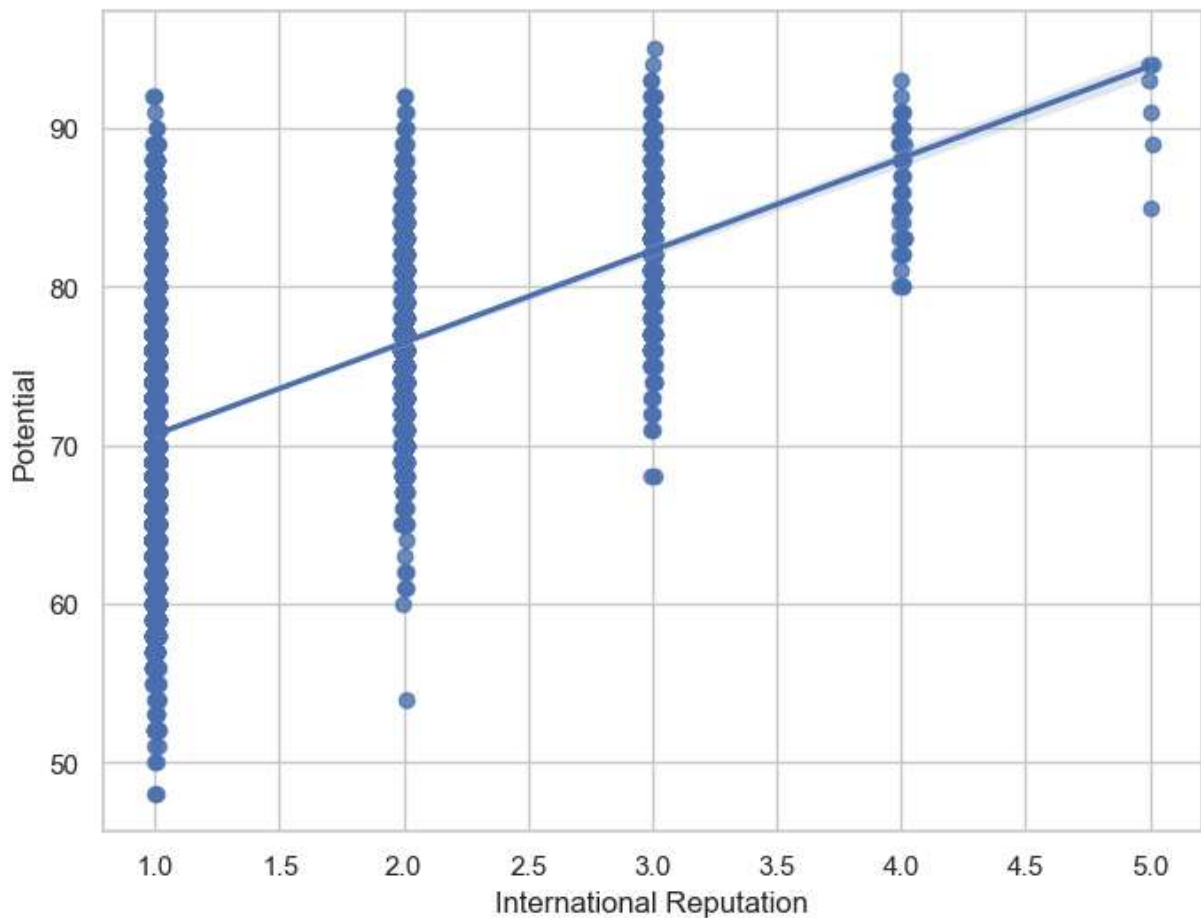
```
In [52]: f, ax = plt.subplots(figsize=(8, 6))
ax = sns.regplot(x="Overall", y="Potential", data=fifa)
plt.show()
```



```
In [53]: f, ax = plt.subplots(figsize=(8, 6))
ax = sns.regplot(x="Overall", y="Potential", data=fifa, color="g", marker="+")
plt.show()
```



```
In [54]: f, ax = plt.subplots(figsize=(8, 6))
sns.regplot(x="International Reputation", y="Potential", data=fifa, x_jitter=.01)
plt.show()
```



Seaborn `lplot()` function

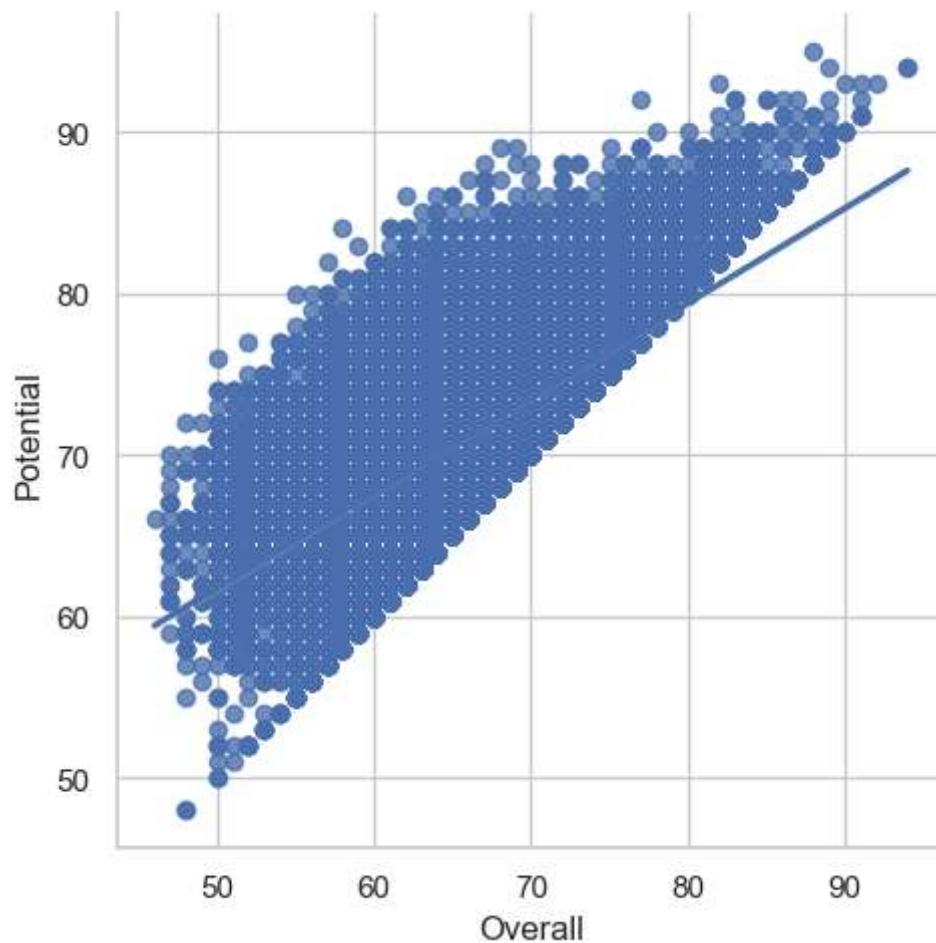
- This function plots data and regression model fits across a `FacetGrid`.
- This function combines `regplot()` and `FacetGrid`.
- It is intended as a convenient interface to fit regression models across conditional subsets of a dataset.
- We can plot a linear regression model between `Overall` and `Potential` variable with `lplot()` function as follows-

```
In [55]: g = sns.lplot(x="Overall", y="Potential", data=fifa)
```

```
In [56]: g
```

```
Out[56]: <seaborn.axisgrid.FacetGrid at 0x2540b118550>
```

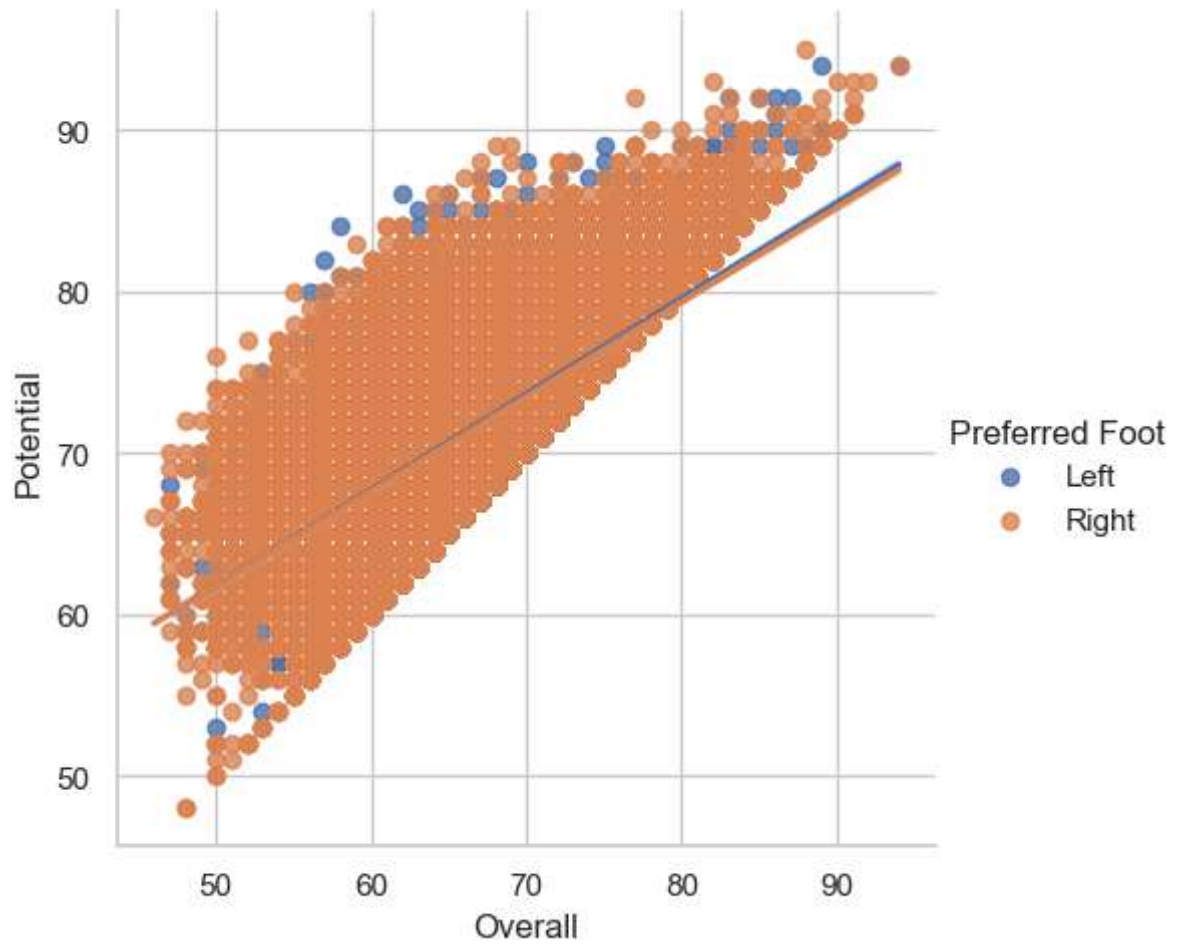
```
In [57]: plt.show()
```

```
In [58]: g = sns.lmplot(x="Overall", y="Potential", hue="Preferred Foot", data=fifa)
g
```

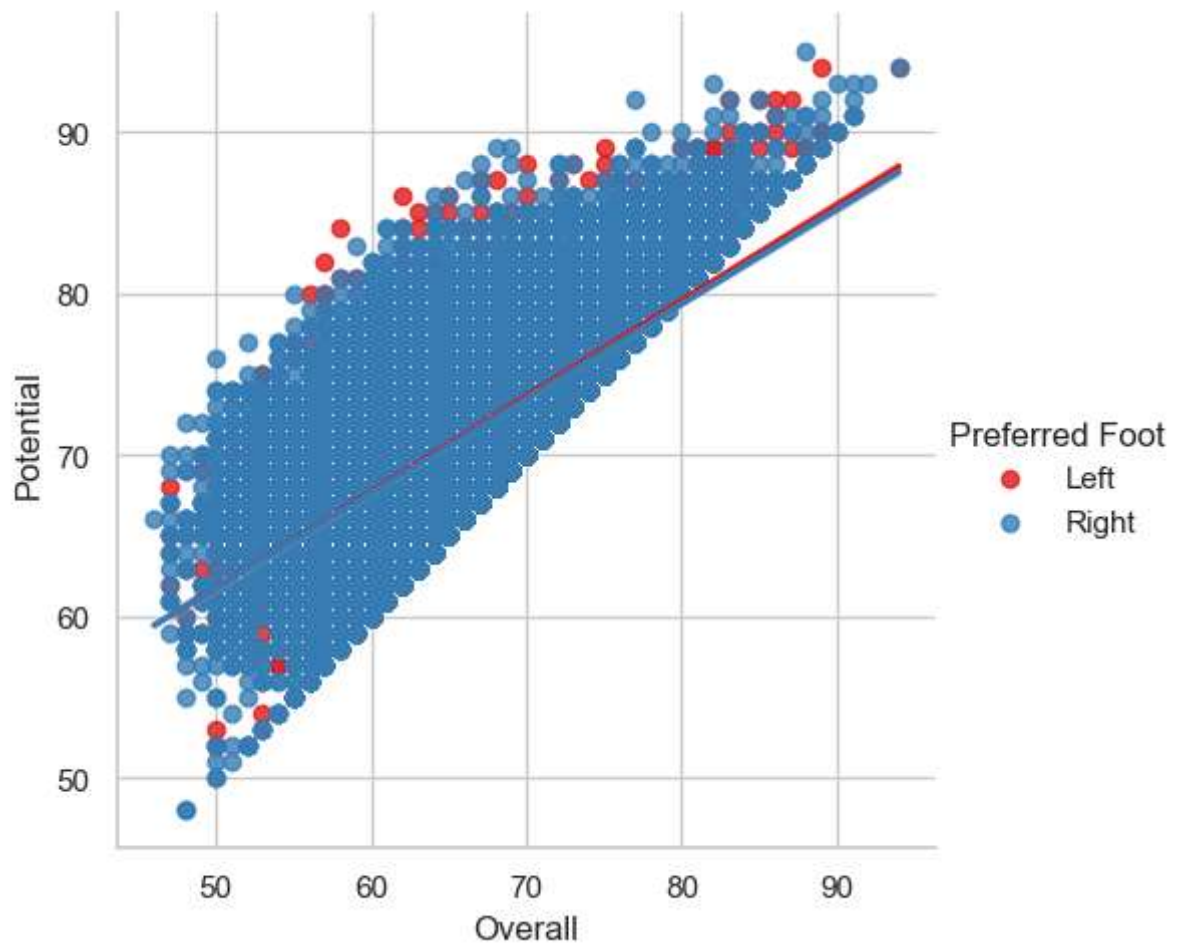
```
Out[58]: <seaborn.axisgrid.FacetGrid at 0x25409b35590>
```

```
In [59]: plt.show()
```

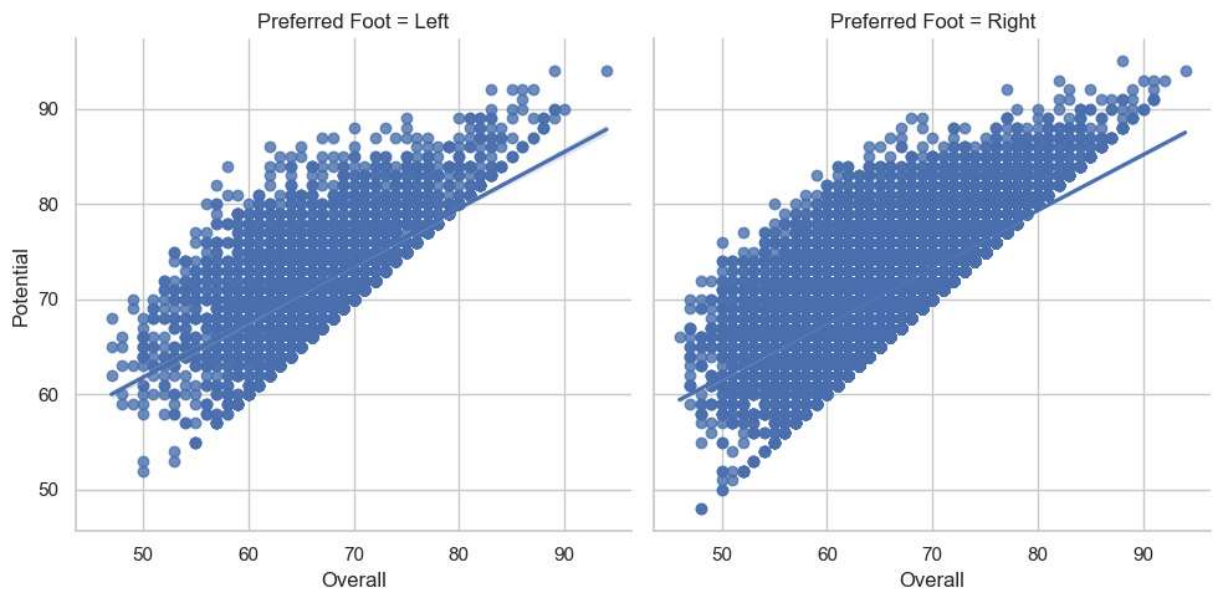


```
In [60]: g = sns.lmplot(x="Overall", y="Potential", hue="Preferred Foot", data=fifa, palette=
```

```
In [61]: g
plt.show()
```

```
In [62]: g = sns.lmplot(x="Overall", y="Potential", col="Preferred Foot", data=fifa)
g
plt.show()
```



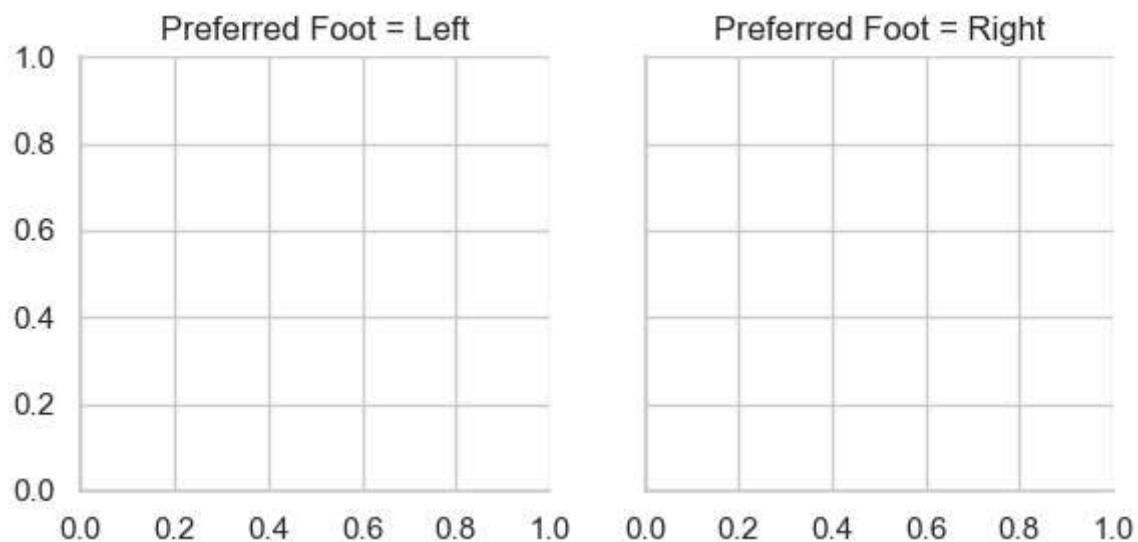
Multi-plot grids

Seaborn FacetGrid() function

- The FacetGrid class is useful when you want to visualize the distribution of a variable or the relationship between multiple variables separately within subsets of your dataset.
- A FacetGrid can be drawn with up to three dimensions - `row`, `col` and `hue`. The first two have obvious correspondence with the resulting array of axes - the `hue` variable is a third dimension along a depth axis, where different levels are plotted with different colors.
- The class is used by initializing a FacetGrid object with a dataframe and the names of the variables that will form the `row`, `column` or `hue` dimensions of the grid.
- These variables should be categorical or discrete, and then the data at each level of the variable will be used for a facet along that axis.

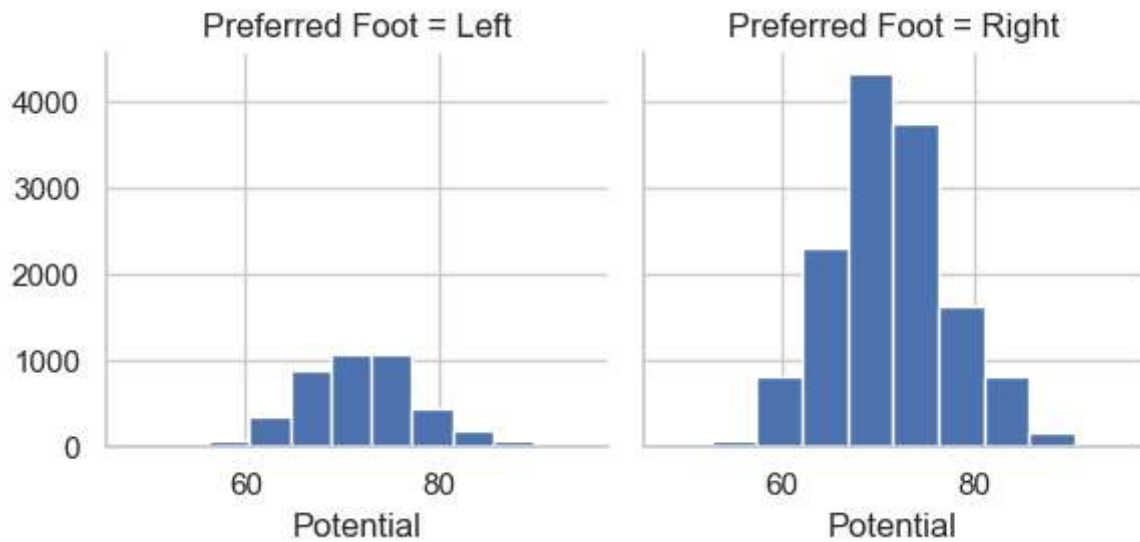
```
In [63]: g = sns.FacetGrid(fifa, col="Preferred Foot")
```

```
In [64]: g
plt.show()
```



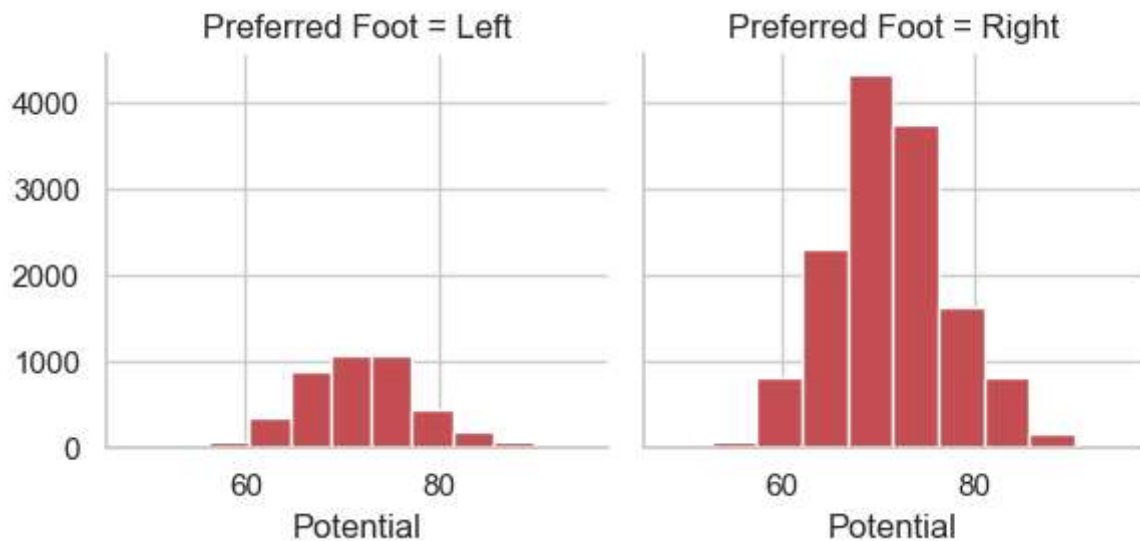
```
In [65]: g = sns.FacetGrid(fifa, col="Preferred Foot")
g = g.map(plt.hist, "Potential")
```

```
In [66]: g
plt.show()
```

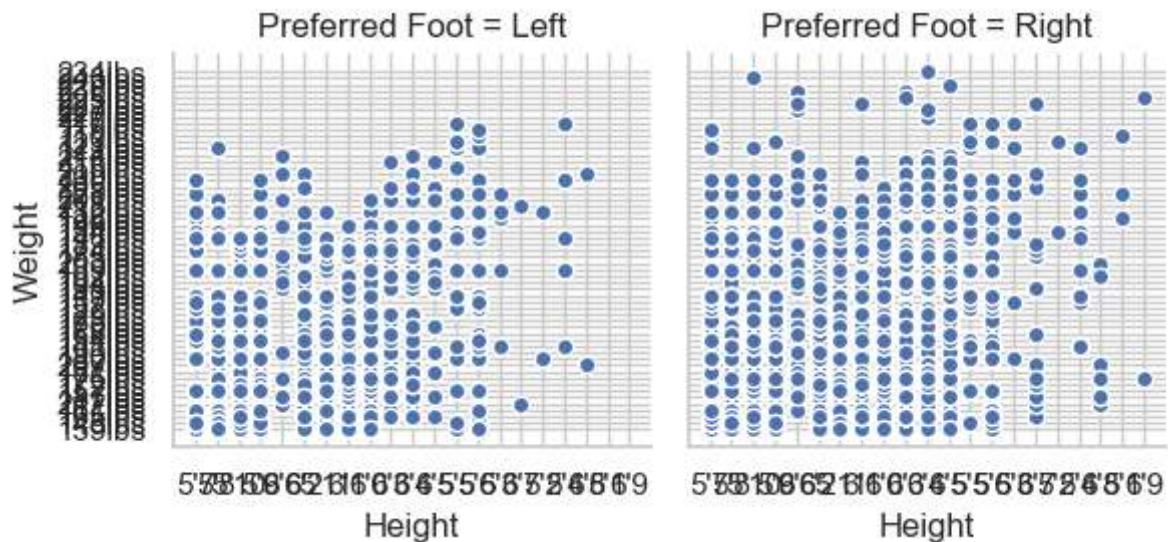


```
In [67]: g = sns.FacetGrid(fifa, col="Preferred Foot")
g = g.map(plt.hist, "Potential", bins=10, color="r")
```

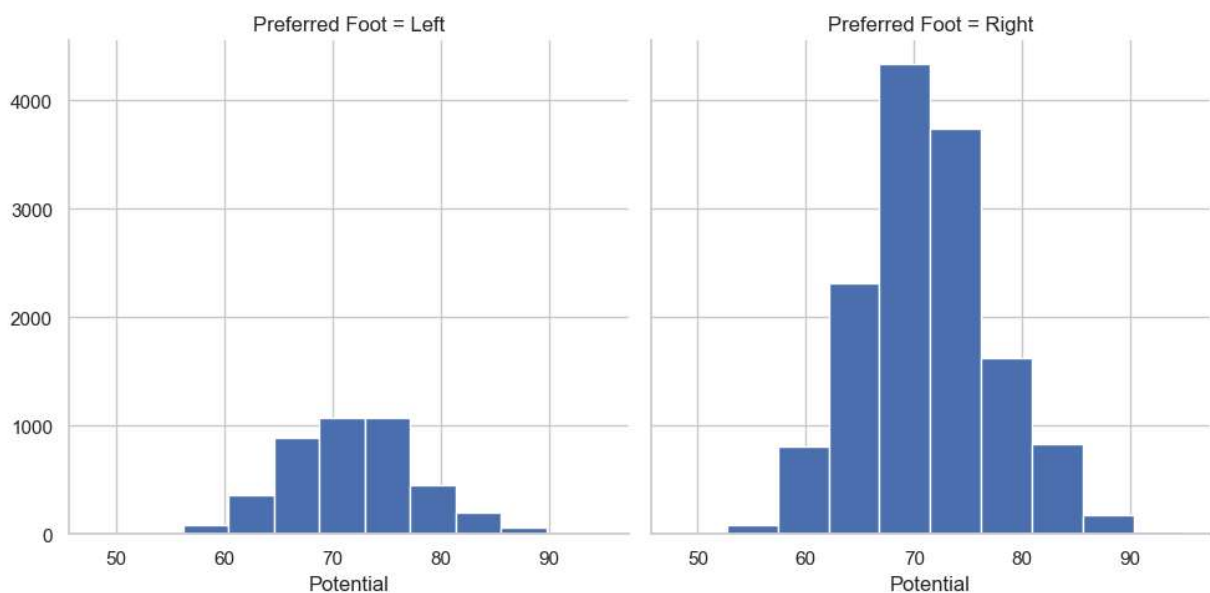
```
In [68]: g
plt.show()
```



```
In [69]: g = sns.FacetGrid(fifa, col="Preferred Foot")
g = (g.map(plt.scatter, "Height", "Weight", edgecolor="w").add_legend())
g
plt.show()
```



```
In [70]: g = sns.FacetGrid(fifa, col="Preferred Foot", height=5, aspect=1)
g = g.map(plt.hist, "Potential")
g
plt.show()
```



Seaborn Pairgrid() function

- This function plots subplot grid for plotting pairwise relationships in a dataset.
- This class maps each variable in a dataset onto a column and row in a grid of multiple axes.
- Different axes-level plotting functions can be used to draw bivariate plots in the upper and lower triangles, and the the marginal distribution of each variable can be shown on the diagonal.

- It can also represent an additional level of conditionalization with the hue parameter, which plots different subsets of data in different colors.
- This uses color to resolve elements on a third dimension, but only draws subsets on top of each other and will not tailor the hue parameter for the specific visualization the way that axes-level functions that accept hue will.

```
In [75]: fifa19_new = fifa[['Age', 'Potential', 'Strength', 'Stamina', 'Preferred Foot']]
```

```
In [76]: fifa19_new
```

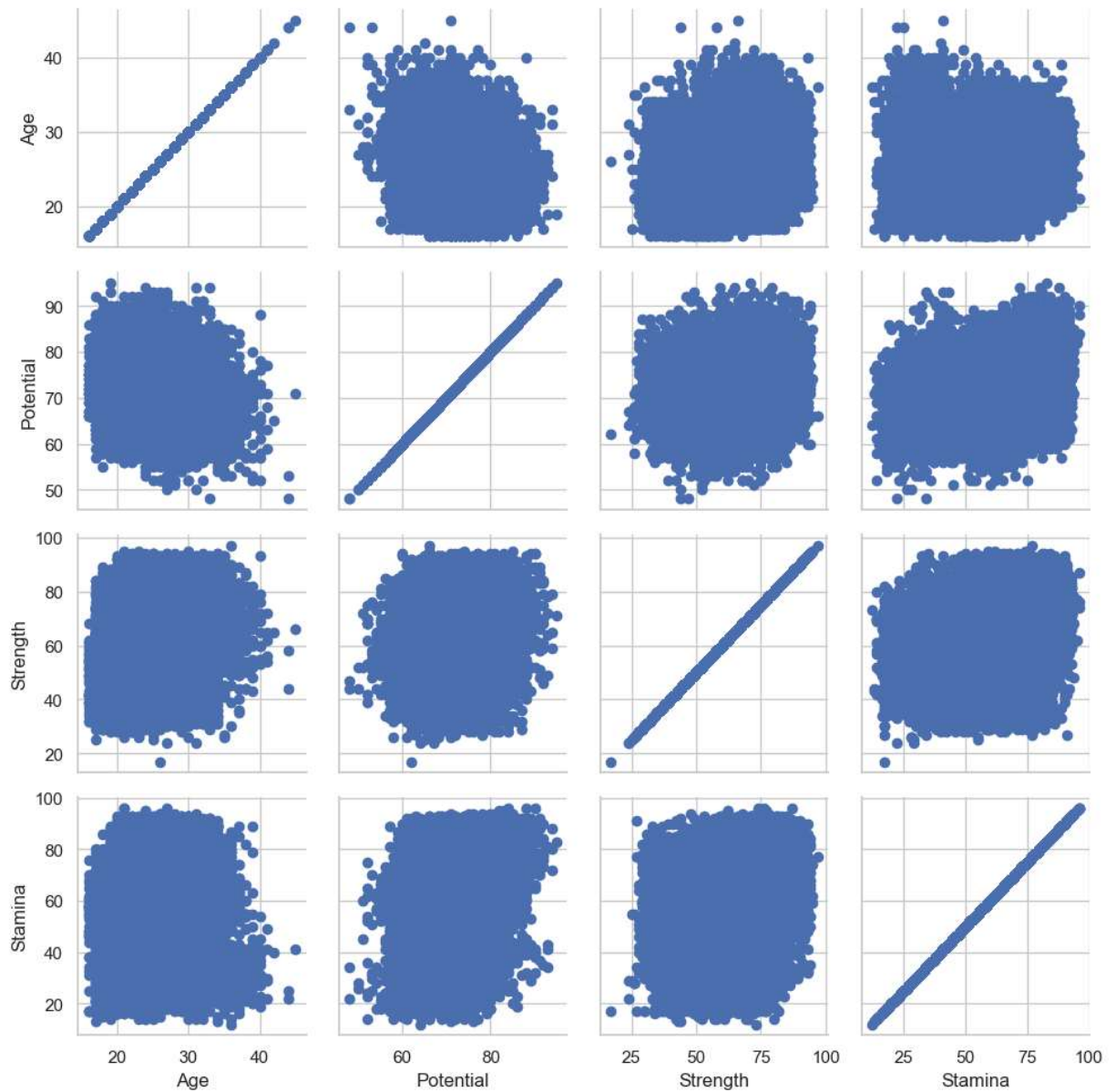
```
Out[76]:
```

	Age	Potential	Strength	Stamina	Preferred Foot
0	31	94	59.0	72.0	Left
1	33	94	79.0	88.0	Right
2	26	93	49.0	81.0	Right
3	27	93	64.0	43.0	Right
4	27	92	75.0	90.0	Right
...	...	...	...	...	...
18202	19	65	47.0	40.0	Right
18203	19	63	67.0	43.0	Right
18204	16	67	32.0	55.0	Right
18205	17	66	48.0	40.0	Right
18206	16	66	60.0	47.0	Right

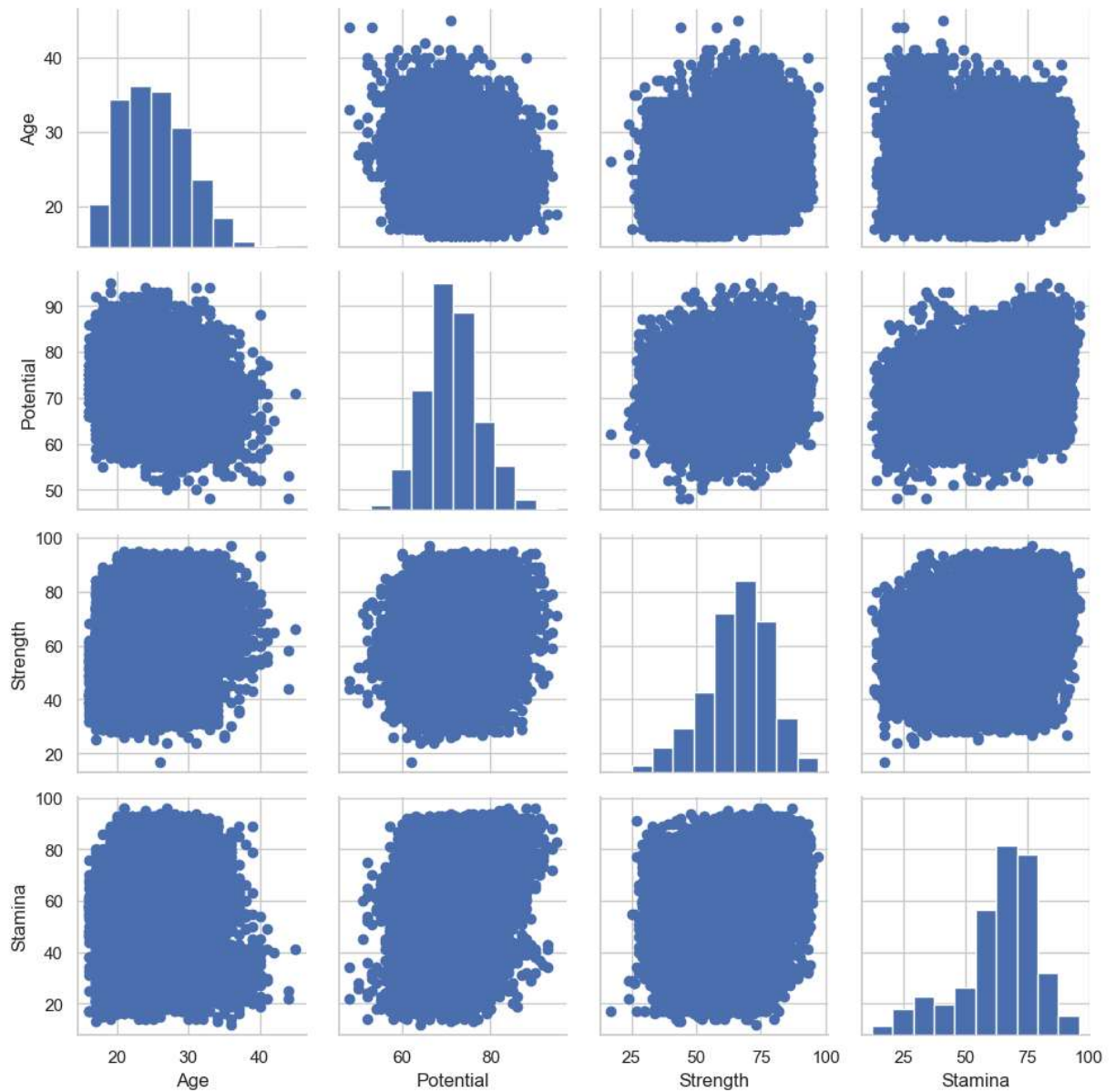
18207 rows × 5 columns

```
In [77]: g = sns.PairGrid(fifa19_new)
g = g.map(plt.scatter)
```

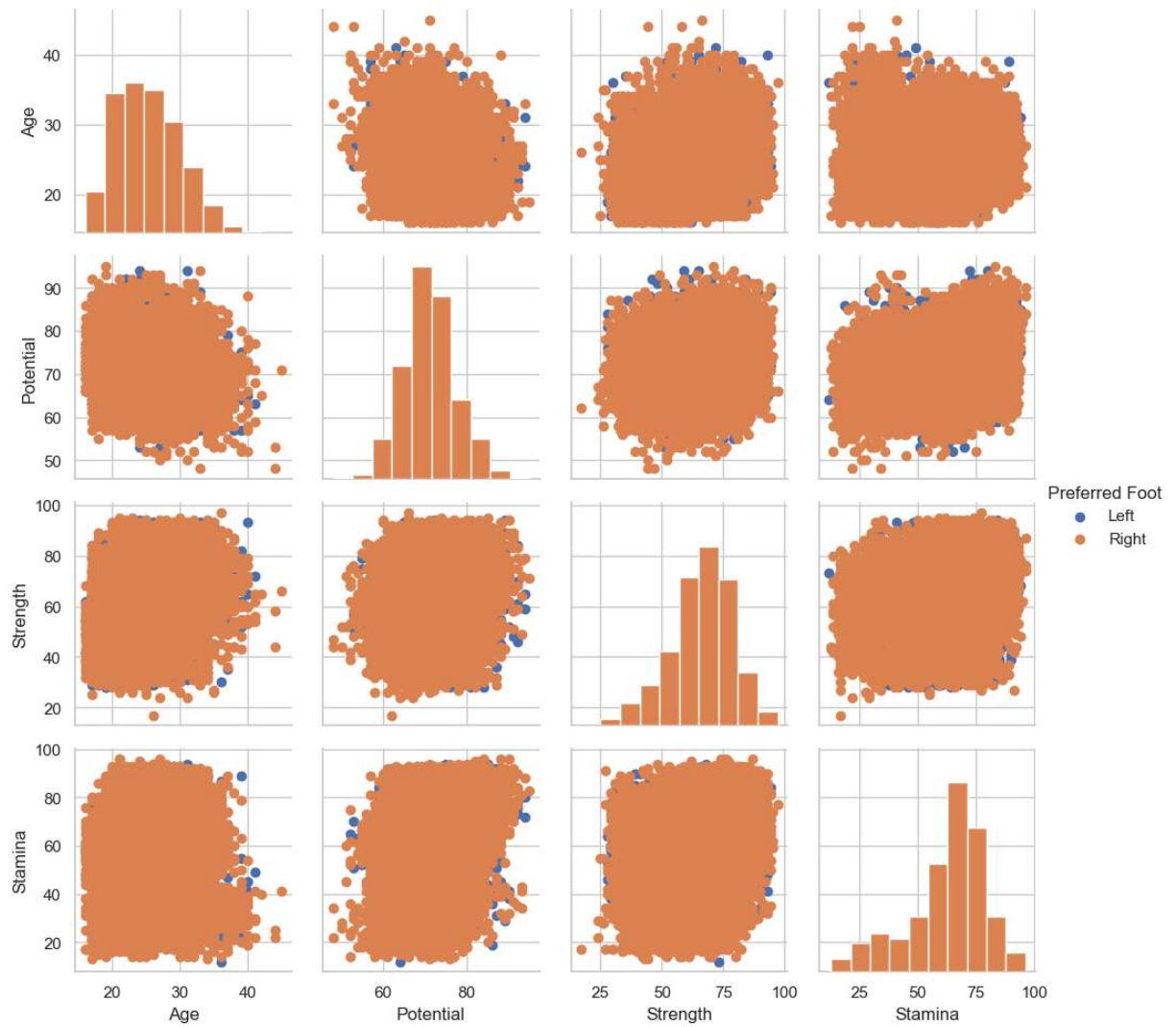
```
In [78]: g
plt.show()
```



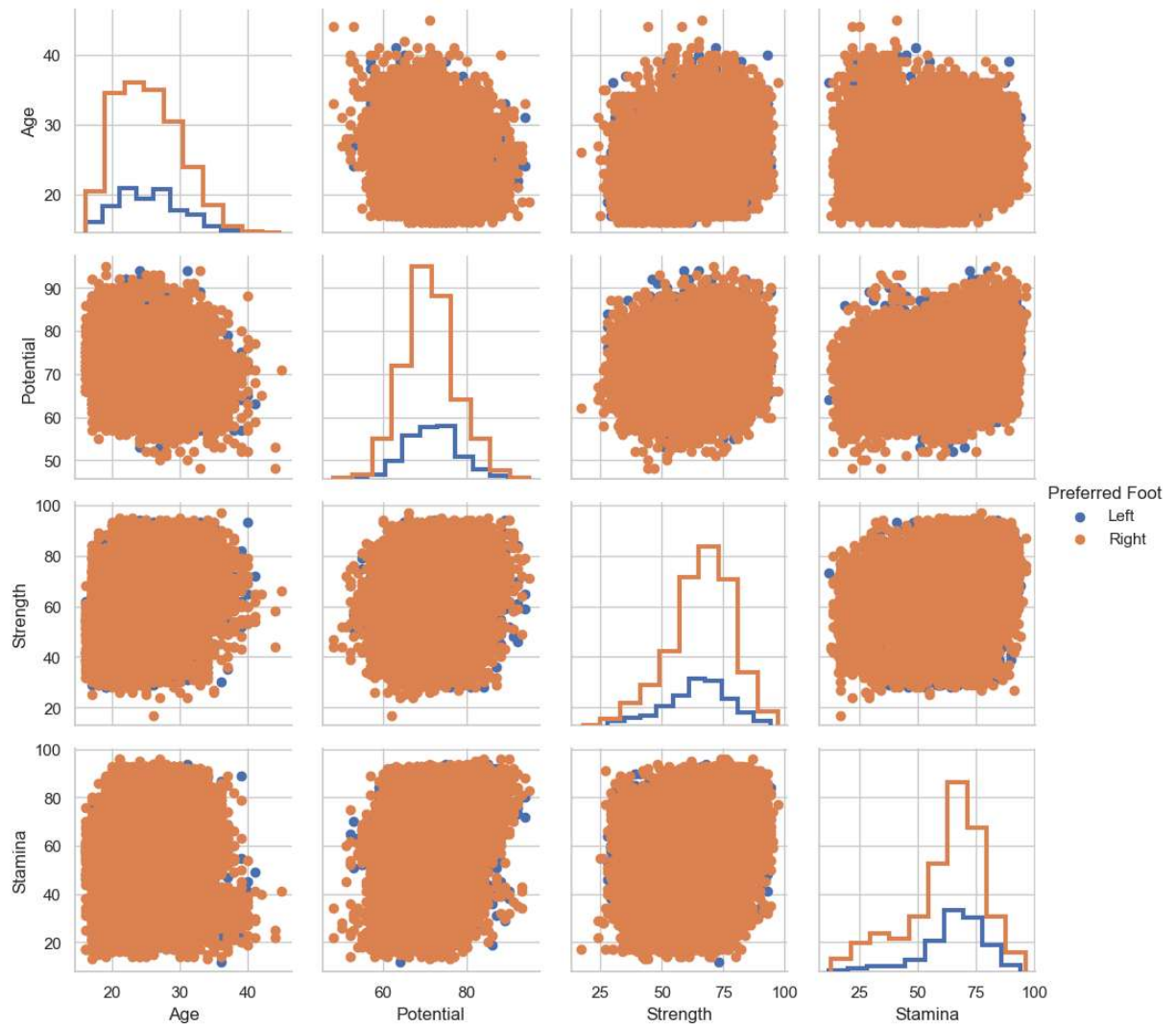
```
In [79]: g = sns.PairGrid(fifa19_new)
g = g.map_diag(plt.hist)
g = g.map_offdiag(plt.scatter)
g
plt.show()
```

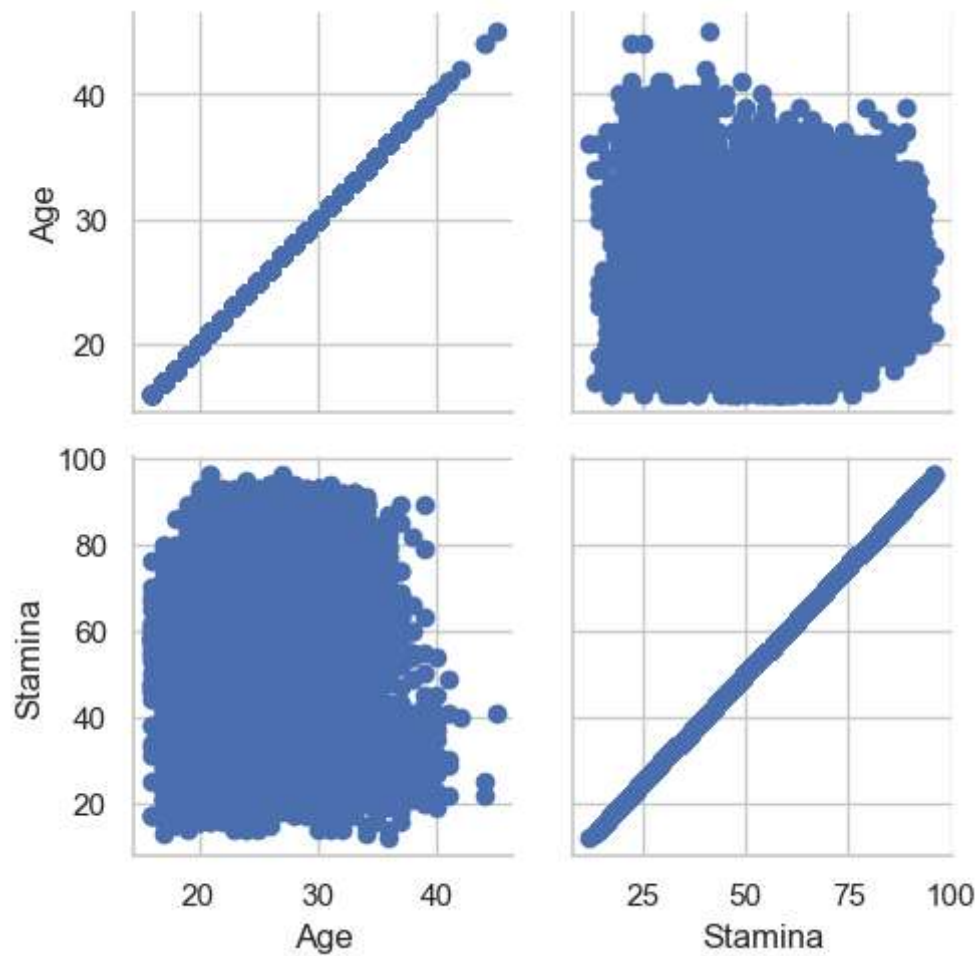
```
In [80]: g = sns.PairGrid(fifa19_new, hue="Preferred Foot")
g = g.map_diag(plt.hist)
g = g.map_offdiag(plt.scatter)
g = g.add_legend()
plt.show()
```



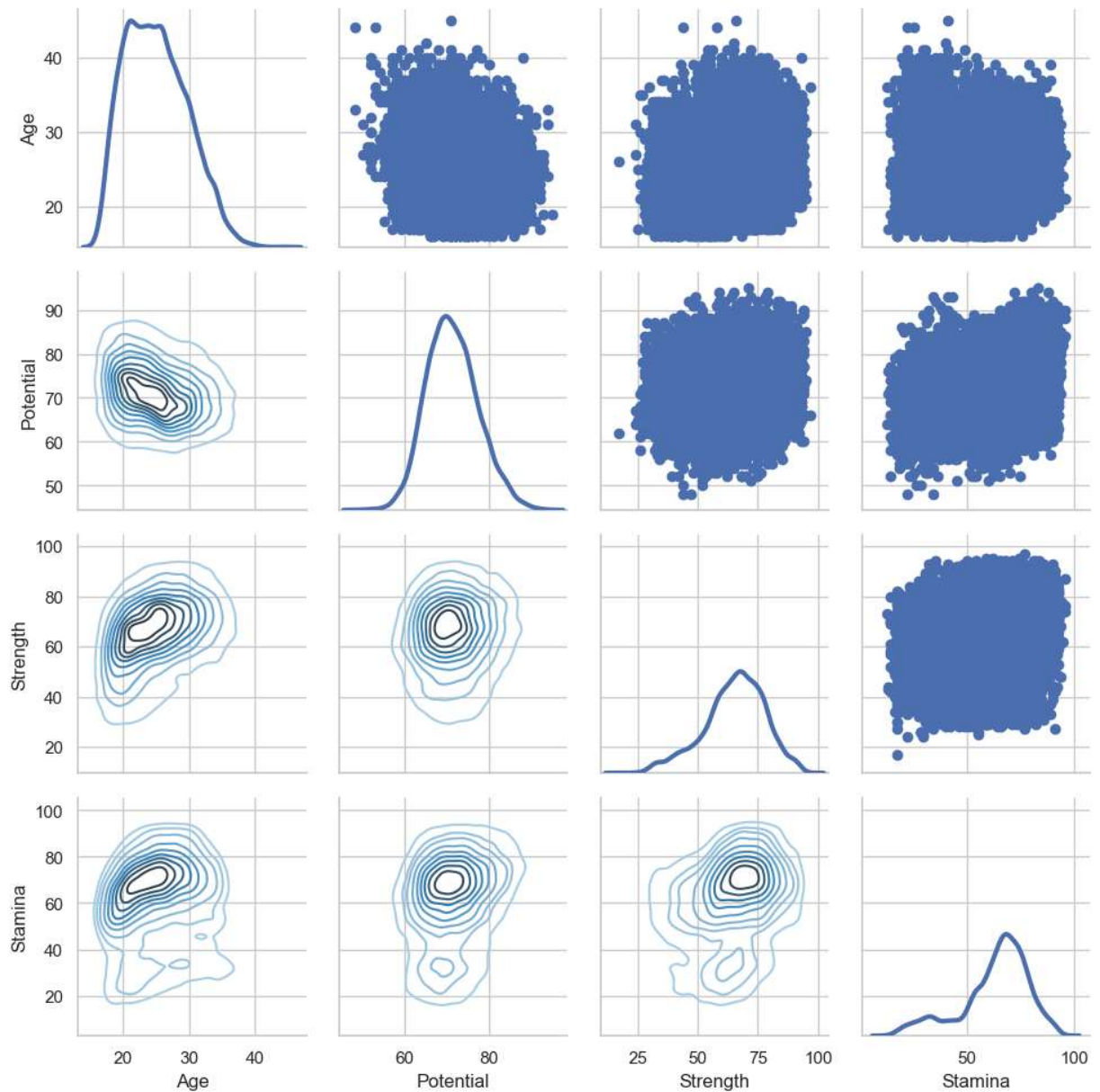
```
In [81]: g = sns.PairGrid(fifa19_new, hue="Preferred Foot")
g = g.map_diag(plt.hist, histtype="step", linewidth=3)
g = g.map_offdiag(plt.scatter)
g = g.add_legend()
plt.show()
```

```
In [82]: g = sns.PairGrid(fifa19_new, vars=['Age', 'Stamina'])
g = g.map(plt.scatter)
plt.show()
```



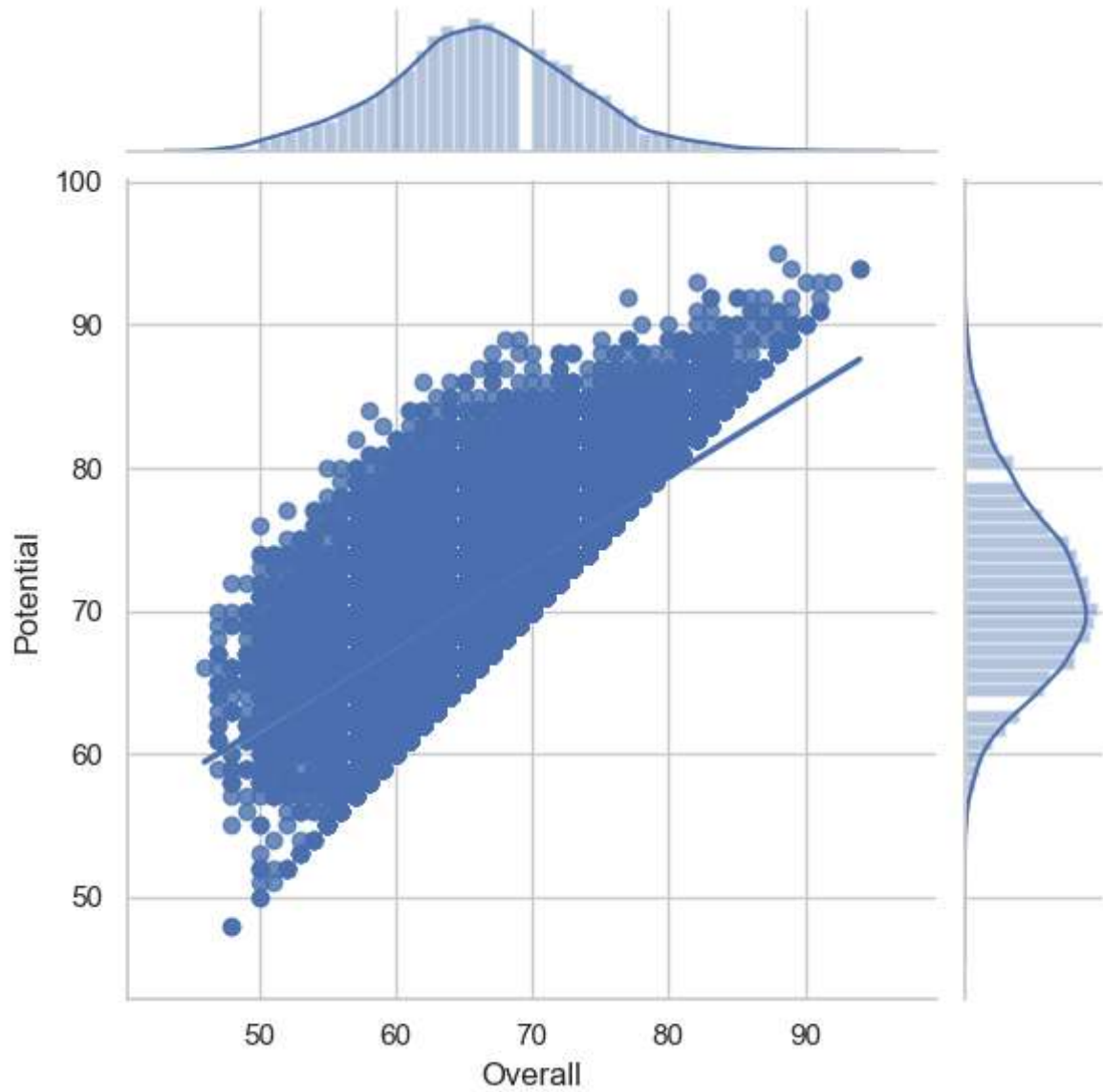
```
In [84]: g = sns.PairGrid(fifa19_new)
g = g.map_upper(plt.scatter)
g = g.map_lower(sns.kdeplot, cmap="Blues_d")
g = g.map_diag(sns.kdeplot, lw=3, legend=False)
plt.show()
```



Seaborn `JointGrid()` function

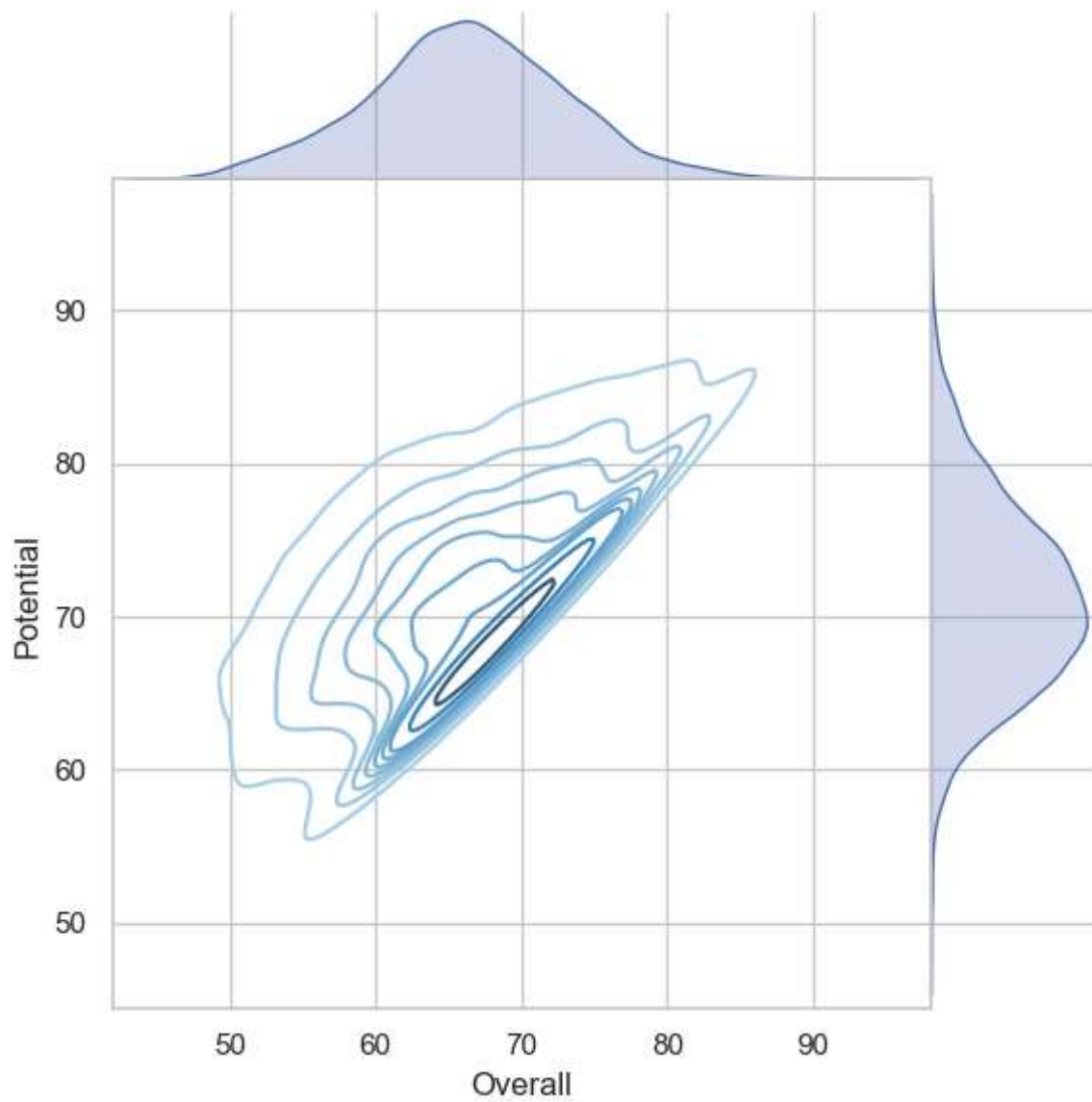
- This function provides a grid for drawing a bivariate plot with marginal univariate plots.
- It set up the grid of subplots.

```
In [86]: g = sns.JointGrid(x="Overall", y="Potential", data=fifa)
g = g.plot(sns.regplot, sns.distplot)
plt.show()
```



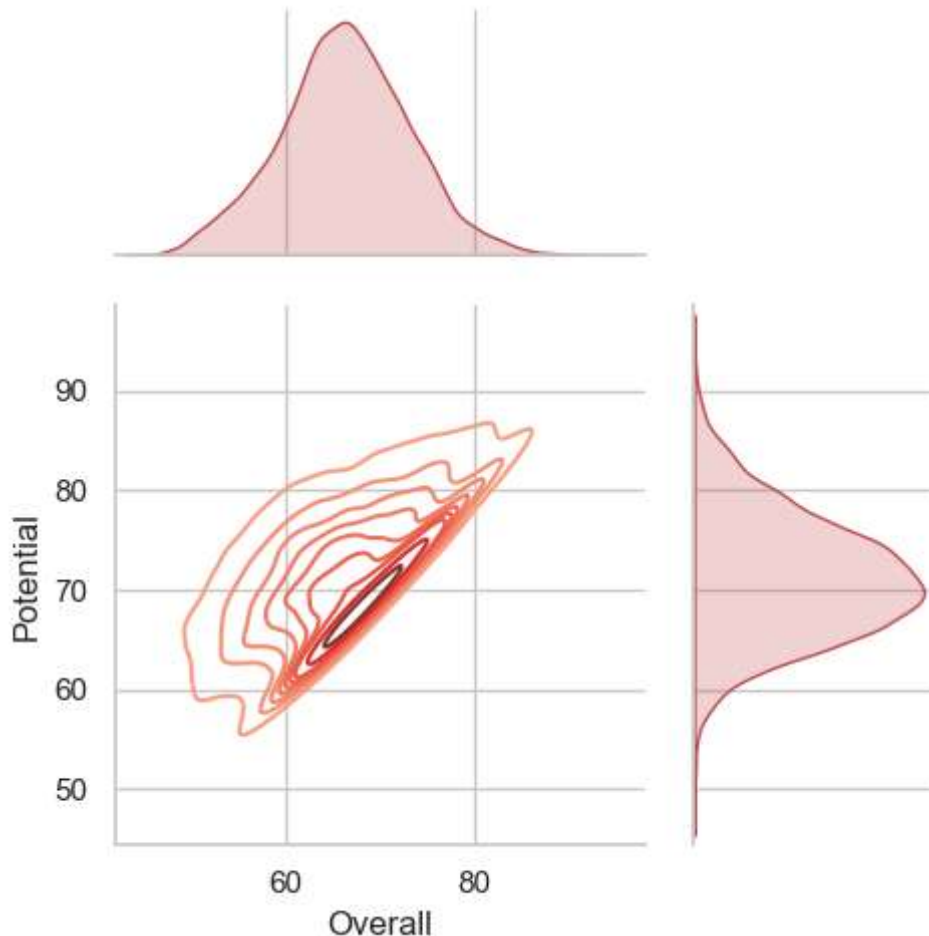
```
In [87]: g = sns.JointGrid(x="Overall", y="Potential", data=fifa, space=0)
g = g.plot_joint(sns.kdeplot, cmap="Blues_d")
g = g.plot_marginals(sns.kdeplot, shade=True)
```

```
In [88]: plt.show()
```



```
In [89]: g = sns.JointGrid(x="Overall", y="Potential", data=fifa, height=5, ratio=2)
g = g.plot_joint(sns.kdeplot, cmap="Reds_d")
g = g.plot_marginals(sns.kdeplot, color="r", shade=True)
```

```
In [90]: plt.show()
```

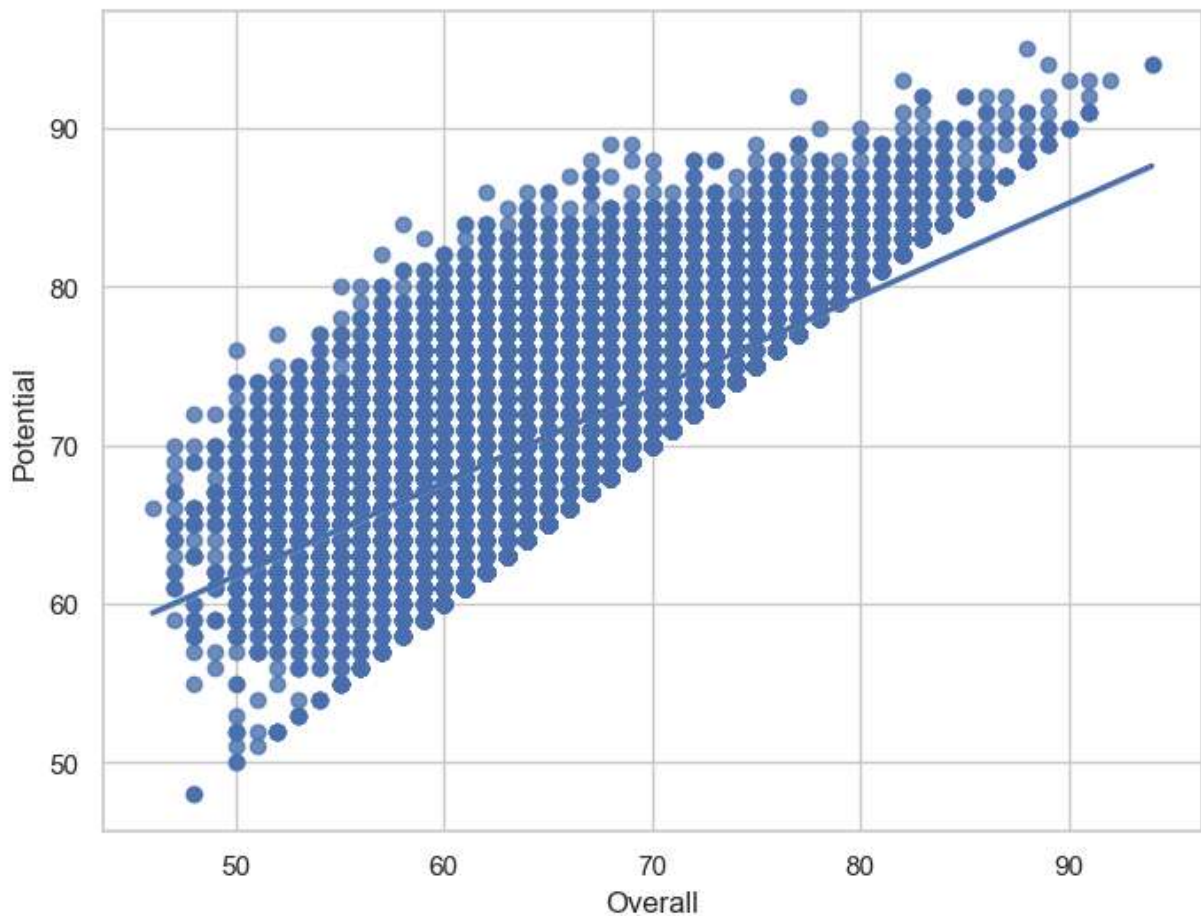


Controlling the size and shape of the plot

- The default plots made by `regplot()` and `lmpplot()` look the same but on axes that have a different size and shape.
- This is because `regplot()` is an “axes-level” function draws onto a specific axes.
- This means that you can make multi-panel figures yourself and control exactly where the regression plot goes.
- If no axes object is explicitly provided, it simply uses the “currently active” axes, which is why the default plot has the same size and shape as most other matplotlib functions.
- To control the size, we need to create a figure object ourselves as follows-

```
In [91]: f, ax = plt.subplots(figsize=(8, 6))  
ax = sns.regplot(x="Overall", y="Potential", data=fifa);
```

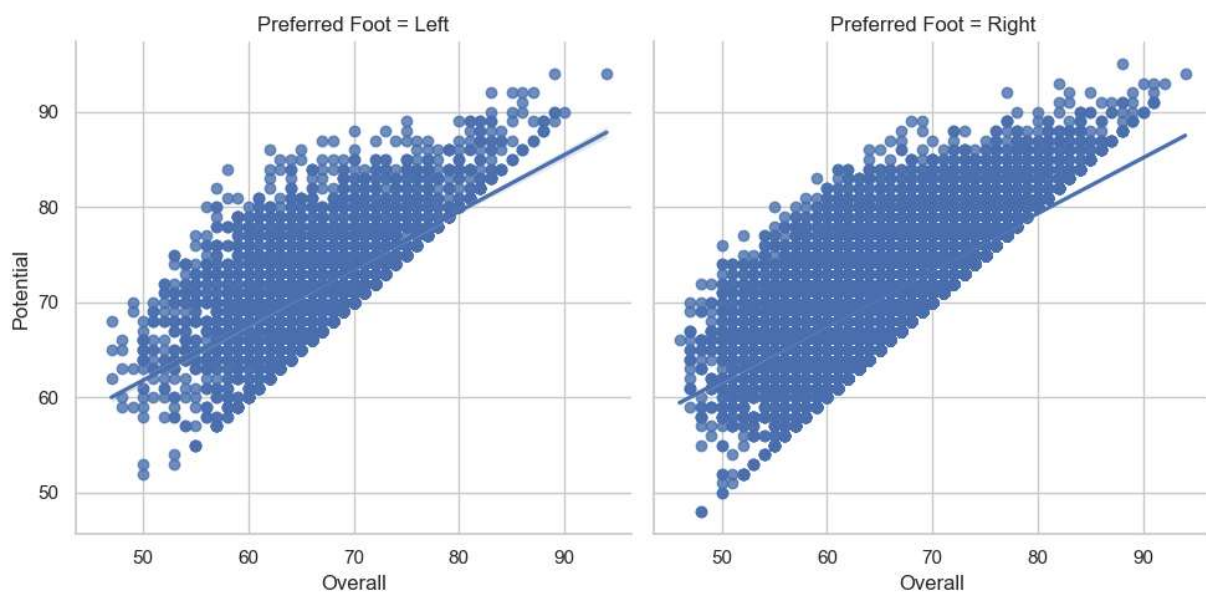
```
In [92]: plt.show()
```

```
In [93]: sns.lmplot(x="Overall", y="Potential", col="Preferred Foot", data=fifa, col_wrap=2,
```

```
Out[93]: <seaborn.axisgrid.FacetGrid at 0x25405b69f90>
```

```
In [94]: plt.show()
```



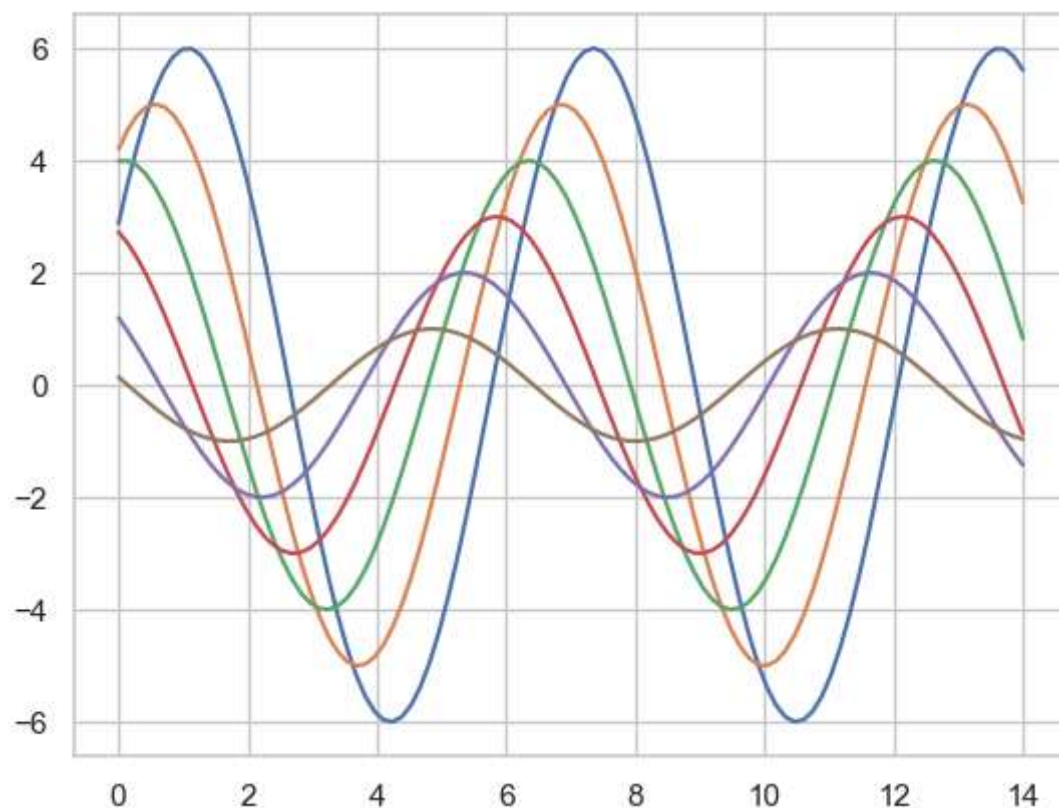
Seaborn figure styles

- There are five preset seaborn themes: `darkgrid`, `whitegrid`, `dark`, `white` and `ticks`.
- They are each suited to different applications and personal preferences.
- The default theme is `darkgrid`.
- The grid helps the plot serve as a lookup table for quantitative information, and the white-on grey helps to keep the grid from competing with lines that represent data.
- The `whitegrid` theme is similar, but it is better suited to plots with heavy data elements:

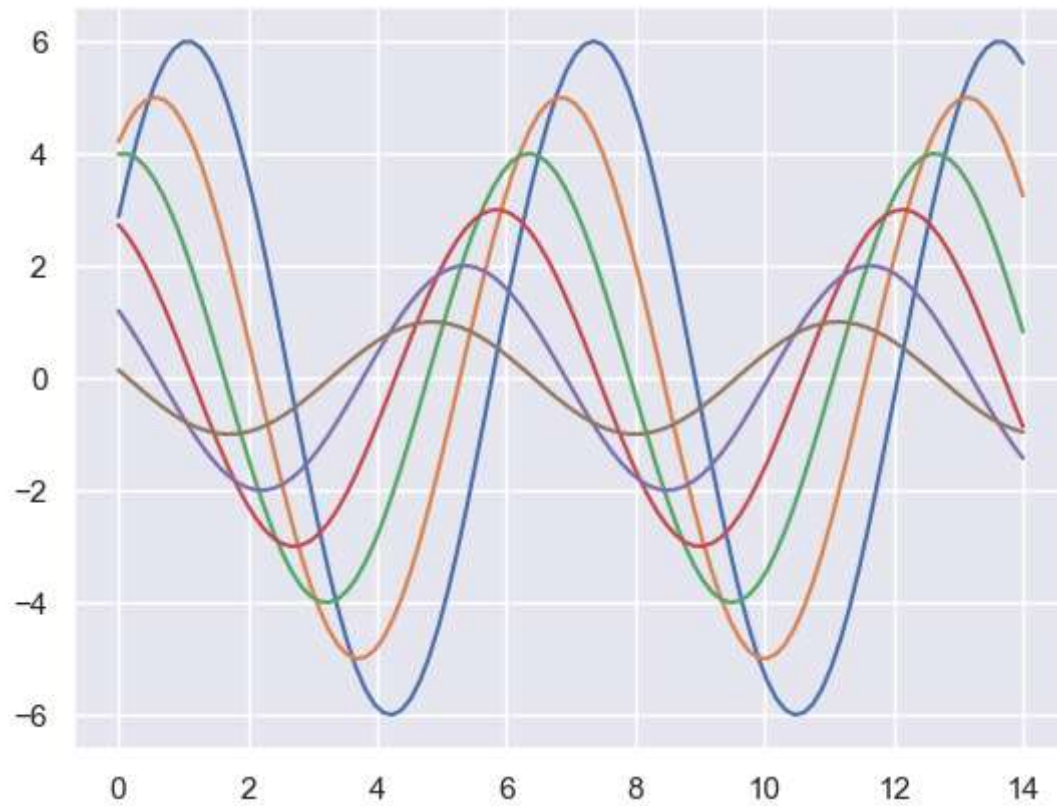
```
In [95]: def sinplot(flip=1):  
         x = np.linspace(0, 14, 100)  
         for i in range(1, 7):  
             plt.plot(x, np.sin(x + i * .5) * (7 - i) * flip)
```

```
In [97]: sinplot()
```

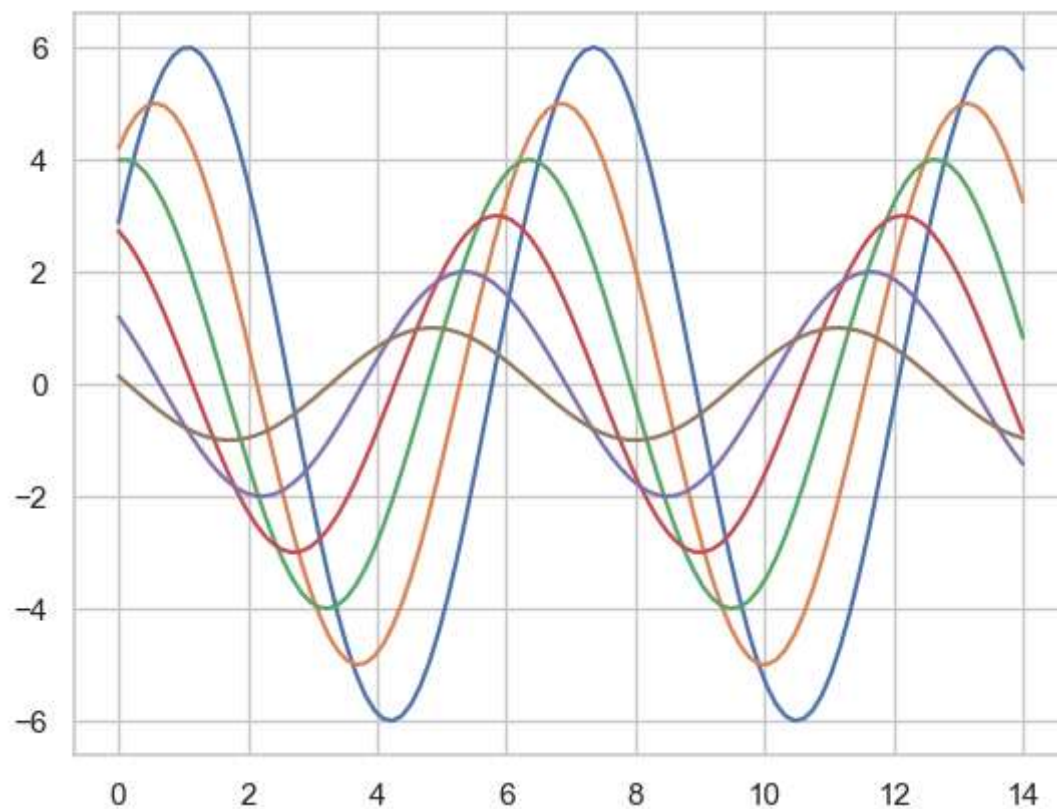
```
In [98]: plt.show()
```



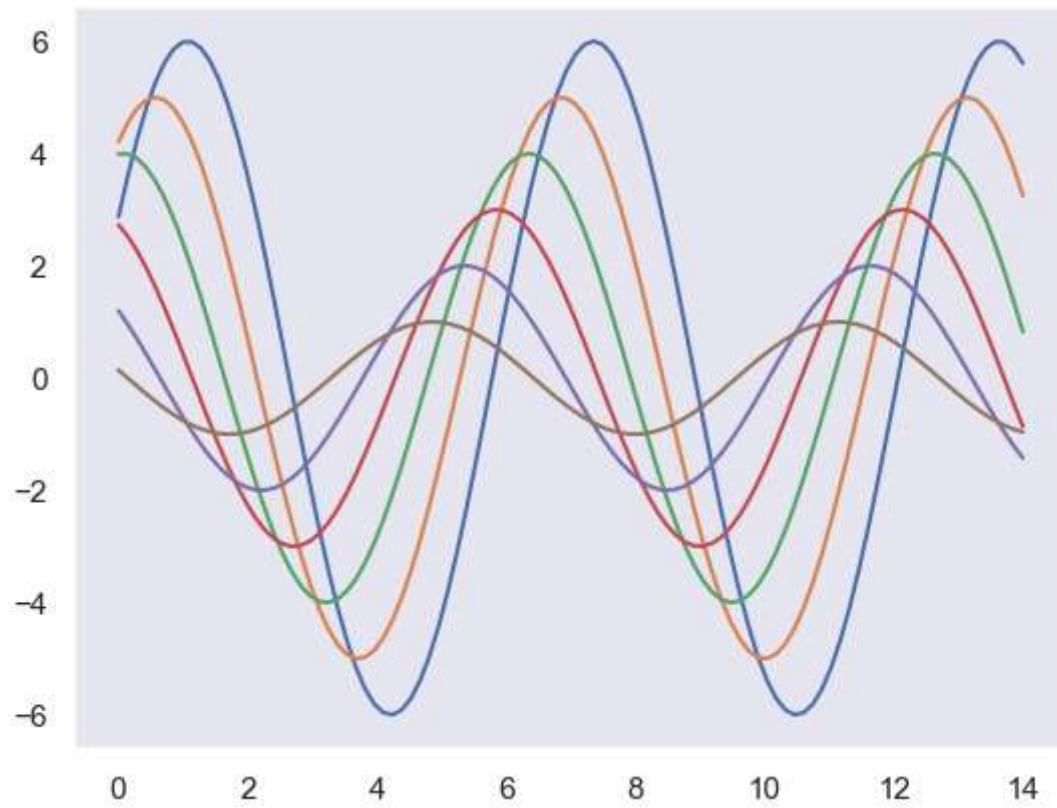
```
In [99]: sns.set()  
         sinplot()  
         plt.show()
```

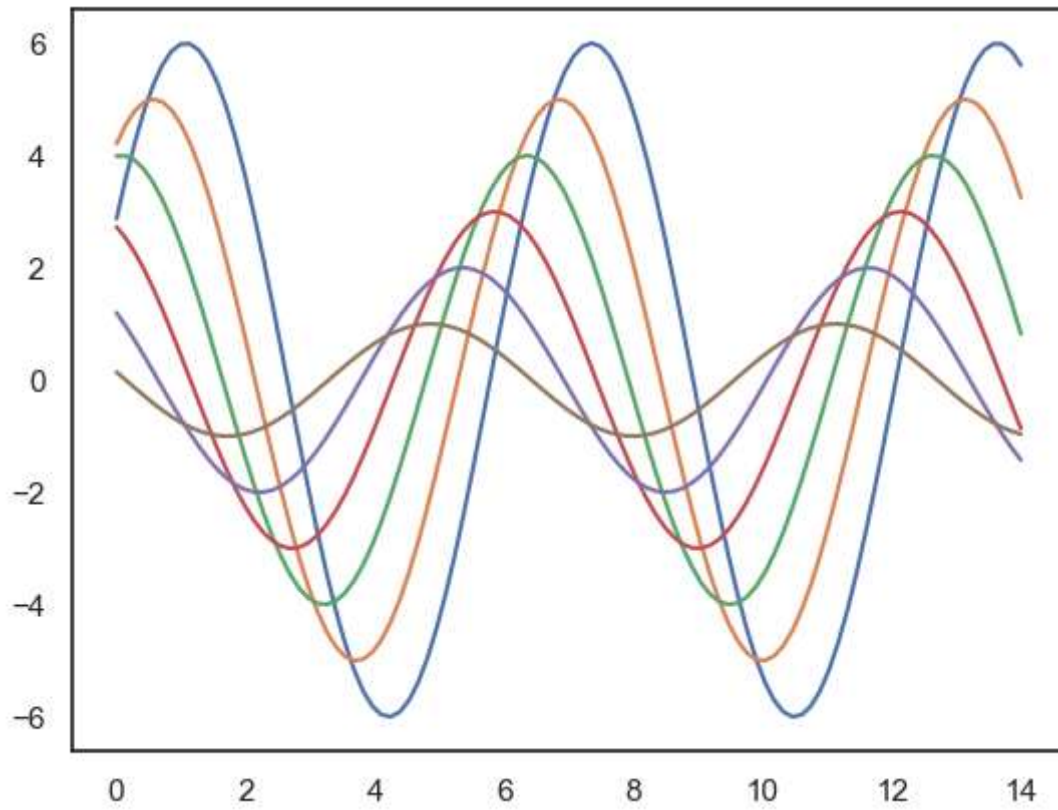
```
In [100... sns.set_style("whitegrid")  
sinplot()  
plt.show()
```



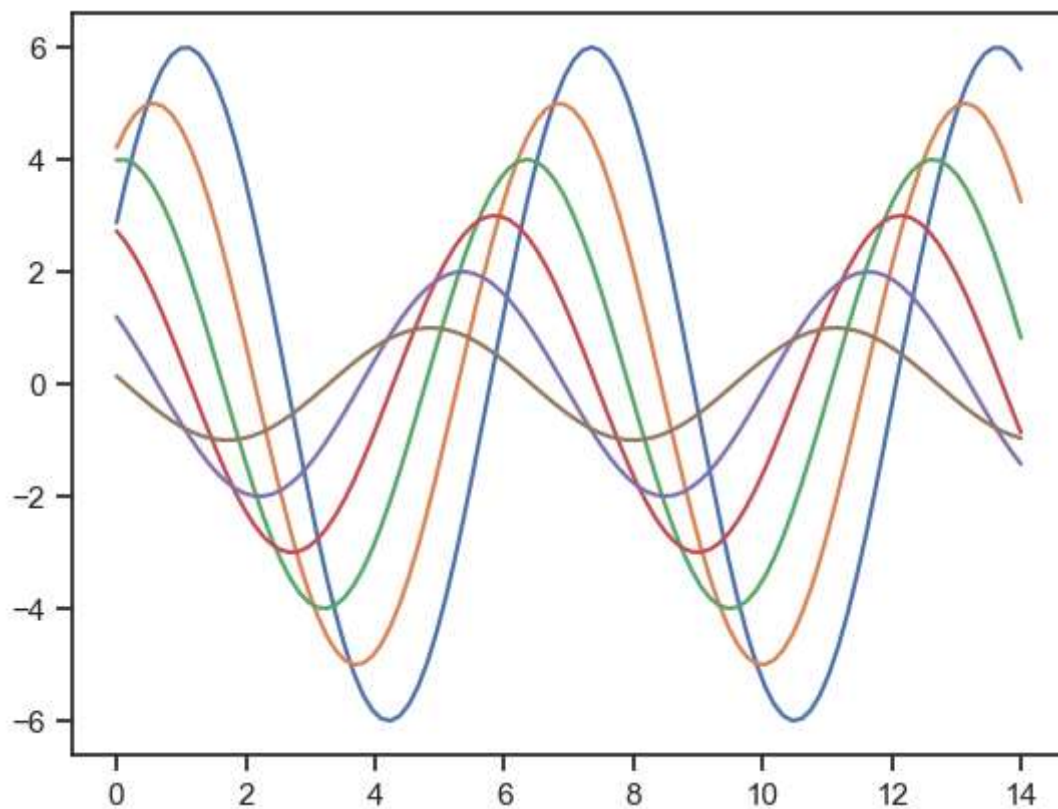
```
In [101... sns.set_style("dark")  
sinplot()  
plt.show()
```



```
In [102... sns.set_style("white")  
sinplot()  
plt.show()
```



```
In [103... sns.set_style("ticks")  
sinplot()  
plt.show()
```



In []: